



ระบบบริหารโครงการ และคิดค่าบริการแบบเครดิตในสภาพแวดล้อมหลายผู้ให้บริการคลาวด์
Project Management and Credit-Based Billing System in
Multi-Cloud Environments

นายฐิติภัทร ขุนรอง

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิทยาศาสตรบัณฑิต
ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2568

ปริญญานิพนธ์เรื่อง	:	ระบบบริหารโครงการ และคิดค่าบริการแบบเครดิต ในสภาพแวดล้อมหลายผู้ให้บริการคลาวด์ Project Management and Credit-Based Billing System in Multi-Cloud Environments
โดย	:	นายฐิติภัทร ชุนรอง
สาขาวิชา	:	วิทยาการคอมพิวเตอร์
ภาควิชา	:	วิทยาการคอมพิวเตอร์และสารสนเทศ
คณะ	:	วิทยาศาสตร์ประยุกต์
อาจารย์ที่ปรึกษา	:	ผู้ช่วยศาสตราจารย์ ดร.อภิสิทธิ์ รัตนตรานุรักษ์
ปีการศึกษา	:	2568

คณะวิทยาศาสตร์ประยุกต์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ อนุมัติให้
ปริญญานิพนธ์นี้ เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิทยาศาสตรบัณฑิต สาขาวิชา
วิทยาการคอมพิวเตอร์

.....ประธานกรรมการ
(ผู้ช่วยศาสตราจารย์ ดร.อภิสิทธิ์ รัตนตรานุรักษ์)

.....กรรมการ
(รองศาสตราจารย์ ดร. เฉียบบุฒิ รัตนวิไลสกุล)

.....กรรมการ
(ผู้ช่วยศาสตราจารย์ ดร.สรรร รัตนสัญญา)

.....หัวหน้าภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ
(รองศาสตราจารย์ ดร.ธนภัทร์ อนุศาสน์อมรกุล)

ลิขสิทธิ์ของภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2568

(ก)

บทคัดย่อ

บริษัท เมทราไบต์ คลาวด์ จำกัด เป็นผู้ให้บริการคลาวด์ (Infrastructure as a Service) โดยในปัจจุบันบริษัทใช้เทคโนโลยี OpenStack และ Proxmox ในการให้บริการแก่ลูกค้า อย่างไรก็ตาม กระบวนการจัดสรรและบริหารทรัพยากรในปัจจุบันยังขาดประสิทธิภาพ เนื่องจากผู้ใช้งานไม่สามารถจัดการทรัพยากรได้ด้วยตนเอง ต้องดำเนินการผ่านผู้ดูแลระบบเท่านั้น ส่งผลให้เกิดความล่าช้าในการให้บริการและเพิ่มภาระงานให้กับผู้ดูแลระบบ

โครงการนี้มีวัตถุประสงค์เพื่อพัฒนา Web Application แพลตฟอร์มบริหารจัดการโครงการสำหรับคลาวด์แบบบริการตนเอง ซึ่งรองรับทั้งฝั่งผู้ใช้ และฝั่งผู้ดูแลระบบ โดยฝั่งผู้ใช้สามารถสร้างและบริหารจัดการโครงการได้ด้วยตนเองผ่านบัญชีผู้ใช้เดียว พร้อมทั้งจัดการสมาชิกและสิทธิ์การเข้าถึงของสมาชิกภายในโครงการ สำหรับฝั่งผู้ดูแลระบบสามารถติดตามและจัดการโครงการของผู้ใช้ทั้งหมด รวมถึงจัดการสมาชิกและตรวจสอบการใช้งานได้อย่างครบวงจร

ระบบยังมีฟังก์ชันการคิดค่าบริการแบบเครดิต โดยลูกค้าสามารถเติมเงินเข้าสู่บัญชีเพื่อใช้จ่ายในแต่ละโครงการ ระบบจะบันทึกข้อมูลสเปกของทรัพยากรที่สร้างขึ้นจากแต่ละแพลตฟอร์มไว้ในฐานข้อมูล และเชื่อมต่อกับระบบหลังบ้านเพื่อตรวจสอบการใช้งานจริง นำมาคำนวณค่าใช้จ่ายและตัดยอดเครดิตโดยอัตโนมัติ ผู้ใช้สามารถตรวจสอบค่าใช้จ่ายแบบเรียลไทม์ แพลตฟอร์มนี้ช่วยลดภาระของผู้ดูแลระบบ เพิ่มความรวดเร็วในการให้บริการ และมอบความสะดวก โปร่งใส และมีประสิทธิภาพในการบริหารจัดการคลาวด์ให้กับผู้ใช้

กิตติกรรมประกาศ

การจัดทำปริญญานิพนธ์เรื่อง ระบบบริหารโครงการ และคิดค่าบริการแบบเครดิต ในสภาพแวดล้อมหลายผู้ให้บริการคลาวด์ นั้น สำเร็จลุล่วงไปด้วยดีด้วยการนำความรู้จากภาควิชา วิทยาการคอมพิวเตอร์ พร้อมทั้งความช่วยเหลือจาก ผู้ช่วยศาสตราจารย์ ดร.อภิสิทธิ์ รัตนตรานุรักษ์ ซึ่งเป็นอาจารย์ที่ปรึกษาปริญญานิพนธ์ ที่ได้ให้ความช่วยเหลือด้านข้อมูล และให้คำปรึกษา แนะนำ แนวทางในการแก้ปัญหาข้อผิดพลาดที่เกิดขึ้น

ขอขอบคุณพี่ๆ พนักงานและผู้บริหารบริษัท เมทราไบต์ คลาวด์ จำกัด ที่ให้ความช่วยเหลือด้านการพัฒนาระบบ และอำนวยความสะดวกเรื่องสถานที่ในการจัดทำโครงการ รวมถึงอาจารย์ภายใน ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

ในท้ายที่สุดนี้ ทางผู้จัดทำปริญญานิพนธ์ฉบับนี้ ขอขอบคุณทุกท่านที่ทำให้ปริญญานิพนธ์ฉบับนี้ สำเร็จลุล่วงไปด้วยดี ขอกราบขอบพระคุณบิดา มารดา ซึ่งสนับสนุนในด้านการเงินและให้กำลังใจ แก่ผู้วิจัยเสมอมาจนสำเร็จการศึกษา

ฐิติภัทร ขุนรอง

	สารบัญ	หน้า
เรื่อง		
บทคัดย่อ		ข
กิตติกรรมประกาศ		ค
สารบัญ		ง
สารบัญตาราง		ฉ
สารบัญรูปภาพ		ช
บทที่ 1 บทนำ		1
1.1 ความเป็นมา และความสำคัญของโครงการ		1
1.2 วัตถุประสงค์ของโครงการ		1
1.3 ขอบเขตโครงการ		2
1.4 ประโยชน์ที่คาดว่าจะได้รับ		2
บทที่ 2 ทฤษฎีที่เกี่ยวข้อง		3
2.1 NodeJS		3
2.2 React		4
2.3 Kotlin Spring Boot		4
2.4 MariaDB		5
2.5 Omise		7
บทที่ 3 ขั้นตอน และวิธีดำเนินงาน		8
3.1 ภาพรวมการทำงานของระบบ		8
3.2 Use Case Diagram		9
3.3 Use Case description		12
3.4 ER Diagram		28
3.5 Data Dictionary		29
บทที่ 4 การพัฒนาระบบ		44

4.1 เครื่องมือที่ใช้พัฒนาระบบ	44
4.2 การพัฒนาของโครงการ	45
บทที่ 5 ผลการดำเนินโครงการ	121
5.1 ผลการดำเนินงาน	121
5.2 การแสดงผลของ Web Application	121
บทที่ 6 บทสรุปและแนวทางในการพัฒนาต่อ	143
6.1 สรุปผลการดำเนินงาน	143
6.2 ปัญหาที่พบระหว่างการดำเนินงาน	143
6.3 แนวทางการแก้ไขปัญหา	143
6.4 แนวทางในการพัฒนาต่อ	144
บรรณานุกรม	145

สารบัญตาราง

ตารางที่	หน้า
ตารางที่ 3.1 Use Case description : Create Project	12
ตารางที่ 3.2 Use Case description: Update project	13
ตารางที่ 3.3 Use Case description: Delete project	14
ตารางที่ 3.4 Use Case description: Invite member	15
ตารางที่ 3.5 Use Case description: Assign role to member	16
ตารางที่ 3.6 Use Case description: Remove member from project	17
ตารางที่ 3.7 Use Case description: Create role	18
ตารางที่ 3.8 Use Case description: Update role permission	19
ตารางที่ 3.9 Use Case description: Delete role	20
ตารางที่ 3.10 Use Case description: View all user's project	21
ตารางที่ 3.11 Use Case description: Topup credit	22
ตารางที่ 3.12 Use Case description: Create tax address	23
ตารางที่ 3.13 Use Case description: Update tax address	24
ตารางที่ 3.14 Use Case description: Delete tax address	25
ตารางที่ 3.15 Use Case description: View all billing	26
ตารางที่ 3.16 Use Case description: View all user's topup history	27
ตารางที่ 3.17 Data Dictionary : accounts	29
ตารางที่ 3.18 Data Dictionary : user_profiles	30
ตารางที่ 3.19 Data Dictionary : tax_invoice	31
ตารางที่ 3.20 Data Dictionary : bills	32
ตารางที่ 3.21 Data Dictionary : payment_histories	33
ตารางที่ 3.22 Data Dictionary : projects	34
ตารางที่ 3.23 Data Dictionary : roles ข้อมูล Role	35
ตารางที่ 3.24 Data Dictionary : members	36
ตารางที่ 3.25 Data Dictionary : billing_history	37
ตารางที่ 3.26 Data Dictionary : credit_transaction	38
ตารางที่ 3.27 Data Dictionary : plans	39
ตารางที่ 3.28 Data Dictionary : resource_cost	40
ตารางที่ 3.29 Data Dictionary : instance_resources_usage	41

ตารางที่ 3.30 Data Dictionary : instances	42
ตารางที่ 3.31 Data Dictionary : volumes	43

สารบัญรูปภาพ

ภาพที่	หน้า
ภาพที่ 3.1 ภาพรวมการทำงานของระบบ	8
ภาพที่ 3.2 Use case diagram: Project Management Subsystem	9
ภาพที่ 3.3 Use case diagram: Billing Subsystem	10
ภาพที่ 3.4 ER Diagram	28
ภาพที่ 4.1 ฟังก์ชันสร้างโครงการ	45
ภาพที่ 4.2 ฟังก์ชันสร้างโครงการให้ผู้ใช้สิทธิ์ User	46
ภาพที่ 4.3 ฟังก์ชันการสร้างโครงการบนฐานข้อมูล	47
ภาพที่ 4.4 ฟังก์ชันการสร้างโครงการฝั่งของคลาวด์	48
ภาพที่ 4.5 ฟังก์ชันการสร้าง Role และเพิ่มสมาชิกเริ่มต้นในโครงการ	49
ภาพที่ 4.6 ฟังก์ชันการเรียก API สร้างบัญชีฝั่ง Openstack	50
ภาพที่ 4.7 ฟังก์ชันการเพิ่มสมาชิกเข้าโครงการฝั่ง Openstack	51
ภาพที่ 4.8 ฟังก์ชัน getMyProjects	52
ภาพที่ 4.9 ฟังก์ชันเปลี่ยนโครงการ	53
ภาพที่ 4.10 ฟังก์ชันเปลี่ยนโครงการ (ต่อ)	54
ภาพที่ 4.11 ฟังก์ชัน updateProjectRole	54
ภาพที่ 4.12 ฟังก์ชัน updateProject	55
ภาพที่ 4.13 ฟังก์ชัน deleteProject	56
ภาพที่ 4.14 ฟังก์ชัน deleteProject (ต่อ)	57
ภาพที่ 4.15 ฟังก์ชัน leaveProject	58
ภาพที่ 4.16 ฟังก์ชันเปลี่ยนเจ้าของโครงการ	59
ภาพที่ 4.17 ฟังก์ชัน getRoles	60
ภาพที่ 4.18 ฟังก์ชัน createRole	61
ภาพที่ 4.19 การตรวจสอบสิทธิ์ภายในโครงการ	62
ภาพที่ 4.20 การตรวจสอบสิทธิ์ภายในโครงการ (ต่อ)	63
ภาพที่ 4.21 ตัวอย่างการใช้งาน RequireProjectPermission	64
ภาพที่ 4.22 ฟังก์ชัน updateRole	65
ภาพที่ 4.23 ฟังก์ชัน deleteRole	66
ภาพที่ 4.24 ฟังก์ชัน deleteRole (ต่อ)	67
ภาพที่ 4.25 ฟังก์ชัน getMembers	68

ภาพที่ 4.26 ฟังก์ชัน invite	69
ภาพที่ 4.27 ฟังก์ชัน invite (ต่อ)	70
ภาพที่ 4.28 ฟังก์ชัน notifyInvite	71
ภาพที่ 4.29 ฟังก์ชัน cancelInvite	72
ภาพที่ 4.30 ฟังก์ชัน respondToInvitation	73
ภาพที่ 4.31 ฟังก์ชัน respondToInvitation (ต่อ)	74
ภาพที่ 4.32 ฟังก์ชัน respondToInvitation (ต่อ)	75
ภาพที่ 4.33 ฟังก์ชัน changeMemberRole	76
ภาพที่ 4.34 ฟังก์ชัน removeMember	77
ภาพที่ 4.35 ฟังก์ชัน removeMember(ต่อ)	78
ภาพที่ 4.36 ฟังก์ชัน getCurrentUsage	79
ภาพที่ 4.37 ฟังก์ชัน getCurrentUsage (ต่อ)	80
ภาพที่ 4.38 ฟังก์ชัน getProjectBilling	81
ภาพที่ 4.39 ฟังก์ชัน getProjectBilling (ต่อ)	82
ภาพที่ 4.40 ฟังก์ชัน getAdminBillingStats	83
ภาพที่ 4.41 ฟังก์ชัน getAdminBillingStats (ต่อ)	84
ภาพที่ 4.42 ฟังก์ชัน createToken	85
ภาพที่ 4.43 ฟังก์ชัน createCharge ฝั่งหน้าบ้าน	86
ภาพที่ 4.44 ฟังก์ชัน createCharge	87
ภาพที่ 4.45 ฟังก์ชัน handlePromptpayChage	88
ภาพที่ 4.46 ฟังก์ชัน handleCreditCardCharge	89
ภาพที่ 4.47 ฟังก์ชันเติมเงินสำหรับบัตรเครดิต	90
ภาพที่ 4.48 ฟังก์ชันเติมเงินสำหรับบัตรเครดิต (ต่อ)	91
ภาพที่ 4.49 ฟังก์ชันเติมเงินสำหรับพร้อมเพย์	92
ภาพที่ 4.50 ฟังก์ชันเติมเงินสำหรับพร้อมเพย์ (ต่อ)	93
ภาพที่ 4.51 ฟังก์ชัน startPolling	94
ภาพที่ 4.52 ฟังก์ชัน startPolling (ต่อ)	95
ภาพที่ 4.53 ฟังก์ชันดึงข้อมูลสถานะของการเติมเงิน	96
ภาพที่ 4.54 ฟังก์ชัน Webhook รับข้อมูล Omise	97
ภาพที่ 4.55 ฟังก์ชัน verifyAndGetEvent	97
ภาพที่ 4.56 ฟังก์ชัน processChargeEvent	98

ภาพที่ 4.57 ฟังก์ชัน processChargeEvent (ต่อ)	99
ภาพที่ 4.58 ฟังก์ชันขอข้อมูลบัตรเครดิตจาก Omise	100
ภาพที่ 4.59 ฟังก์ชันสร้างลูกค้ายอม Omise	101
ภาพที่ 4.60 ฟังก์ชันเพิ่มบัตรเครดิต	101
ภาพที่ 4.61 ฟังก์ชันลบบัตรเครดิต	102
ภาพที่ 4.62 ฟังก์ชันเก็บค่าใช้จ่ายตามเวลา	103
ภาพที่ 4.63 ฟังก์ชันเก็บค่าใช้จ่ายตามเวลา (ต่อ)	104
ภาพที่ 4.64 ฟังก์ชันเก็บค่าใช้จ่ายตามเวลา (ต่อ)	105
ภาพที่ 4.65 ฟังก์ชันเก็บค่าใช้จ่ายตามเวลา (ต่อ)	106
ภาพที่ 4.66 ฟังก์ชัน charge	106
ภาพที่ 4.67 ฟังก์ชัน charge (ต่อ)	107
ภาพที่ 4.68 ฟังก์ชันยกเลิกคลาวด์	108
ภาพที่ 4.69 ฟังก์ชันยกเลิกคลาวด์ (ต่อ)	108
ภาพที่ 4.70 ฟังก์ชันยกเลิกคลาวด์ (ต่อ)	109
ภาพที่ 4.71 ฟังก์ชันยกเลิกคลาวด์ (ต่อ)	110
ภาพที่ 4.72 ฟังก์ชันยกเลิกคลาวด์ (ต่อ)	111
ภาพที่ 4.73 ฟังก์ชันคำนวณราคาของ Volume	112
ภาพที่ 4.74 ฟังก์ชันคำนวณราคาของ Instance แบบ Custom	112
ภาพที่ 4.75 ฟังก์ชันสำหรับเปลี่ยนแพ็คเกจ Instance	113
ภาพที่ 4.76 ฟังก์ชันเปลี่ยนสเปคของ Instance แบบ Custom	114
ภาพที่ 4.77 ฟังก์ชันเปลี่ยนขนาดพื้นที่เก็บข้อมูล Volume	115
ภาพที่ 4.78 ฟังก์ชันส่งข้อมูลผู้เสียภาษี	116
ภาพที่ 4.79 ฟังก์ชันส่งข้อมูลที่อยู่ภาษีหลัก	116
ภาพที่ 4.80 ฟังก์ชันลบที่อยู่ผู้เสียภาษี	117
ภาพที่ 4.81 ฟังก์ชันกำหนดที่อยู่เริ่มต้น	118
ภาพที่ 4.82 ฟังก์ชันแก้ไขที่อยู่ผู้เสียภาษี	119
ภาพที่ 4.83 ฟังก์ชันสร้างที่อยู่ผู้เสียภาษี	120
ภาพที่ 5.1 หน้าเลือกโครงการ	121
ภาพที่ 5.2 Modal ตอบรับคำเชิญเข้าร่วมโครงการ	122
ภาพที่ 5.3 Modal สร้างโครงการ	122
ภาพที่ 5.4 หน้าแรกของโครงการ	123

ภาพที่ 5.5 Modal แก้ไขข้อมูลโครงการ	123
ภาพที่ 5.6 ปุ่มลบโครงการ	124
ภาพที่ 5.7 ปุ่มออกจากโครงการ	124
ภาพที่ 5.8 หน้าจัดการสมาชิกในโครงการ	124
ภาพที่ 5.9 Modal เปลี่ยน Role สมาชิก	125
ภาพที่ 5.10 ยืนยันการลบสมาชิกออกจากโครงการ	125
ภาพที่ 5.11 Modal เชิญสมาชิกเข้าร่วมโครงการ	126
ภาพที่ 5.12 Modal เปลี่ยนเจ้าของโครงการ	126
ภาพที่ 5.13 สมาชิกที่รอการตอบรับคำเชิญ	127
ภาพที่ 5.14 หน้าจัดการ Role ในโครงการ	127
ภาพที่ 5.15 หน้าการสร้าง Role ใหม่	128
ภาพที่ 5.16 ยืนยันการลบ Role	128
ภาพที่ 5.17 หน้าดูค่าใช้จ่ายในโครงการปัจจุบัน	129
ภาพที่ 5.18 ตารางค่าใช้จ่ายทั้งหมดตั้งแต่สร้างโครงการ	129
ภาพที่ 5.19 หน้าการเงินของบัญชีผู้ใช้	130
ภาพที่ 5.20 ตารางธุรกรรมเครดิต	130
ภาพที่ 5.21 Modal เพิ่มเครดิต	131
ภาพที่ 5.22 Modal เลือกช่องทางการจ่ายเงิน	131
ภาพที่ 5.23 เลือกที่อยู่ผู้เสียภาษี	132
ภาพที่ 5.24 เลือกจ่ายด้วยพร้อมเพย์	132
ภาพที่ 5.25 Modal สแกนจ่ายพร้อมเพย์	133
ภาพที่ 5.26 เลือกจ่ายด้วยบัตรเครดิต	133
ภาพที่ 5.27 Modal เพิ่มบัตรเครดิตใหม่	134
ภาพที่ 5.28 Modal แก้ไขบัตรเครดิตเก่า	134
ภาพที่ 5.29 Modal สร้างที่อยู่ผู้เสียภาษี	135
ภาพที่ 5.30 Modal บันทึกที่อยู่ผู้เสียภาษี	135
ภาพที่ 5.31 หน้ารวมการใช้จ่ายทั้งระบบ	136
ภาพที่ 5.32 ตารางค่าใช้จ่ายจัดกลุ่มตามโครงการ	136
ภาพที่ 5.33 หน้าดูค่าใช้จ่ายในโครงการ	137
ภาพที่ 5.34 หน้าจัดการโครงการในระบบ	137
ภาพที่ 5.35 ตารางโครงการในระบบ	138

ภาพที่ 5.36 Modal สร้างโครงการให้ผู้ใช้	138
ภาพที่ 5.37 Modal แก้ไขโครงการของผู้ใช้	139
ภาพที่ 5.38 หน้าผู้ดูแลเข้าสู่โครงการ	139
ภาพที่ 5.39 หน้าจัดการสมาชิกในโครงการ	140
ภาพที่ 5.40 หน้าจัดการ Role ของโครงการ	140
ภาพที่ 5.41 Modal เพิ่มสมาชิกโครงการ	141
ภาพที่ 5.42 หน้าประวัติการเติมเครดิต	141
ภาพที่ 5.43 หน้าธุรกรรมเครดิต	142
ภาพที่ 5.44 หน้าดูใบเสร็จต่างๆ	142

บทที่ 1

บทนำ

1.1 ความเป็นมา และความสำคัญของโครงการ

บริษัท เมทราไบต์ คลาวด์ จำกัด เป็นผู้ให้บริการคลาวด์ที่มีโครงสร้างพื้นฐานหลักสองระบบ ได้แก่ OpenStack และ Proxmox เพื่อให้บริการทรัพยากรคอมพิวเตอร์แก่ลูกค้า อย่างไรก็ตาม กระบวนการให้บริการในปัจจุบันยังขาดระบบบริการตนเอง ลูกค้าจำเป็นต้องติดต่อผู้ดูแลระบบโดยตรงเพื่อแจ้งความต้องการและดำเนินการชำระเงิน จากนั้นผู้ดูแลระบบจะเป็นผู้จัดเตรียมทรัพยากรและส่งข้อมูลการใช้งาน เช่น ที่อยู่ไอพี ให้กับลูกค้า กระบวนการที่ดำเนินการด้วยตนเอง เช่นนี้ก่อให้เกิดปัญหาข้อขัด ทำให้เกิดความล่าช้าในการให้บริการ และเพิ่มภาระงานให้กับผู้ดูแลระบบ

เพื่อปรับปรุงกระบวนการให้บริการและเพิ่มขีดความสามารถในการแข่งขันขององค์กร โครงการนี้จึงมุ่งพัฒนาระบบ Portal บริการตนเองสำหรับคลาวด์ขึ้น ระบบนี้รองรับทั้ง ผู้ใช้ และ ผู้ดูแลระบบ โดยผู้ใช้งานสามารถสร้างโครงการของตนเอง บริหารจัดการสมาชิกในโครงการ กำหนดสิทธิ์การเข้าถึง และตรวจสอบค่าใช้จ่ายได้ด้วยตนเอง ส่วนผู้ดูแลระบบสามารถติดตามและจัดการโครงการทั้งหมดของผู้ใช้ ตรวจสอบสมาชิกและค่าใช้จ่ายได้ด้วยระบบในฝั่งของผู้ดูแลระบบ นอกจากนี้ ผู้ดูแลระบบยังสามารถดูภาพรวมการใช้งานและค่าใช้จ่ายทั้งหมดของระบบผ่าน Data Visualization ทำให้เห็นแนวโน้มการใช้ทรัพยากรและค่าใช้จ่ายในแต่ละเดือน เพื่อสนับสนุนการวางแผนและตัดสินใจเชิงบริหารได้อย่างมีประสิทธิภาพ

ระบบยังมีฟังก์ชันการคิดค่าบริการแบบเครดิต โดยลูกค้าสามารถเติมเงินเข้าสู่บัญชีเพื่อใช้จ่ายภายในแต่ละโครงการ ข้อมูลสเปกของทรัพยากรจากแต่ละแพลตฟอร์มจะถูกบันทึกไว้ในฐานข้อมูลเพื่อนำมาคำนวณค่าใช้จ่าย ระบบจะเชื่อมต่อกับระบบหลังบ้านเพื่อตรวจสอบการใช้งานจริงและตัดยอดเครดิตโดยอัตโนมัติ ทำให้ผู้ใช้งานสามารถติดตามค่าใช้จ่ายแบบเรียลไทม์ ระบบนี้ช่วยลดภาระผู้ดูแลระบบ เพิ่มความรวดเร็วในการให้บริการ และมอบความสะดวก โปร่งใส และมีประสิทธิภาพในการบริหารจัดการคลาวด์ให้กับผู้ใช้

1.2 วัตถุประสงค์ของโครงการ

- 1.2.1 เพื่อพัฒนาระบบที่ให้ผู้ใช้งานจัดการโครงการของคลาวด์ รวมทั้งจัดการสมาชิกในโครงการได้ด้วยตนเอง
- 1.2.2 เพื่อพัฒนาระบบเติมเครดิตให้ผู้ใช้งานนำมาซื้อบริการคลาวด์ได้
- 1.2.3 เพื่อพัฒนาระบบคิดค่าบริการสำหรับคลาวด์ Openstack และ Proxmox ได้

1.3 ขอบเขตโครงการ

1.3.1 ส่วนของผู้ใช้

- 1.3.1.1 สามารถสร้างโครงการสำหรับ Openstack และ Proxmox ได้
- 1.3.1.2 สามารถเพิ่ม แก้ไข ลบ สมาชิกภายในโครงการได้
- 1.3.1.3 สามารถแก้ไขสิทธิ์การจัดการส่วนต่างๆในโครงการให้สมาชิกได้
- 1.3.1.4 สามารถเติมเงินเป็นเครดิตสำหรับใช้จ่ายในระบบ พร้อมออกใบเสร็จ/ใบกำกับภาษีได้
- 1.3.1.5 สามารถเพิ่ม แก้ไข ลบ ที่อยู่ผู้เสียภาษีได้
- 1.3.1.6 หน้าสรุปรวมค่าใช้จ่ายภายในโครงการ

1.3.2 ส่วนของผู้ดูแลระบบ

- 1.3.2.1 สามารถดู สร้าง แก้ไข ลบโครงการของผู้ใช้ในระบบได้
- 1.3.2.2 สามารถดู สร้าง แก้ไข ลบสมาชิกภายในโครงการได้
- 1.3.2.3 หน้า Table แสดงรายการค่าใช้จ่ายของแต่ละโครงการ
- 1.3.2.4 หน้า Data Visualization แสดงข้อมูลสถิติการใช้จ่ายของโครงการทั้งหมดในระบบ

1.3.3 ส่วนของระบบ

- 1.3.3.1 การคิดค่าบริการคลาวด์ทั้งหมดในระบบตามเวลาที่กำหนด

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- 1.4.1 ลดความซับซ้อนและประหยัดเวลาในการบริหารคลาวด์แบบหลายผู้ให้บริการ
- 1.4.2 เพิ่มความคล่องตัวในการบริหารจัดการสมาชิกในโครงการ
- 1.4.3 เพิ่มความสะดวกและโปร่งใสให้ผู้ใช้ในการบริหารค่าใช้จ่ายคลาวด์
- 1.4.4 ช่วยให้ผู้ดูแลระบบสามารถบริหารจัดการโครงการและสมาชิกของผู้ใช้ได้อย่างสะดวกและมีประสิทธิภาพ

บทที่ 2

ทฤษฎีที่เกี่ยวข้อง

2.1 NodeJS

Node.js คือ JavaScript Runtime Environment แบบโอเพนซอร์ส (Open Source) ที่ทำงานบน V8 Engine ซึ่งเป็นเอนจินที่พัฒนาโดย The Chromium Project เพื่อใช้ในเว็บเบราว์เซอร์ Google Chrome หัวใจสำคัญของ Node.js คือการทำให้ภาษา JavaScript ซึ่งเดิมที่ถูกจำกัดให้ทำงานได้เฉพาะบนฝั่งไคลเอนต์ (Client-side) หรือบนเว็บเบราว์เซอร์ สามารถทำงานบนฝั่งเซิร์ฟเวอร์ (Server-side) ได้

หลักการทำงานที่สำคัญคือ V8 Engine จะทำการคอมไพล์ (Compile) โค้ด JavaScript ให้กลายเป็น Machine Code ก่อนการประมวลผล ทำให้ Node.js มีประสิทธิภาพสูงและสามารถทำงานข้ามแพลตฟอร์ม (Cross-platform) ได้ทั้งบนระบบปฏิบัติการ Windows, Linux และ macOS

จุดเด่นที่ทำให้ Node.js ได้รับความนิยมอย่างแพร่หลายคือสถาปัตยกรรมแบบ Non-blocking I/O และ Event-driven ซึ่งทำให้มันสามารถจัดการกับการเชื่อมต่อจำนวนมากพร้อมกันได้อย่างมีประสิทธิภาพโดยใช้ทรัพยากรน้อย เหมาะอย่างยิ่งสำหรับแอปพลิเคชันที่ต้องการประมวลผลข้อมูลแบบเรียลไทม์ (Real-time) หรือมีการรับ-ส่งข้อมูลเกิดขึ้นบ่อยครั้ง

นอกจากนี้ Node.js ยังมีระบบนิเวศ (Ecosystem) ที่แข็งแกร่ง โดยมีเครื่องมือจัดการแพ็คเกจ (Package Manager) ที่ทรงพลังอย่าง NPM (Node Package Manager) และ YARN เป็นศูนย์กลางในการเข้าถึงไลบรารีและเฟรมเวิร์กจำนวนมากมหาศาล เช่น Express.js Next.js และอื่นๆ ซึ่งช่วยลดระยะเวลาและเพิ่มประสิทธิภาพในการพัฒนาได้อย่างมาก

2.1.1 ประโยชน์และข้อดีที่สำคัญของ Node.js

2.1.1.1 ประสิทธิภาพสูง ด้วยสถาปัตยกรรมแบบ Non-blocking I/O ทำให้สามารถรองรับการทำงานพร้อมกันจำนวนมาก (High Concurrency) ได้ดี เหมาะสำหรับแอปพลิเคชันที่ต้องการการตอบสนองที่รวดเร็ว

2.1.1.2 ใช้ภาษาเดียวทั้ง Frontend และ Backend นักพัฒนาสามารถใช้ภาษา JavaScript ในการพัฒนาทั้งส่วนหน้าบ้านและหลังบ้าน (Full-stack Development) ทำให้การพัฒนาเป็นไปอย่างราบรื่นและง่ายต่อการดูแลรักษา

2.1.1.3 ระบบนิเวศขนาดใหญ่ มีไลบรารีและเครื่องมือสำเร็จรูปจำนวนมากผ่าน NPM ช่วยให้นักพัฒนาสามารถนำโค้ดมาใช้อย่างยืดหยุ่นได้ทันทีโดยไม่ต้องสร้างทุกอย่างขึ้นมาใหม่

การใช้ทรัพยากรอย่างมีประสิทธิภาพ โมเดลการทำงานแบบ Single-threaded และ Event-driven ทำให้ใช้หน่วยความจำและ CPU น้อยกว่าเมื่อเทียบกับสถาปัตยกรรมแบบ Multi-threaded แบบดั้งเดิมในการจัดการกับผู้ใช้จำนวนเท่ากัน

2.2 React

React เป็น Library ของ JavaScript ที่พัฒนาโดย Meta หรือผู้พัฒนา Facebook เพื่อช่วยสร้าง User Interface ที่มีประสิทธิภาพ มุ่งเน้นไปที่การสร้าง Component ที่สามารถใช้ซ้ำได้ โดยที่ภายในแต่ละ Component จะสามารถเก็บสถานะ (State) และเมทอด (Method) ต่างๆ ในการทำงาน และแสดงผลออกมา

เว็บไซต์ในอดีต การที่ผู้ใช้มีปฏิสัมพันธ์ใดๆ กับระบบจะต้องมีการโหลดทั้งหน้าใหม่ทุกครั้งเพื่อเปลี่ยนสีของปุ่ม ซึ่งส่งผลเสียทั้งในแง่ของระยะเวลาในการแสดงผล และประสบการณ์ของผู้ใช้งาน React จึงถูกพัฒนาขึ้นเพื่อแก้ปัญหาดังกล่าวให้สามารถแสดงผลการเปลี่ยนแปลงหน้าเว็บไซต์ได้ทันทีเมื่อเกิดคำสั่งใหม่บนระบบโดยไม่ต้องโหลดหน้าเว็บใหม่อีกครั้ง ส่วนประกอบหลักของ React มีทั้งหมด 3 ส่วน

2.2.1 โค้ดที่แบ่งออกเป็นบล็อกเล็กๆ หลายอัน (Component) โดย 1 อันจะแทนที่ชิ้นส่วนหนึ่งของ UI บนหน้าเว็บ ซึ่งสามารถนำมาใช้ซ้ำได้ตลอดการสร้างเว็บ Component จึงเป็นส่วนที่ทำให้หน้าเว็บทำงานอิสระต่อกัน

2.2.2 JSX เป็นส่วนขยายสำหรับการเขียนโค้ดภาษา JavaScript ทำให้นักพัฒนาสามารถเขียนโค้ดของ Component ด้วยภาษา HTML จากนั้นระบบจะแปลงเป็น JavaScript เพื่อสร้างการทำงานเอง

2.2.3 Props ใช้สำหรับการส่งข้อมูลระหว่าง Component เพื่อใช้ในการแสดงผลบน Component ใหม่ที่มีการทำงานแตกต่างกัน แต่ยังใช้ข้อมูลชุดเดียวกันอยู่ เช่น ปุ่มกดที่มีหน้าตาเหมือนกัน แต่ข้อความที่แสดงผล และผลลัพธ์ของการกดต่างกัน Props จึงทำหน้าที่รับข้อความ และการทำงานเพื่อมาแสดงผลบนปุ่มนั้น

React เหมาะสำหรับการพัฒนาแอปพลิเคชันเว็บขนาดใหญ่หรือซับซ้อน เว็บไซต์ที่มีการจัดการข้อมูลแบบเรียลไทม์ เช่น เว็บไซต์ขายของออนไลน์ เว็บไซต์โซเชียลมีเดีย เกม เป็นต้น

2.3 Kotlin Spring Boot

Kotlin คือภาษาโปรแกรมสมัยใหม่ที่ทำงานบน Java Virtual Machine (JVM) ได้รับการออกแบบมาเพื่อแก้ไขปัญหาและลดความซับซ้อนของโค้ดที่พบได้บ่อยในภาษา Java ทำให้โค้ดที่ได้มีความกระชับ (Concise) ปลอดภัยจาก Null Pointer Exceptions และสามารถทำงานร่วมกับโค้ดและไลบรารีของ Java เดิมได้อย่างสมบูรณ์ 100% ด้วยเหตุนี้ Kotlin จึงกลายเป็นภาษาที่ Google

ประกาศให้การสนับสนุนอย่างเป็นทางการสำหรับการพัฒนาแอปพลิเคชัน Android และได้รับความนิยมเพิ่มขึ้นอย่างรวดเร็วในการพัฒนาฝั่งเซิร์ฟเวอร์ (Backend)

Spring Boot คือเฟรมเวิร์ก (Framework) ที่พัฒนาต่อยอดมาจาก Spring Framework ซึ่งเป็นที่นิยมอย่างสูงใน Ecosystem ของ Java โดยมีเป้าหมายเพื่อลดความซับซ้อนในการตั้งค่าเริ่มต้นของโปรเจกต์ และช่วยให้นักพัฒนาสามารถสร้างแอปพลิเคชันที่พร้อมใช้งาน (Stand-alone, Production-grade) ได้อย่างรวดเร็วที่สุด

เมื่อนำ Kotlin มาใช้ร่วมกับ Spring Boot จะเป็นการผสานจุดแข็งของทั้งสองเทคโนโลยีเข้าด้วยกัน คือความสามารถในการเขียนโค้ดที่สั้นกระชับและปลอดภัยของ Kotlin เข้ากับความแข็งแกร่งและระบบนิเวศที่ครบครันของ Spring Boot ทำให้การพัฒนาแอปพลิเคชันมีประสิทธิภาพสูงและดูแลรักษาง่าย โดย Spring Boot มีข้อดีที่สำคัญดังนี้

2.3.1 การพัฒนาที่รวดเร็ว (Fast Development) ช่วยให้นักพัฒนาสามารถสร้างแอปพลิเคชันที่พร้อมนำไปใช้งานจริงได้ทันที โดยลดขั้นตอนการตั้งค่าที่ย่างยากซับซ้อนลง

2.3.2 การตั้งค่าอัตโนมัติ (Auto-Configuration) Spring Boot จะพยายามตั้งค่าที่จำเป็นส่วนใหญ่ให้โดยอัตโนมัติตามไลบรารีที่อยู่ในโปรเจกต์ ทำให้นักพัฒนาสามารถมุ่งเน้นไปที่การพัฒนาส่วนของตรรกะทางธุรกิจ (Business Logic) ได้อย่างเต็มที่

2.3.3 รองรับสถาปัตยกรรมไมโครเซอร์วิส (Microservices Friendly) ด้วยความสามารถในการสร้างแอปพลิเคชันที่ทำงานได้ด้วยตัวเองและมีขนาดเล็ก ทำให้ Spring Boot เป็นตัวเลือกที่ยอดเยี่ยมสำหรับการพัฒนาและบริหารจัดการระบบในรูปแบบไมโครเซอร์วิส

2.3.4 การจัดการไลบรารีที่ง่าย (Simplified Dependency Management) มีกลไก "Starters" ที่ช่วยจัดการและรวบรวมไลบรารีที่จำเป็นสำหรับการใช้งานต่างๆ ไว้เป็นกลุ่ม ลดภาระของนักพัฒนาในการจัดการเวอร์ชันที่เข้ากันได้

2.3.5 การทดสอบที่สะดวก (Streamlined Testing) มีเครื่องมือและ Annotation ต่างๆ ที่ถูกออกแบบมาเพื่อช่วยให้การเขียนโค้ดสำหรับทดสอบ (Unit Test และ Integration Test) เป็นเรื่องง่ายและมีประสิทธิภาพ

2.3.6 เว็บเซิร์ฟเวอร์ในตัว (Built-in Web Server) มาพร้อมกับเว็บเซิร์ฟเวอร์ (เช่น Tomcat Jetty) ที่ติดตั้งในตัว ทำให้สามารถรันแอปพลิเคชันได้ทันทีโดยไม่ต้องติดตั้งหรือตั้งค่าเว็บเซิร์ฟเวอร์ภายนอกเพิ่มเติม

2.4 MariaDB

ในยุคที่ข้อมูลสารสนเทศเป็นหัวใจสำคัญของการดำเนินธุรกิจ การมีระบบจัดการฐานข้อมูล (Database Management System) ที่มีประสิทธิภาพและเชื่อถือได้จึงเป็นสิ่งจำเป็น MariaDB คือระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (Relational Database Management System - RDBMS)

แบบโอเพนซอร์ส (Open Source) ที่ได้รับความนิยมอย่างสูง ถูกสร้างขึ้นโดยทีมผู้พัฒนาดั้งเดิมของ MySQL นำโดย Michael "Monty" Widenius

MariaDB ถือกำเนิดขึ้นในฐานะ "Fork" (การแยกสายการพัฒนา) ของโครงการ MySQL หลังจากบริษัท Oracle เข้าซื้อกิจการของ Sun Microsystems (เจ้าของ MySQL ในขณะนั้น) โดยมีเป้าหมายหลักเพื่อรับประกันว่า MariaDB จะยังคงเป็นซอฟต์แวร์เสรีและโอเพนซอร์สภายใต้ใบอนุญาต GPL (General Public License) ตลอดไป ตอบสนองต่อความกังวลของชุมชนนักพัฒนาว่า MySQL อาจถูกจำกัดการใช้งานในเชิงพาณิชย์

จุดเด่นที่สุดของ MariaDB คือการถูกออกแบบมาให้เป็น "Drop-in Replacement" สำหรับ MySQL ซึ่งหมายความว่านักพัฒนาและผู้ดูแลระบบสามารถย้ายฐานข้อมูลจาก MySQL มาเป็น MariaDB ได้อย่างราบรื่นแทบจะทันที โดยไม่จำเป็นต้องแก้ไขโค้ดของแอปพลิเคชัน เนื่องจากใช้ API และ Protocol รวมถึงโครงสร้างที่เข้ากันได้อย่างสมบูรณ์ MariaDB ไม่ได้เป็นเพียงแค่สำเนาของ MySQL แต่มีการพัฒนาและเพิ่มประสิทธิภาพในหลายๆ ด้านอย่างต่อเนื่อง ทำให้มีข้อได้เปรียบที่น่าสนใจดังนี้

2.4.1 Storage Engines ที่หลากหลายกว่า นอกเหนือจาก Storage Engine มาตรฐานอย่าง InnoDB และ MyISAM ที่มีใน MySQL แล้ว MariaDB ยังได้พัฒนาและเพิ่มเติม Engine ที่มีประสิทธิภาพสูงเข้ามา

2.4.2 ประสิทธิภาพและความเร็วที่เหนือกว่า MariaDB มีการปรับปรุง Query Optimizer (ตัววางแผนการสืบค้นข้อมูล) ให้ทำงานได้ฉลาดและรวดเร็วยิ่งขึ้นในหลายๆ สถานการณ์ นอกจากนี้ยังมีฟีเจอร์อย่าง Thread Pooling ที่ช่วยจัดการการเชื่อมต่อจำนวนมากได้อย่างมีประสิทธิภาพมากกว่า ทำให้รองรับผู้ใช้งานพร้อมกันได้ดีขึ้น ซึ่งฟีเจอร์นี้มีให้ใช้ในเวอร์ชันฟรีของ MariaDB แต่มีเฉพาะใน MySQL รุ่น Enterprise ที่ต้องเสียค่าใช้จ่าย

2.4.3 คุณสมบัติที่ทันสมัย MariaDB มีการเพิ่มฟังก์ชันใหม่ๆ เข้ามาอย่างสม่ำเสมอ เช่น การจัดการข้อมูลแบบ เก็บประวัติการเปลี่ยนแปลงของข้อมูล ฟังก์ชันสำหรับจัดการข้อมูล JSON ที่ยืดหยุ่นกว่า และรองรับการทำ Clustering ด้วย Galera Cluster มาตั้งแต่ต้น ทำให้การสร้างระบบฐานข้อมูลที่มีความพร้อมใช้งานสูง ทำได้ง่ายขึ้น

2.4.4 การพัฒนาโดยชุมชนอย่างแท้จริง การพัฒนา MariaDB อยู่ภายใต้การดูแลของ MariaDB Foundation ซึ่งเป็นองค์กรไม่แสวงหาผลกำไร ทำให้มั่นใจได้ว่าทิศทางการพัฒนาจะเป็นไปเพื่อประโยชน์ของชุมชนผู้ใช้งาน ไม่ได้ถูกควบคุมโดยบริษัทใดบริษัทหนึ่ง

ด้วยเหตุผลเหล่านี้ MariaDB จึงไม่ได้เป็นเพียงแค่ "ทางเลือก" ของ MySQL แต่เป็นระบบฐานข้อมูลที่ทรงพลัง มีความยืดหยุ่นสูง และมีอนาคตที่มั่นคงภายใต้การพัฒนาของชุมชนโอเพนซอร์สอย่างแท้จริง

2.5 Omise

Omise คือผู้ให้บริการระบบรับชำระเงินออนไลน์ (Payment Gateway) ที่ออกแบบมาเพื่อมอบความสะดวกสบายสูงสุดทั้งต่อผู้ใช้งานและนักพัฒนา

สำหรับผู้ใช้งาน Omise ช่วยลดความยุ่งยากในขั้นตอนการชำระเงิน ทำให้ผู้ใช้ไม่ต้องกรอกข้อมูลซ้ำซ้อน และรองรับช่องทางการชำระเงินที่หลากหลายและเป็นที่ยอมรับในปัจจุบัน เช่น บัตรเครดิต พร้อมเพย์ และ Rabbit LINE Pay

ในด้านการพัฒนา Omise ได้จัดเตรียมไลบรารี (Library) สำหรับแพลตฟอร์มต่างๆ ไว้ ทำให้ง่ายต่อการนำไปเชื่อมต่อกับแอปพลิเคชันหรือเว็บไซต์ ลดระยะเวลาในการพัฒนาลงได้อย่างมาก และเปิดให้นักพัฒนา สามารถออกแบบหน้ารับชำระเงินได้ด้วยตนเอง เพิ่มความต่อเนื่องในการใช้งานแอปพลิเคชันของผู้ใช้

ด้านความปลอดภัยถือเป็นหัวใจสำคัญ โดย Omise จะเป็นผู้สื่อสารกับผู้ใช้โดยตรงเพื่อจัดการข้อมูลที่ละเอียดอ่อน เช่น ข้อมูลบัตรเครดิต และแปลงข้อมูลเหล่านั้นให้อยู่ในรูปแบบของ "โทเคน" (Token) ที่ปลอดภัย ก่อนจะส่งมายังเซิร์ฟเวอร์ของระบบ ทำให้ข้อมูลสำคัญทั้งหมดถูกจัดเก็บและรักษาความปลอดภัยไว้ที่ Omise โดยตรง

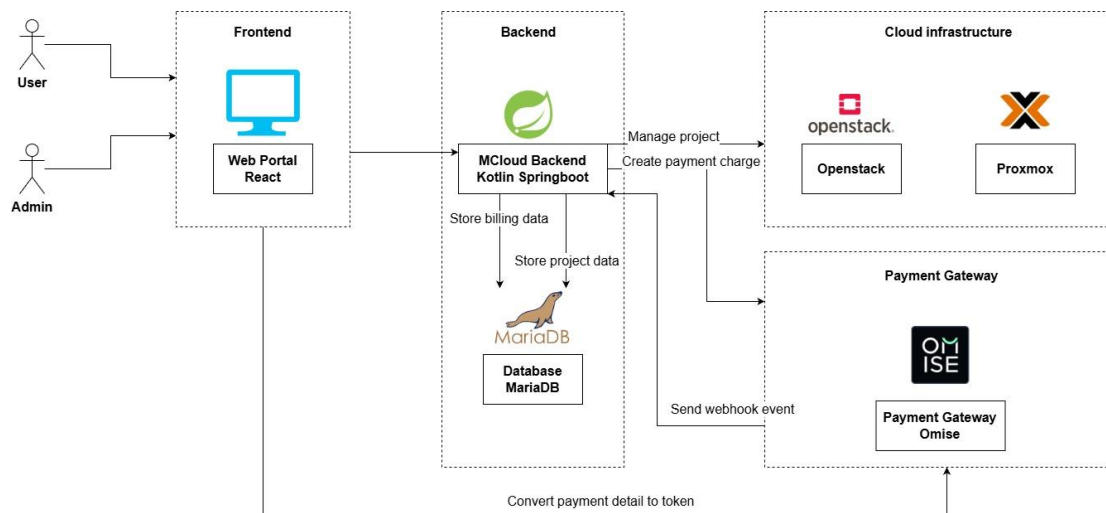
นอกจากนี้ ในมุมมองทางธุรกิจ Omise ยังมีความน่าสนใจด้วยอัตราค่าธรรมเนียมที่ต่ำกว่าและมีกระบวนการถอนเงินที่รวดเร็ว โดยใช้เวลาเพียง 1 วันทำการ ซึ่งช่วยเพิ่มสภาพคล่องให้กับธุรกิจได้เป็นอย่างดี

บทที่ 3

ขั้นตอน และวิธีดำเนินงาน

3.1 ภาพรวมการทำงานของระบบ

ภาพรวมการทำงานของระบบแสดงส่วนประกอบและการติดต่อสื่อสารระหว่างกันดังนี้



ภาพที่ 3.1 ภาพรวมการทำงานของระบบ

อธิบายภาพรวมการทำงานของระบบ

3.1.1 User สามารถใช้งานส่วนของการสร้างโครงการ จัดการสมาชิกในโครงการ เดิมเครดิต

3.1.2 Admin สามารถใช้งานส่วนของการจัดการโครงการ จัดการสมาชิกในโครงการ ดูรายการค่าใช้จ่ายของโครงการทั้งหมดในระบบ

3.1.3 User Web Portal หน้าเว็บเพื่อให้ User และ Admin ติดต่อกับระบบต่างๆ

3.1.4 MCloud Backend มีหน้าที่จัดการคำสั่งต่างๆ รวมถึงคำนวณราคา และเก็บค่าบริการการใช้งานคลาวด์รายวันผ่านเครดิต

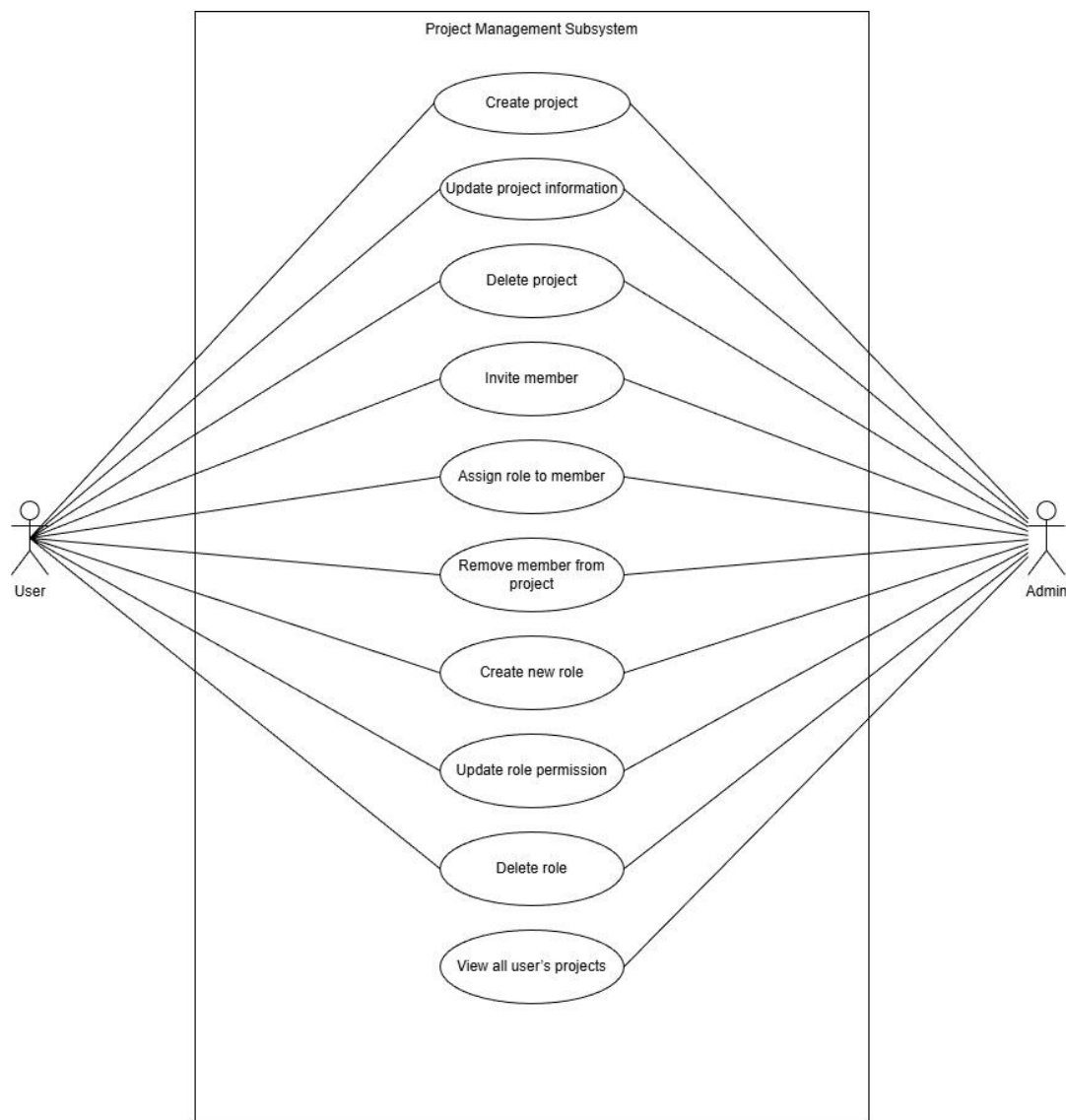
3.1.5 Payment Gateway มีหน้าที่เป็นตัวกลางรับชำระเงิน และเก็บข้อมูลบัตรเครดิตของผู้ใช้ทั้งหมดในระบบ

3.1.6 Cloud Infrastructure มีหน้าที่บริหารจัดการสร้างโครงการร่วมกับ Mcloud Backend รวมถึงการจัดการสมาชิกภายในโครงการ

3.1.7 Database จัดเก็บข้อมูลโครงการ สมาชิกโครงการ และข้อมูลการใช้งานคลาวด์ทั้งหมดในระบบ เพื่อใช้สำหรับการคิดค่าบริการ

3.2 Use Case Diagram

Use case diagram แสดงกระบวนการทำงาน และความสัมพันธ์ของผู้ใช้งานกับระบบส่วนของ Project Management Subsystem และ Billing Subsystem



ภาพที่ 3.2 Use case diagram: Project Management Subsystem

ภาพที่ 3.2 แสดง Use case diagram ส่วนของ Project Management Subsystem ประกอบไปด้วยผู้ใช้ 2 ประเภท และการเข้าถึง Use case ดังนี้

3.2.1 User (ผู้ใช้งาน) สามารถเข้าถึง Use case ดังนี้

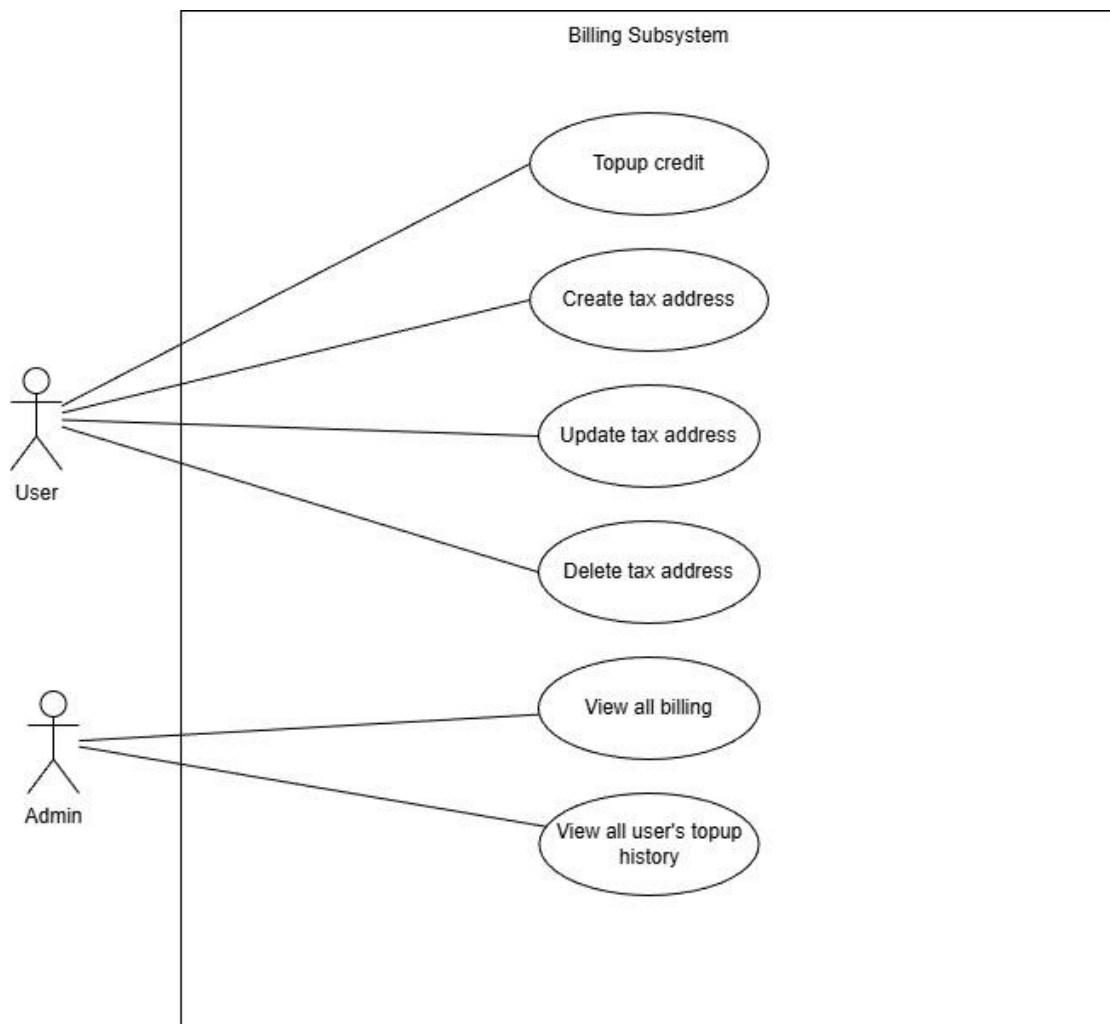
3.2.1.1 Create project การสร้างโครงการ

3.2.1.2 Update project information การแก้ไขข้อมูลโครงการ

3.2.1.3 Delete project การลบโครงการ

3.2.1.4 Invite Member การการเชิญสมาชิก

- 3.2.1.5 Assign role to member การกำหนด Role ให้กับสมาชิก
- 3.2.1.6 Remove member from project การลบสมาชิกออกจากโครงการ
- 3.2.1.7 Create new role การสร้าง Role ใหม่
- 3.2.1.8 Update role permission การแก้ไขสิทธิ์ของ Role
- 3.2.1.9 Delete role การลบ Role
- 3.2.2 Admin (ผู้ดูแลระบบ) สามารถเข้าถึง Use case ของ User ทั้งหมดรวมถึงการดูรายละเอียดโดยรวมดังนี้
 - 3.2.2.1 View all user's projects ดูโครงการของผู้ใช้ทั้งหมดในระบบ



ภาพที่ 3.3 Use case diagram: Billing Subsystem

ภาพที่ 3.3 แสดง Use case diagram ส่วนของ Billing Subsystem ประกอบไปด้วยผู้ใช้ 2 ประเภท และการเข้าถึง Use case ดังนี้

3.2.1 User (ผู้ใช้งาน) สามารถเข้าถึง Use case ดังนี้

3.2.1.1 Top up credit การเติมเงินเป็นเครดิตสำหรับจ่ายค่าบริการคลาวด์

3.2.1.2 Create tax address การสร้างที่อยู่ผู้เสียภาษี

3.2.1.3 Update tax address การแก้ไขที่อยู่ผู้เสียภาษี

3.2.1.4 Delete tax address การลบที่อยู่ผู้เสียภาษี

3.2.2 Admin (ผู้ดูแลระบบ) สามารถเข้าถึง Use case ดังนี้

3.2.2.1 View all billing ดูการใช้จ่ายทั้งหมดในระบบ

3.2.2.2 View all user's topup history ดูประวัติการเติมเครดิต

3.3 Use Case description

Use Case description แสดงการทำงานของระบบจัดการโครงการดังตารางที่ 3.1-3.10

ตารางที่ 3.1 Use Case description : Create Project

Use Case Number	1	
Use Case Name	Create project	
Brief Description	ผู้ใช้งานสามารถสร้างโครงการของคลาวด์ขึ้นมา โดยกำหนดชื่อคำอธิบายโครงการ และสามารถเลือกผู้ให้บริการคลาวด์รองรับในระบบ	
Actors	1. User 2. Admin	
Pre-condition	ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ User หรือ Admin	
Post-condition	ข้อมูลที่กรอกถูกบันทึกลงบนฐานข้อมูล	
Main Flow	Actor	System
	1. เลือกเมนู “สร้างโครงการ” 2. กรอกข้อมูล ชื่อโครงการ คำอธิบายโครงการ 3. เลือกผู้ให้บริการคลาวด์ 4. คลิกปุ่ม สร้างโครงการ	5. ตรวจสอบความถูกต้องของข้อมูล 6. ส่งข้อมูลให้ Backend 7. ส่งข้อมูลให้ผู้ให้บริการคลาวด์ 8. บันทึกข้อมูลโครงการลงบนฐานข้อมูล 9. แสดงข้อความ “สร้างโครงการสำเร็จ”
Alternate Flow / Exceptional Flow	หากกรอกข้อมูลที่ไม่ครบถ้วน: แจ้งว่า “กรุณากรอกข้อมูลให้ครบถ้วน” หากผู้ใช้งานมีสิทธิ์เป็น Admin: กรอกข้อมูลอีเมลของ User ที่จะมาเป็นเจ้าของโครงการ	

ตารางที่ 3.2 Use Case description: Update project

Use Case Number	2	
Use Case Name	Update project	
Brief Description	ผู้ใช้งานสามารถแก้ไขชื่อ คำอธิบาย หรือเจ้าของโครงการได้	
Actors	User	
Pre-condition	1. ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ User หรือ Admin 2. ผู้ใช้งานต้องมีโครงการที่สร้างไว้แล้ว 3. ผู้ใช้ต้องมีสิทธิ์ในการแก้ไขโครงการ (project: update)	
Post-condition	ข้อมูลที่แก้ไขถูกบันทึกลงบนฐานข้อมูล	
Main Flow	Actor	System
	1. เลือกปุ่ม “แก้ไขโครงการ” 2. กรอกข้อมูลใหม่ที่ต้องการแก้ไข 3. กดปุ่มบันทึก	4. ตรวจสอบความถูกต้องของข้อมูลใหม่ 5. ส่งข้อมูลไปยัง Backend 6. อัปเดตข้อมูลในฐานข้อมูล 7. แสดงข้อความแจ้งเตือนว่า “แก้ไขรายละเอียดโครงการสำเร็จ”
Alternate Flow / Exceptional Flow	หากข้อมูลไม่ครบ หรือไม่ถูกต้อง: ระบบแจ้งข้อผิดพลาด และยกเลิกการดำเนินการ	

ตารางที่ 3.3 Use Case description: Delete project

Use Case Number	3	
Use Case Name	Delete project	
Brief Description	ผู้ใช้งานสามารถลบโครงการของตัวเองได้	
Actors	1. User 2. Admin	
Pre-condition	4. ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ User หรือ Admin 5. ผู้ใช้งานต้องมีโครงการที่สร้างไว้แล้ว 6. ผู้ใช้งานต้องเป็นเจ้าของโครงการ หรือมีสิทธิ์เป็น Admin	
Post-condition	โครงการถูกลบออกจากรฐานข้อมูล และผู้ให้บริการคลาวด์	
Main Flow	Actor	System
	1. เลือกปุ่ม “ลบโครงการ” 2. ยืนยันการลบ	3. ตรวจสอบว่ามีโครงการอยู่ในระบบ 4. ตรวจสอบว่าโครงการมีคลาวด์ที่ใช้งานอยู่หรือไม่ 5. ลบข้อมูลที่เกี่ยวข้องออกจากรฐานข้อมูล 6. ลบโครงการบนผู้ให้บริการคลาวด์ 7. ลบโครงการบนฐานข้อมูล 8. แสดงข้อความแจ้งเตือนว่า “ลบโครงการสำเร็จ” 9. ย้ายหน้าเว็บกลับไปหน้าเลือกโครงการ
Alternate Flow / Exceptional Flow	หากมีคลาวด์เปิดใช้งานอยู่ในโครงการระบบจะแจ้งว่า “กรุณายกเลิกคลาวด์ที่ใช้งานอยู่ก่อนลบโครงการนี้”	

ตารางที่ 3.4 Use Case description: Invite member

Use Case Number	4	
Use Case Name	Invite member	
Brief Description	ผู้ใช้งานสามารถเชิญผู้อื่นมาเป็นสมาชิกโครงการได้	
Actors	1. User 2. Admin	
Pre-condition	1. ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ User หรือ Admin 2. ผู้ใช้งานต้องมีโครงการที่สร้างไว้แล้ว หรือเข้าร่วมอยู่แล้ว 3. ถ้าผู้ใช้มีสิทธิ์เป็น User ต้องมี Role ที่สามารถเชิญสมาชิกได้ (member: create)	
Post-condition	บันทึกข้อมูลคำเชิญเข้าร่วมโครงการลงบนฐานข้อมูล	
Main Flow	Actor	System
	1. เลือกปุ่ม “เชิญสมาชิก” 2. กรอกข้อมูล Email และ Role ของผู้ใช้ที่ต้องการเชิญ 3. กดปุ่ม “ตกลง”	4. ตรวจสอบความถูกต้องของข้อมูล 5. ส่งข้อมูลไปยัง Backend 6. บันทึกข้อมูลคำเชิญลงบนฐานข้อมูล 7. แสดงข้อความแจ้งเตือนว่า “เชิญสมาชิกเรียบร้อยแล้ว” 8. ส่งการแจ้งเตือนไปที่สมาชิกที่ถูกเชิญว่า “คุณถูกเชิญให้เข้าร่วมโครงการ”
Alternate Flow / Exceptional Flow	หากข้อมูลไม่ครบ หรือไม่ถูกต้อง: ระบบแจ้งข้อผิดพลาด และยกเลิกการดำเนินการ	

ตารางที่ 3.5 Use Case description: Assign role to member

Use Case Number	5	
Use Case Name	Assign role to member	
Brief Description	ผู้ใช้งานสามารถเปลี่ยน Role ของสมาชิกภายในโครงการได้	
Actors	1. User 2. Admin	
Pre-condition	1. ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ User หรือ Admin 2. ผู้ใช้งานต้องมีโครงการที่สร้างไว้แล้ว หรือเข้าร่วมอยู่แล้ว 3. ถ้าผู้ใช้งานมีสิทธิ์เป็น User ต้องมี Role ที่สามารถแก้ไขสมาชิกได้ (member: update)	
Post-condition	ข้อมูลที่แก้ไขถูกบันทึกลงบนฐานข้อมูล	
Main Flow	Actor	System
	1. กดปุ่ม “เปลี่ยน Role” บนสมาชิกที่ต้องการเปลี่ยน 2. เลือก Role ใหม่ 3. กดปุ่ม “เปลี่ยน”	4. ส่งข้อมูลคำขอไปที่ Backend 5. ตรวจสอบว่ามีสมาชิกอยู่ภายในโครงการ 6. แก้ไขข้อมูลบนฐานข้อมูล 7. แสดงข้อความแจ้งเตือนว่า “เปลี่ยน Role ของสมาชิกแล้ว”
Alternate Flow / Exceptional Flow	-	

ตารางที่ 3.6 Use Case description: Remove member from project

Use Case Number	6	
Use Case Name	Remove member from project	
Brief Description	ผู้ใช้งานสามารถลบสมาชิกออกจากโครงการได้	
Actors	1. User 2. Admin	
Pre-condition	4. ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ User หรือ Admin 5. ผู้ใช้งานต้องมีโครงการที่สร้างไว้แล้ว หรือเข้าร่วมอยู่แล้ว 6. ถ้าผู้ใช้งานมีสิทธิ์เป็น User ต้องมี Role ที่สามารถลบสมาชิกได้ (member: delete)	
Post-condition	ลบข้อมูลสมาชิกออกจากโครงการ	
Main Flow	Actor	System
	1. กดปุ่ม “ลบ” บนสมาชิกที่ต้องการลบ 2. กดปุ่ม “ยืนยัน”	3. ส่งข้อมูลคำขอไปที่ Backend 4. ตรวจสอบว่ามีสมาชิกอยู่ในโครงการ 5. ลบสมาชิกออกจากฐานข้อมูล 6. ส่งคำขอลบสมาชิกออกจากโครงการไปที่ผู้ให้บริการคลาวด์ 7. แสดงข้อความแจ้งเตือนว่า “ลบสมาชิกออกจากโครงการแล้ว”
Alternate Flow / Exceptional Flow	-	

ตารางที่ 3.7 Use Case description: Create role

Use Case Number	7	
Use Case Name	Create role	
Brief Description	ผู้ใช้งานสามารถสร้าง Role ในโครงการขึ้นมา โดยกำหนดชื่อ และ สิทธิ์การจัดการของ Role	
Actors	1. User 2. Admin	
Pre-condition	1. ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ User หรือ Admin 2. ผู้ใช้งานต้องมีโครงการที่สร้างไว้แล้ว หรือเข้าร่วมอยู่แล้ว 3. ถ้าผู้ใช้งานมีสิทธิ์เป็น User ต้องมี Role ที่สามารถสร้าง Role ได้ (member: create)	
Post-condition	ข้อมูลที่กรอกถูกบันทึกลงบนฐานข้อมูล	
Main Flow	Actor	System
	1. เลือกเมนู “Roles” ภายในโครงการ 2. กดปุ่ม “Role ใหม่” กรอกข้อมูลชื่อ สิทธิ์การจัดการของ Role 3. กดปุ่ม “บันทึก”	4. ตรวจสอบความถูกต้องของข้อมูล 5. ส่งข้อมูลให้ Backend 6. บันทึกข้อมูล Role ลงในฐานข้อมูล 7. แสดงข้อความ “สร้าง Role สำเร็จ”
Alternate Flow / Exceptional Flow	หากกรอกข้อมูลที่ไม่ครบถ้วน: แจ้งว่า “กรุณากรอกข้อมูลให้ครบถ้วน”	

ตารางที่ 3.8 Use Case description: Update role permission

Use Case Number	8	
Use Case Name	Update role permission	
Brief Description	ผู้ใช้งานสามารถแก้ไขชื่อ สิทธิการจัดการของ Role ได้	
Actors	1. User 2. Admin	
Pre-condition	1. ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ User หรือ Admin 2. ผู้ใช้งานต้องมีโครงการที่สร้างไว้แล้ว หรือเข้าร่วมอยู่แล้ว 3. ถ้าผู้ใช้มีสิทธิ์เป็น User ต้องมี Role ที่สามารถแก้ไข Role ได้ (member: update)	
Post-condition	ข้อมูลที่แก้ไขถูกบันทึกลงบนฐานข้อมูล	
Main Flow	Actor	System
	1. เลือก Role ที่ต้องการ 2. กรอกข้อมูลชื่อ Role ใหม่ หรือดัดแปลงสิทธิ์ที่ต้องการให้ Role นั้น เข้าถึงได้คือ Create Read Update และ Delete แบ่งเป็นส่วนของ Project Member และ Cloud 3. กดปุ่มบันทึก	4. ตรวจสอบความถูกต้องของข้อมูลใหม่ 5. ส่งข้อมูลไปยัง Backend 6. อัปเดตข้อมูลในฐานข้อมูล 7. แสดงข้อความแจ้งเตือนว่า “แก้ไข Role สำเร็จ”
Alternate Flow / Exceptional Flow	หากข้อมูลไม่ครบ หรือไม่ถูกต้อง: ระบบแจ้งข้อผิดพลาด และยกเลิกการดำเนินการ	

ตารางที่ 3.9 Use Case description: Delete role

Use Case Number	9	
Use Case Name	Delete role	
Brief Description	ผู้ใช้งานสามารถลบ Role ออกจากโครงการได้	
Actors	1. User 2. Admin	
Pre-condition	1. ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ User หรือ Admin 2. ผู้ใช้งานต้องมีโครงการที่สร้างไว้แล้ว หรือเข้าร่วมอยู่แล้ว 3. ถ้าผู้ใช้งานมีสิทธิ์เป็น User ต้องมี Role ที่สามารถแก้ไข Role ได้ (member: delete)	
Post-condition	ลบข้อมูล Role ออกจากโครงการ	
Main Flow	Actor	System
	1. เลือก Role ที่ต้องการลบ 2. กดปุ่ม “ลบ” 3. กดปุ่ม “ยืนยัน”	4. ส่งข้อมูลคำขอไปที่ Backend 5. ตรวจสอบว่ามี Role อยู่ภายในโครงการ 6. ตรวจสอบว่า Role ที่ต้องการลบมีสมาชิกใช้อยู่ 7. ลบข้อมูล Role ออกจากฐานข้อมูล 8. แสดงข้อความแจ้งเตือนว่า “ลบ Role แล้ว”
Alternate Flow / Exceptional Flow	หากเลือกลบ Role ที่มีสมาชิกเป็นอยู่: ระบบจะเปลี่ยน Role ของสมาชิกคนดังกล่าวเป็น Default (เป็น Role ที่สร้างขึ้นมาเริ่มต้นพร้อมการสร้างโครงการ และไม่สามารถลบได้)	

ตารางที่ 3.10 Use Case description: View all user's project

Use Case Number	10	
Use Case Name	View all user's project	
Brief Description	ผู้ใช้งานสามารถดูข้อมูลโครงการทั้งหมดในระบบได้	
Actors	Admin	
Pre-condition	ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ Admin	
Post-condition	แสดงรายการโครงการทั้งหมดในรูปแบบตาราง	
Main Flow	Actor	System
	1. เลือกเมนู “จัดการโครงการ”	2. ส่งคำขอรายการไปยัง Backend 3. แสดงรายการโครงการที่มีในระบบในรูปแบบตารางแบ่งตามผู้ให้บริการคลาวด์
Alternate Flow / Exceptional Flow	-	

Use Case description แสดงการทำงานของระบบจัดการโครงการดังตารางที่ 3.11-3.16
 ตารางที่ 3.11 Use Case description: Topup credit

Use Case Number	11	
Use Case Name	Topup credit	
Brief Description	ผู้ใช้งานสามารถเติมเงินเพื่อแลกเป็นเครดิตในระบบได้	
Actors	User	
Pre-condition	ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ User	
Post-condition	บันทึกผลลัพธ์การชำระเงินลงบนฐานข้อมูล	
Main Flow	Actor	System
	1. กดปุ่ม “เติมเงิน” 2. กรอกข้อมูล จำนวนเครดิตที่ต้องการเติม 3. เลือกช่องทางการเติมเงิน 4. กรอกข้อมูลการชำระเงิน 5. กดปุ่ม “ชำระเงิน”	6. ส่งคำขอสร้างรายการ ชำระเงินไปที่ Omise 7. ส่งข้อมูลที่ได้รับจาก Omise ไปที่ Backend 8. ตรวจสอบความถูกต้อง ของข้อมูล 9. สร้างรายการชำระเงิน ใหม่บนฐานข้อมูล 10. แสดงการแจ้งเตือน ชำระเงินสำเร็จ
Alternate Flow / Exceptional Flow	หากกรอกข้อมูลที่กรอกไม่ครบถ้วน: แจ้งว่า “กรุณากรอกข้อมูลให้ครบถ้วน” หากไม่ชำระเงินในเวลาที่กำหนด: แจ้งว่า “การชำระเงินไม่สำเร็จ โปรดทำรายการใหม่”	

ตารางที่ 3.12 Use Case description: Create tax address

Use Case Number	12	
Use Case Name	Create tax address	
Brief Description	ผู้ใช้งานสามารถเพิ่มที่อยู่ผู้เสียภาษีได้	
Actors	User	
Pre-condition	ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ User	
Post-condition	ข้อมูลที่กรอกถูกบันทึกลงบนฐานข้อมูล	
Main Flow	Actor	System
	1. เลือกเมนู “เพิ่มที่อยู่ผู้เสียภาษี” 2. กรอกข้อมูลที่อยู่ผู้เสียภาษี 3. กดปุ่ม “บันทึก”	4. ตรวจสอบความถูกต้องของข้อมูล 5. ส่งข้อมูลให้ Backend 6. บันทึกข้อมูลโครงการลงในฐานข้อมูล 7. แสดงข้อความ “เพิ่มที่อยู่สำเร็จ”
Alternate Flow / Exceptional Flow	หากกรอกข้อมูลที่กรอกไม่ครบถ้วน: แจ้งว่า “กรุณากรอกข้อมูลให้ครบถ้วน”	

ตารางที่ 3.13 Use Case description: Update tax address

Use Case Number	13	
Use Case Name	Update tax address	
Brief Description	ผู้ใช้งานสามารถแก้ไขที่อยู่ผู้เสียภาษีได้	
Actors	User	
Pre-condition	ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ User	
Post-condition	ข้อมูลที่แก้ไขถูกบันทึกลงบนฐานข้อมูล	
Main Flow	Actor	System
	1. เลือกปุ่ม “แก้ไข” บนที่อยู่ที่ต้องการ 2. กรอกข้อมูลใหม่ที่ต้องการแก้ไข 3. กดปุ่มบันทึก	4. ตรวจสอบความถูกต้องของข้อมูลใหม่ 5. ส่งข้อมูลไปยัง Backend 6. อัปเดตข้อมูลในฐานข้อมูล 7. แสดงข้อความแจ้งเตือนว่า “แก้ไขที่อยู่สำเร็จ”
Alternate Flow / Exceptional Flow	หากข้อมูลไม่ครบ หรือไม่ถูกต้อง: ระบบแจ้งข้อผิดพลาด และยกเลิกการดำเนินการ	

ตารางที่ 3.14 Use Case description: Delete tax address

Use Case Number	14	
Use Case Name	Delete tax address	
Brief Description	ผู้ใช้งานสามารถลบที่อยู่ผู้เสียภาษีของตนเองได้	
Actors	User	
Pre-condition	ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ User	
Post-condition	ข้อมูลที่อยู่ผู้เสียภาษีถูกลบออกจากรฐานข้อมูล	
Main Flow	Actor	System
	1. เลือกปุ่ม “แก้ไข” บนที่ อยู่ที่ต้องการลบ 2. กดปุ่ม “ลบ” 3. ยืนยันการลบ	4. ตรวจสอบว่ามีข้อมูล อยู่ในระบบ 5. ลบข้อมูลบนฐานข้อมูล 6. แสดงข้อความแจ้ง เตือนว่า “ลบที่อยู่ สำเร็จ”
Alternate Flow / Exceptional Flow	-	

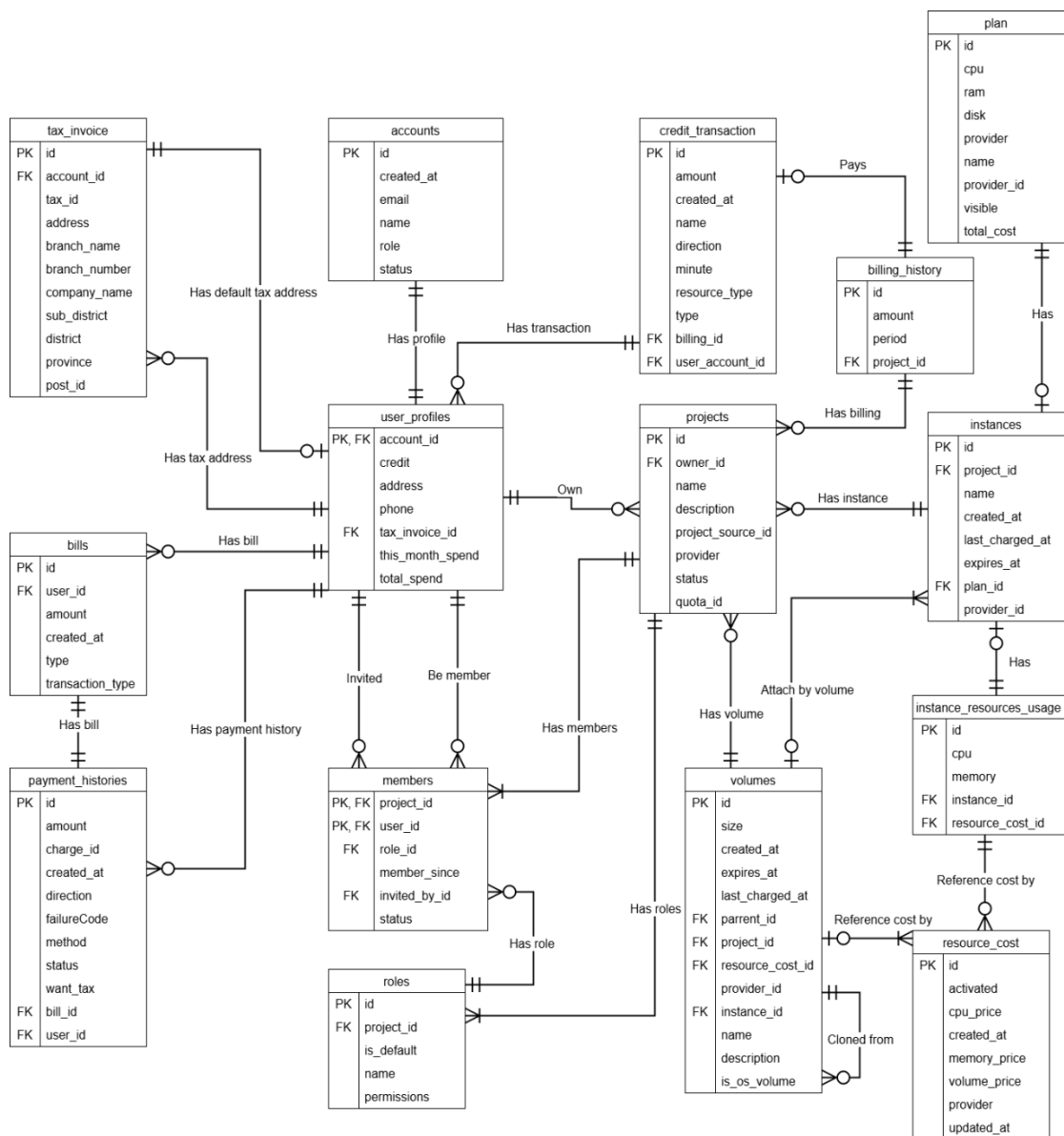
ตารางที่ 3.15 Use Case description: View all billing

Use Case Number	15	
Use Case Name	View all billing	
Brief Description	ผู้ใช้งานสามารถดูข้อมูลประวัติการใช้จ่ายของโครงการทั้งหมดในระบบได้	
Actors	Admin	
Pre-condition	ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ Admin	
Post-condition	แสดงรายการประวัติการใช้จ่ายของโครงการทั้งหมดในรูปแบบตาราง	
Main Flow	Actor	System
	1. เลือกเมนู “การเก็บเงิน”	2. ส่งคำขอรายการไปยัง Backend 3. แสดงรายการประวัติการใช้จ่ายของโครงการทั้งหมดในรูปแบบตาราง
Alternate Flow / Exceptional Flow	-	

ตารางที่ 3.16 Use Case description: View all user's topup history

Use Case Number	16	
Use Case Name	View all user's topup history	
Brief Description	ผู้ใช้งานสามารถดูข้อมูลประวัติการเติมเงินของผู้ใช้ทั้งหมดในระบบได้	
Actors	Admin	
Pre-condition	ผู้ใช้งานต้องเข้าสู่ระบบด้วยสิทธิ์ Admin	
Post-condition	แสดงรายการประวัติการเติมเงินทั้งหมดในรูปแบบตาราง	
Main Flow	Actor	System
	1. เลือกเมนู “การเติมเงิน”	2. ส่งคำขอรายการไปยัง Backend 3. แสดงรายการประวัติการเติมเงินในรูปแบบตาราง
Alternate Flow / Exceptional Flow	-	

3.4 ER Diagram



ภาพที่ 3.4 ER Diagram

ภาพที่ 3.4 เป็นแผนภาพ Entity Relationship อธิบายโครงสร้างและความสัมพันธ์ของข้อมูลในแต่ละตาราง โดยการเก็บข้อมูลจะแบ่งส่วนที่เกี่ยวข้องทั้งหมด 2 ส่วนหลักๆ ได้แก่ ส่วนข้อมูลของผู้ใช้ และส่วนข้อมูลของโครงการ โดยข้อมูลบางส่วนที่ใช้งานจะถูกเก็บไว้กับระบบภายนอกเช่น ข้อมูล Customer_ID ของผู้ใช้ที่จะมีการเรียกเพื่อขอข้อมูลบัตรเครดิตจาก Omise ข้อมูล Project_ID ว่าผู้ใช้งานกำลังใช้งานโครงการอะไรอยู่ รายละเอียดของแต่ละตารางและความหมายของแต่ละฟิลด์สามารถดูเพิ่มเติมได้ที่หัวข้อ 3.5 Data Dictionary

3.5 Data Dictionary

ตารางที่ 3.17 – 3.31 แสดงรายละเอียด และความหมายของข้อมูลแต่ละฟิลด์ภายในฐานข้อมูลของระบบ

ตารางที่ 3.17 Data Dictionary : accounts

Key	Column Name	Data Type	Nullable	Description
PK	id	UUID	NOT NULL	ID ที่เป็น Unique
	created_at	Datetime(6)	NOT NULL	วันเวลาที่สร้างผู้ใช้
	email	Varchar(255)	NOT NULL	Email ของผู้ใช้
	name	Varchar(255)	NOT NULL	ชื่อ-นามสกุลของผู้ใช้
	role	Enum	NOT NULL	Role ของผู้ใช้ในทั้งระบบ ("ADMIN", "USER")
	status	Enum	NOT NULL	สถานะบัญชีของผู้ใช้ว่าสามารถเข้าสู่ระบบได้หรือไม่ ("ACTIVE", "INACTIVE")

ตารางที่ 3.18 Data Dictionary : user_profiles

Key	Column Name	Data Type	Nullable	Description
PK, FK	account_id	UUID	NOT NULL	ID ของ Account ที่เป็น Unique
	credit	Decimal(14,4)	NOT NULL	จำนวนเครดิตที่มี สำหรับใช้ จ่ายในระบบ
	address	Varchar(255)	NULL	ที่อยู่ของผู้ใช้
	phone	Varchar(255)	NULL	เบอร์โทรศัพท์ของผู้ใช้
FK	tax_invoice_id	UUID	NULL	ID ของที่อยู่ผู้เสียภาษี โดยจะ ถูกใช้เป็นค่าเริ่มต้นหากมีการ ขอใบกำกับภาษี
	this_month_spend	Decimal(14,4)	NOT NULL	จำนวนเครดิตที่ใช้ภายในเดือน ปัจจุบัน จะถูกรีเซ็ตเป็น 0 ทุก วันที่ 1
	total_spend	Decimal(14,4)	NOT NULL	จำนวนเครดิตที่ใช้ภายในระบบ ทั้งหมดของผู้ใช้

ตารางที่ 3.19 Data Dictionary : tax_invoice

Key	Column Name	Data Type	Nullable	Description
PK	id	UUID	NOT NULL	ID ที่เป็น Unique
FK	account_id	UUID	NOT NULL	ID ของ User เจ้าของข้อมูลนี้
	tax_id	Varchar(255)	NOT NULL	เลขประจำตัวผู้เสียภาษี
	branch_name	Enum	NOT NULL	ประเภทของที่อยู่ ("Headquarters", "Individual", "Branch")
	branch_number	Varchar(255)	NULL	เลขสาขาสำหรับที่อยู่ Headquarters และ Branch
	company_name	Varchar(255)	NOT NULL	ชื่อของเจ้าของที่อยู่ ชื่อบริษัท สาขา หรือชื่อคน
	sub_district	Varchar(255)	NOT NULL	แขวง/ตำบล
	district	Varchar(255)	NOT NULL	เขต/อำเภอ
	province	Varchar(255)	NOT NULL	จังหวัด
	post_id	Varchar(255)	NOT NULL	รหัสไปรษณีย์

ตารางที่ 3.20 Data Dictionary : bills

Key	Column Name	Data Type	Nullable	Description
PK	id	BigInt(20)	NOT NULL	ID ที่เป็น Unique
FK	user_id	UUID	NOT NULL	ID ของ User เจ้าของข้อมูลนี้
	amount	Decimal(14,4)	NOT NULL	จำนวนเงินของใบเสร็จนี้
	created_at	Datetime(6)	NOT NULL	วันที่ใบเสร็จนี้ถูกสร้าง
	type	Enum	NOT NULL	ประเภทของใบเสร็จได้แก่ ใบเสร็จรับเงิน และใบเสนอ ราคา (“TAX_QUOTATION”, “QUOTATION”, “TAX_RECEIPT”, “RECEIPT”)
	transaction_type	Enum	NOT NULL	ประเภทของธุรกรรมของ ใบเสร็จนี้ (“ADD”, “CANCEL”, “CHARGE”, “CHARGE_OVERDUE”, “DEDUCT”, “REFUND”, “TOPUP”)

ตารางที่ 3.21 Data Dictionary : payment_histories

Key	Column Name	Data Type	Nullable	Description
PK	id	BigInt(20)	NOT NULL	ID ที่เป็น Unique
	amount	Decimal(14,4)	NOTNULL	จำนวนเงินของการเติมเงิน
	charge_id	Varchar(255)	NULL	รหัสอ้างอิงธุรกรรมที่เกิดขึ้นบน Omise ใช้เพื่อเชื่อมโยงข้อมูลกับฐานข้อมูล
	created_at	Datetime(6)	NULL	
	direction	Enum	NOT NULL	ทิศทางของเงินในธุรกรรมครั้งนี้เป็นบวกหรือลบ (“IN”, “OUT”)
	failureCode	Varchar(255)	NULL	รหัสข้อผิดพลาดที่ได้รับจาก Omise ใช้สำหรับแปลงเป็นข้อความกรณีเกิดข้อผิดพลาดต่างๆ จากฝั่ง Omise
	method	Enum	NULL	ช่องทางการชำระเงิน (“CREDIT_CARD”, “PROMPTPAY”)
	status	Enum	NULL	สถานะของธุรกรรม (“CANCELLED”, “EXPIRED”, “FAILED”, “PENDING”, “REFUNDED”, “SUCCESS”)
	want_tax	Boolean	NOT NULL	ธุรกรรมนี้ต้องการใบกำกับภาษีหรือไม่
FK	bill_id	BigInt(20)	NULL	ID ของใบเสร็จที่เกี่ยวข้องกับธุรกรรมนี้
FK	user_id	UUID	NULL	ID ของผู้เป็นเจ้าของข้อมูลนี้

ตารางที่ 3.22 Data Dictionary : projects

Key	Column Name	Data Type	Nullable	Description
PK	id	UUID	NOT NULL	ID ที่เป็น Unique
FK	owner_id	UUID	NOT NULL	ID ของ User เจ้าของโครงการ นี้
	name	Varchar(255)	NOT NULL	ชื่อของโครงการ
	description	Varchar(255)	NULL	คำอธิบายของโครงการ
	project_source_id	Varchar(255)	NULL	ID ของโครงการจากฝั่งผู้ ให้บริการคลาวด์ภายนอก ใช้ เพื่อเชื่อมโยงข้อมูลระหว่างกัน
	provider	Enum	NOT NULL	ผู้ให้บริการคลาวด์ของ โครงการนี้ (“OPENSTACK”, “PROXMOX”)
	status	Enum	NOT NULL	สถานะของโครงการว่าถูกปิด ไปแล้วหรือไม่ (“ACTIVE”, “INACTIVE”)
	quota_id	UUID	NULL	ID ของโควต้าทรัพยากรของ โครงการนี้

ตารางที่ 3.23 Data Dictionary : roles ข้อมูล Role

Key	Column Name	Data Type	Nullable	Description
PK	id	Int(11)	NOT NULL	ID ที่เป็น Unique
FK	Project_id	UUID	NOT NULL	ID ของโครงการที่ Role นี้ถูกสร้างขึ้นภายใน
	Is_default	Boolean	NOT NULL	Role นี้เป็นค่าเริ่มต้นหรือไม่ ใช้เพื่อกำกับไม่ให้ผู้ใช้ลบ Role ออกทั้งหมด
	name	Varchar(255)	NOT NULL	ชื่อของ Role
	permissions	Text	NOT NULL	สิทธิ์การเข้าถึงของ Role ถูกเก็บในรูปแบบ JSON โดยแต่ละ key แทนหมวดการทำงาน เช่น project cloud member และมีค่าเป็นสิทธิ์ย่อย { "create": true, "read": true, "update": true, "delete": true } เพื่อกำหนดสิทธิ์ CRUD ของแต่ละหมวด

ตารางที่ 3.24 Data Dictionary : members

Key	Column Name	Data Type	Nullable	Description
PK, FK	project_id	UUID	NOT NULL	ID ของโครงการที่สมาชิกคนนี้อยู่
PK, FK	user_id	UUID	NOT NULL	ID ของ User ที่เป็นสมาชิก
FK	role_id	Int(11)	NOT NULL	ID ของ Role ที่สมาชิกคนนี้ได้รับ
	member_since	Datetime(6)	NOT NULL	วันที่สมาชิกคนนี้เข้าร่วมโครงการ
FK	invited_by_id	UUID	NULL	ID ของ User ที่เชิญสมาชิกคนนี้เข้าโครงการ
	status	Enum	NOT NULL	สถานะของสมาชิกว่าเข้าร่วมโครงการแล้วหรือยัง (“INVITED”, “JOINED”)

ตารางที่ 3.25 Data Dictionary : billing_history

Key	Column Name	Data Type	Nullable	Description
PK	id	BigInt(20)	NOT NULL	ID ที่เป็น Unique
	amount	Decimal(14,4)	NOT NULL	จำนวนเงินค่าใช้จ่าย
	period	Date	NOT NULL	ช่วงเวลาที่มีการเก็บค่าใช้จ่าย
FK	project_id	UUID	NOT NULL	ID ของโครงการของค่าใช้จ่าย นี้

ตารางที่ 3.26 Data Dictionary : credit_transaction

Key	Column Name	Data Type	Nullable	Description
PK	id	BigInt(20)	NOT NULL	ID ที่เป็น Unique
	amount	Decimal(14,4)	NOT NULL	จำนวนเครดิตในธุรกรรมนี้
	created_at	Datetime(6)	NOT NULL	วันเวลาที่เกิดธุรกรรมนี้
	name	Varchar(255)	NULL	ชื่อของ Resource ที่เกี่ยวข้องกับธุรกรรมนี้
	direction	Enum	NOT NULL	ทิศทางการเงินของธุรกรรมนี้ ว่าเป็นบวกหรือลบ (“IN”, “OUT”)
	minute	BigInt(20)	NULL	จำนวนเวลาที่ส่วนเกิน สำหรับแสดงผลรณียกเลิก ว่า ใช้เกินเท่าไร หรือมีเวลาเหลือ เท่าไร ก่อนยกเลิก
	resource_type	Enum	NULL	ประเภทของ Resource (“VOLUME”, “INSTANCE”)
	type	Enum	NOT NULL	ประเภทของธุรกรรมนี้ (“ADD”, “CANCEL”, “CHARGE”, “CHARGE_OVERDUE”, “DEDUCT”, “REFUND”, “TOPUP”)
FK	billing_id	BigInt(20)	NULL	ID ของการเรียกเก็บเงินที่ เกี่ยวข้องกับธุรกรรมนี้
FK	user_account_id	UUID	NOT NULL	ID ของ User เจ้าของธุรกรรม นี้

ตารางที่ 3.27 Data Dictionary : plans

Key	Column Name	Data Type	Nullable	Description
PK	id	UUID	NOT NULL	ID ที่เป็น Unique
	cpu	Int(11)	NOT NULL	จำนวน vCPU ในแพ็คเกจนี้ มีหน่วยเป็น จำนวน Core
	ram	Int(11)	NOT NULL	จำนวน Ram ในแพ็คเกจนี้ มีหน่วยเป็น GB
	disk	Int(11)	NOT NULL	จำนวนความจุ Storage ใน แพ็คเกจนี้ มีหน่วยเป็น GB
	provider	Enum	NOT NULL	ผู้ให้บริการคลาวด์ของแพ็คเกจ นี้ (“OPENSTACK”, “PROXMOX”)
	name	Varchar(255)	NOT NULL	ชื่อของแพ็คเกจ
	provider_id	Varchar(255)	NULL	ID ของแพ็คเกจจากฝั่งของผู้ ให้บริการคลาวด์ ใช้เพื่อ เชื่อมต่อข้อมูล
	visible	Boolean	NOT NULL	สถานะของแพ็คเกจ ว่าเปิดให้ ผู้ใช้ใหม่สามารถซื้อได้หรือไม่
	total_cost	Decinal(14,4)	NOT NULL	ราคารวมของแพ็คเกจราย ชั่วโมง

ตารางที่ 3.28 Data Dictionary : resource_cost

Key	Column Name	Data Type	Nullable	Description
PK	id	UUID	NOT NULL	ID ที่เป็น Unique
	activated	Boolean	NOT NULL	สถานะของราคากลาง โดยจะมีเพียงแถวเดียวต่อหนึ่งผู้ให้บริการที่มีสถานะเป็น true
	cpu_price	Decimal(14,4)	NOT NULL	ราคา CPU ต่อ 1 core
	created_at	Datetime(6)	NOT NULL	วันที่สร้างราคากลางนี้
	memory_price	Decimal(14,4)	NOT NULL	ราคา Ram ต่อ 1 GB
	volume_price	Decimal(14,4)	NOT NULL	ราคา Storage ต่อ 1 GB
	provider	Enum	NOT NULL	ผู้ให้บริการคลาวด์ที่จะใช้ราคานี้ (“OPENSTACK”, “PROXMOX”)
	updated_at	Datetime(6)	NOT NULL	วันเวลาที่ล่าสุดที่แก้ไขราคา

ตารางที่ 3.29 Data Dictionary : instance_resources_usage

Key	Column Name	Data Type	Nullable	Description
PK	id	UUID	NOT NULL	ID ที่เป็น Unique
	cpu	Int(11)	NOT NULL	จำนวน vCPU มีหน่วยเป็น Core
	memory	Int(11)	NOT NULL	จำนวน Ram มีหน่วยเป็น GB
FK	instance_id	UUID	NOT NULL	ID ของ Instance ที่ใช้ Spec นี้
FK	resource_cost_id	UUID	NOT NULL	ID ของราคากลาง ที่ Spec นี้ กำลังอ้างอิงในการคิดราคา

ตารางที่ 3.30 Data Dictionary : instances

Key	Column Name	Data Type	Nullable	Description
PK	id	UUID	NOT NULL	ID ที่เป็น Unique
FK	project_id	UUID	NOT NULL	ID ของโครงการที่ Instance นี้ อยู่
	name	Varchar(255)	NOT NULL	ชื่อของ Instance
	created_at	Datetime(6)	NOT NULL	วันที่ Instance นี้สร้าง
	last_charged_at	Datetime(6)	NOT NULL	วันเวลาครั้งล่าสุดที่มีการเก็บ ค่าบริการ
	expires_at	Datetime(6)	NOT NULL	วันเวลาหมดอายุของ Instance
FK	plan_id	UUID	NULL	แพ็คเกจที่ Instance นี้ใช้งาน อยู่
	provider_id	Varchar(255)	NOT NULL	ID ของ Instance จากฝั่งผู้ ให้บริการคลาวด์ ใช้สำหรับ เชื่อมโยงข้อมูล

ตารางที่ 3.31 Data Dictionary : volumes

Key	Column Name	Data Type	Nullable	Description
PK	id	UUID	NOT NULL	ID ที่เป็น Unique
	size	Int(11)	NOT NULL	จำนวน GB ของ Volume
	created_at	Datetime(6)	NOT NULL	วันที่สร้าง Volume
	expires_at	Datetime(6)	NOT NULL	วันหมดอายุของ Volume
	last_charged_at	Datetime(6)	NOT NULL	วันที่ครั้งสุดท้ายที่มีการเก็บค่าบริการ
FK	parent_id	Varchar(255)	NULL	ID ของ Volume ในกรณีที่ Volume นี้เกิดจากการ Clone มา
FK	project_id	Varchar(255)	NOT NULL	ID ของโครงการที่ Volume นี้ อยู่
FK	resource_cost_id	UUID	NOT NULL	ID ของราคากลางที่ใช้คิดค่าบริการ Volume นี้
	provider_id	Varchar(255)	NULL	ID ของ Volume จากฝั่งผู้ให้บริการคลาวด์ ใช้สำหรับ เชื่อมโยงข้อมูล
FK	instance_id	UUID	NULL	ID ของ Instance ที่ Volume นี้เชื่อมต่ออยู่
	name	Varchar(255)	NOT NULL	ชื่อของ Volume
	description	Varchar(255)	NULL	คำอธิบายของ Volume
	is_os_volume	Boolean	NOT NULL	Volume นี้เป็นที่เก็บของ OS หรือไม่

บทที่ 4

การพัฒนาระบบ

4.1 เครื่องมือที่ใช้พัฒนาระบบ

4.1.1 เครื่องมือ

- React Vite Typescript
- Kotlin Springboot
- Ant Design
- Node JS
- MariaDB
- DBeaver
- Git, Github Desktop
- Visual Studio Code
- Postman
- IntelliJ IDEA

4.1.2 ระบบปฏิบัติการ

- Windows 11 Home

4.1.3 เครื่องคอมพิวเตอร์ที่ใช้ในการพัฒนาระบบ

- Acer Nitro 5 Amd Ryzen 5 7535hs

4.1.4 Server ที่ใช้

- Debian 12
- vCPU 3
- RAM 8GB
- จำนวน 6 เครื่อง

4.2 การพัฒนาของโครงการงาน

4.2.1 การแสดงผลและจัดการโครงการ

4.2.1.1 การสร้างโครงการ

```
@Transactional
override fun createProject(
    request: CreateProjectRequest,
    token: String
): ResponseEntity<ApiResponse<Void>> {
    val payload = authService.verifyTokenSafer(token)
    when {
        payload.role == AccountRole.ADMIN && request.userId != null ->
            createProjectForUser(request, request.userId!!, token)
        payload.role == AccountRole.ADMIN && request.userId == null ->
            throw IllegalArgumentException("Admin must specify userId for project creation")
        payload.role == AccountRole.USER && request.userId != null ->
            throw IllegalArgumentException("Can't create project for others")
        else ->
            createProjectForUser(request, UUID.fromString(payload.id), token, payload)
    }
    return ResponseEntity.ok(ApiResponse("success", "project.createSuccess"))
}
```

ภาพที่ 4.1 ฟังก์ชันสร้างโครงการ

อธิบายภาพที่ 4.1

ฟังก์ชัน createProject รับข้อมูลโครงการประกอบด้วย ชื่อโครงการ (projectName) คำอธิบายโครงการ (projectDescription) และผู้ให้บริการคลาวด์ (provider) จากหน้าบ้านพร้อมแนบโทเคนเพื่อใช้ในการยืนยันตัวตนของผู้เรียกฟังก์ชัน หากผู้เรียกมีสิทธิ์เป็น Admin ต้องระบุ userId ของ User เพื่อสร้างโครงการให้ หากผู้เรียกมีสิทธิ์เป็น User จะใช้ userId ของผู้เรียกเองในการสร้าง โดยเรียกฟังก์ชัน createProjectForUser ต่อไป

```

private fun createProjectForUser(
    request: CreateProjectRequest,
    userId: UUID,
    token: String,
    payload: TokenDataSafer? = null
) {
    val owner = userRepository.findById(userId).orElseThrow { ItemNotFoundException("User not found") }
    val project = createBaseProject(request, owner)

    setupProviderResources(request, token, project, owner, payload)
    assignDefaultRoleAndMember(owner, project)

    if (payload == null) {
        notificationService.info(owner.account.id, "Project Service", "Admin created a project for you")
    }
}

```

ภาพที่ 4.2 ฟังก์ชันสร้างโครงการให้ผู้ใช้สิทธิ์ User

อธิบายภาพที่ 4.2

ฟังก์ชัน createProjectForUser รับข้อมูลจากฟังก์ชัน createProject เริ่มต้นด้วยการตรวจสอบว่า userId มีจริงในระบบ เรียกฟังก์ชันสร้างโครงการบนฐานข้อมูล (createBaseProject) ฟังก์ชันสร้างโครงการบนฝั่งผู้ให้บริการ setupProviderResources จากนั้นเรียกฟังก์ชันสร้าง Role และเพิ่มสมาชิกเริ่มต้นลงในโครงการ assignDefaultRoleAndMember ตามลำดับเมื่อดำเนินการเสร็จ หากผู้ดูแลระบบเป็นคนสร้างโครงการให้จะส่งการแจ้งเตือนไปบอกผู้ใช้นั้นด้วย

```

private fun createBaseProject(
    request: CreateProjectRequest,
    owner: UserEntity
): ProjectEntity {
    val quota = if (request.provider == Provider.PROXMOX) {
        quotaRepository.findFirstByIsDefaultTrueOrderByCreatedAtDesc()
    } else null

    return projectRepository.save(
        ProjectEntity(
            owner = owner.account,
            name = request.name,
            description = request.description,
            provider = request.provider,
            projectSourceId = null,
            quota = quota
        )
    )
}

```

ภาพที่ 4.3 ฟังก์ชันการสร้างโครงการบนฐานข้อมูล

อธิบายภาพที่ 4.3

โครงการบนฐานข้อมูลจะเก็บข้อมูลที่มีการเรียกใช้บ่อยๆ เพื่อความรวดเร็วในการใช้งาน เริ่มต้นด้วยการสร้างโควตา สำหรับผู้ให้บริการ Proxmox เนื่องจากระบบ Proxmox ไม่มีการจำกัดการสร้างคลาวด์ จากนั้นบันทึกข้อมูลโครงการลงในฐานข้อมูล โดยใส่ projectSourceId เป็น null ไว้ก่อนเพื่อรอการสร้างโครงจากฝั่งผู้ให้บริการต่อไป

```

private fun setupProviderResources(
    request: CreateProjectRequest,
    token: String,
    project: ProjectEntity,
    owner: UserEntity,
    payload: TokenDataSafer?
) {
    when (request.provider) {
        Provider.OPENSTACK -> {
            val (osUserId, email) = if (payload != null) {
                val id = if (payload.osUserId.isNullOrBlank() || payload.osUserId == "-") {
                    val newId = openStack.createAccount(payload.id, payload.email)
                    authService.updateUserAttribute(token, "osUserId", newId)
                    newId
                } else payload.osUserId
                id to payload.email
            } else {
                val user = authService.getUserDetail(owner.accountId.toString())
                val id = if (user.osUserId.isNullOrBlank() || user.osUserId == "-") {
                    val newId = openStack.createAccount(user.id, user.email)
                    authService.updateUserAttribute(token, "osUserId", newId, owner.accountId.toString())
                    newId
                } else user.osUserId
                id to user.email
            }
            val openStackProjectId = openStack.createProject(email, osUserId, project.id.toString())
            project.projectSourceId = openStackProjectId
        }
        Provider.PROXMOX -> null
    }
}

```

ภาพที่ 4.4 ฟังก์ชันการสร้างโครงการฝั่งของคลาวด์

อธิบายภาพที่ 4.4

เมื่อมีการสร้างโครงการใหม่ ระบบจะตรวจสอบว่าผู้ให้บริการ (Provider) ที่เลือกเป็น OpenStack หรือ Proxmox แล้วดำเนินการแตกต่างกัน หาก provider คือ Openstack จะดึง ID ผู้ใช้บน Openstack จากระบบยืนยันตัวตน หากไม่มีจะทำการสร้างบัญชีผู้ใช้ใหม่ให้ จากนั้นเรียก ฟังก์ชันสร้างโครงการบน Openstack `openstack.createProject()` เมื่อสร้างเสร็จจะได้รับ ID โครงการบน Openstack นำมาเก็บบนฟิลด์ `projectSourceId`

หาก provider คือ Proxmox จะไม่มีการเรียก API หรือสร้างโครงการใด ๆ เพิ่มเติมในระบบ Proxmox เนื่องจาก Proxmox ไม่ได้ใช้โครงสร้างแบบ Project ในการจัดการทรัพยากร การสร้าง และจัดการคลาวด์สามารถทำได้โดยตรงโดยไม่ต้องอยู่ภายใต้โครงการเหมือนกับ OpenStack

```
private fun assignDefaultRoleAndMember(owner: UserEntity, project: ProjectEntity) {
    val role = roleRepository.save(
        RoleEntity(
            project = project,
            permissions = Permission(
                project = PermissionSet(read = true, update = true, delete = true),
                cloud = PermissionSet(create = true, read = true, update = true, delete = true),
                member = PermissionSet(create = true, read = true, update = true, delete = true)
            ),
            isDefault = true,
            name = "Default"
        )
    )
    memberRepository.save(
        MemberEntity(
            id = MemberId(userId = owner.account.id, projectId = project.id),
            user = owner.account,
            project = project,
            role = role,
            invitedBy = null
        )
    )
}
```

ภาพที่ 4.5 ฟังก์ชันการสร้าง Role และเพิ่มสมาชิกเริ่มต้นในโครงการ

อธิบายภาพที่ 4.5

สร้าง Role เริ่มต้นบนฐานข้อมูล โดยกำหนดให้มีสิทธิ์ในการจัดการทั้งหมด และกำหนดฟิลด์ isDefault เป็น true เพื่อใช้ตรวจสอบไม่ให้อาจลบได้ภายหลัง จากนั้นสร้างข้อมูลสมาชิกใหม่ลงบนฐานข้อมูล

```

override fun createAccount(id: String, email: String): String {
    val osToken = getToken()
    val headers = HttpHeaders().apply {
        contentType = MediaType.APPLICATION_JSON
        set("X-Auth-Token", osToken)
    }
    val accountRequest = mapOf(
        "user" to mapOf(
            "domain_id" to config.domain.id,
            "enabled" to true,
            "name" to id,
            "password" to email,
            "email" to email
        )
    )
    val entity = HttpEntity(accountRequest, headers)
    val url = "${config.url.keystone}/v3/users"
    val response = restTemplate.exchange(
        url,
        HttpMethod.POST,
        entity,
        JsonNode::class.java
    )
    val userId = response.body?.path("user")?.path("id")?.asText()
    ?: throw InternalError("Can't create user in OpenStack")
    return userId
}

```

ภาพที่ 4.6 ฟังก์ชันการเรียก API สร้างบัญชีฝั่ง Openstack

อธิบายภาพที่ 4.6

เริ่มต้นด้วยการขอ Token ระดับผู้ดูแลระบบของ Openstack เพื่อใช้ในการขอสร้างบัญชีผู้ใช้ใหม่ นำไปใส่ใน header ในชื่อ X-Auth-Token และสร้าง requestBody จากนั้นส่ง request ไปที่ API ของ Openstack เพื่อสร้างบัญชีผู้ใช้

```

override fun assignProject(
    projectId: String,
    osUserId: String
) {
    val token = getToken()
    val headers = HttpHeaders().apply {
        contentType = MediaType.APPLICATION_JSON
        set("X-Auth-Token", token)
    }
    val url =
        "${config.url.keystone}/v3/projects/${projectId}/users/${osUserId}/roles/${config.memberRoleId}"
    val selectEntity = HttpEntity<Void>(headers)
    restTemplate.exchange(
        url,
        HttpMethod.PUT,
        selectEntity,
        JsonNode::class.java
    )
}

```

ภาพที่ 4.7 ฟังก์ชันการเพิ่มสมาชิกเข้าโครงการฝั่ง Openstack

อธิบายภาพที่ 4.7

เริ่มต้นด้วยการขอ Token ระดับผู้ดูแลระบบของ Openstack นำไปใส่ใน header ในชื่อ X-Auth-Token และสร้าง requestBody จากนั้นส่ง request ไปที่ API ของ Openstack เพิ่มสมาชิกลงในโครงการ

4.2.1.2 การแสดงโครงการที่ผู้ใช้เป็นสมาชิกอยู่

```

override fun getMyProjects(
    token: String
): ResponseEntity<ApiResponse<List<MyProjectList?>>> {
    val payload = authService.verifyToken(token)
    val userId = payload.data.path("sub")?.asText()
    require(!userId.isNullOrEmpty()) { "Missing 'user id' in token payload" }
    val userUUID = UUID.fromString(userId)
    val list = memberRepository.findMyProjects(userUUID)
    return ResponseEntity
        .ok()
        .body(
            ApiResponse(
                "success",
                "Project List",
                list
            )
        )
}

```

ภาพที่ 4.8 ฟังก์ชัน getMyProjects

อธิบายภาพที่ 4.8

ฟังก์ชันดึงข้อมูลรายการโครงการที่ผู้ใช้เป็นสมาชิกอยู่ โดยยืนยันตัวตนและได้รับข้อมูลผู้เรียกผ่านฟังก์ชัน authService.verifyToken() นำข้อมูล userId ไปค้นหาในตาราง members บนฐานข้อมูล และส่งรายการกลับไป

4.2.1.3 การเลือกโครงการ

```
@Transactional
override fun switch(
    token: String,
    projectId: String
): ResponseEntity<ApiResponse<SwitchProjectResponse>> {
    val payload = authService.verifyTokenSafer(token)
    val project = projectRepository
        .findById(UUID.fromString(projectId))
        .orElseThrow {
            EntityNotFoundException("Project with ID $projectId not found")
        }
    authService.switchProject(token, projectId)
    val xAuthToken = if (
        project.provider == Provider.OPENSTACK &&
        project.status != ProjectStatus.INACTIVE
    ) {
```

ภาพที่ 4.9 ฟังก์ชันเปลี่ยนโครงการ

อธิบายภาพที่ 4.9

ฟังก์ชันเลือกโครงการที่ขอมา โดยยืนยันตัวตนและได้รับข้อมูลผู้เรียกผ่านฟังก์ชัน `authService.verifyTokenSafer()` เริ่มต้นด้วยการตรวจสอบว่าโครงการมีอยู่จริงในฐานข้อมูล หากมีอยู่จริงจะตรวจสอบว่าเป็นโครงการของอะไร ซึ่งถ้าเป็นโครงการของ Openstack จะต้องขอ X-Auth-Token (token สำหรับเรียก API ครั้งต่อ ๆ ไปภายในโครงการ)

```

        openStack.switchProject(token, project.projectSourceId!!)
    } else null

    if (payload.role === AccountRole.USER) {
        updateProjectRole(token)
    }

    return ResponseEntity
        .ok(
            ApiResponse(
                "success", "Switching project",
                SwitchProjectResponse(
                    xAuthToken = xAuthToken,
                    provider = project.provider.toString()
                )
            )
        )
    }
}

```

ภาพที่ 4.10 ฟังก์ชันเปลี่ยนโครงการ (ต่อ)

อธิบายภาพที่ 4.10

หากไม่ใช่โครงการของ Openstack จะไม่ต้องขอ Token สุดท้ายตรวจสอบว่าสิทธิ์ของผู้ใช้เป็น User หรือไม่ ถ้าใช่จะต้องเรียก updateProjectRole() เพื่ออัปเดตข้อมูล Role ในโครงการของผู้ใช้

```

private fun updateProjectRole(token: String): RoleEntity {
    val payload = authService.verifyTokenSafer(token)
    val projectId = payload.projectId
    val userId = payload.id
    val member = memberRepository.findByProjectIdAndUserId(UUID.fromString(projectId),
        UUID.fromString(userId))
        .orElseThrow { ItemNotFoundException("User not found in this project") }
    authService.updateUserAttribute(token, "projectRole", member.role.id)
    return member.role
}

```

ภาพที่ 4.11 ฟังก์ชัน updateProjectRole

อธิบายภาพที่ 4.11

ฟังก์ชันอัปเดต Role ในโครงการทำหน้าที่ดึง Role ของผู้ใช้ที่อยู่ในโครงการใหม่ทีเลือก มาใส่ให้กับระบบยืนยันตัวตน สำหรับการเรียกใช้งานต่อไปภายในโครงการ โดยดึงข้อมูลโครงการที่ใช้งานอยู่จากระบบยืนยันตัวตน นำไปค้นหาในฐานข้อมูลว่า

4.2.1.4 การแก้ไขข้อมูลโครงการ

```
@Transactional
override fun updateProject(
    request: UpdateProjectRequest,
    token: String
): ResponseEntity<ApiResponse<String>> {
    val payload = authService.verifyToken(token)
    val projectId = payload.data.path("projectId")?.asText()
    val projectUUID = UUID.fromString(projectId)
    val project = projectRepository
        .findById(projectUUID)
        .orElseThrow { ItemNotFoundException("Project $projectId not found") }
    if (!request.projectName.isNullOrBlank()) {
        project.name = request.projectName
    }
    if (!request.projectDescription.isNullOrBlank()) {
        project.description = request.projectDescription
    }
    return ResponseEntity.ok(
        ApiResponse(
            "success",
            "project.edit.success",
        )
    )
}
```

ภาพที่ 4.12 ฟังก์ชัน updateProject

อธิบายภาพที่ 4.12

ฟังก์ชันแก้ไขข้อมูลโครงการ โดยยืนยันตัวตนและได้รับข้อมูลผู้เรียก ผ่านฟังก์ชัน authService.verifyToken() เริ่มต้นด้วยการตรวจสอบว่าโครงการมีอยู่จริงในฐานข้อมูล จากนั้นแก้ไขข้อมูลใหม่ลงบนฐานข้อมูล

4.2.1.5 การลบโครงการ

```
@Transactional
override fun deleteProject(token: String, projectId: String?): ResponseEntity<ApiResponse<String>>
{
    val payload = authService.verifyTokenSafer(token)
    val pid = if (payload.role === AccountRole.ADMIN && !projectId.isNullOrEmpty()) {
        projectId
    } else payload.projectId
    val projectUUID = UUID.fromString(pid)
    val project = projectRepository.findById(projectUUID)
        .orElseThrow { EntityNotFoundException("Project ${pid} not found") }
    val instanceExists = instanceRepository.existsByProjectId(projectUUID)
    val volumeExists = volumeRepository.existsByProjectId(projectUUID)
    if (instanceExists || volumeExists) {
        val reasons = mutableListOf<String>()
        if (instanceExists) reasons.add("terminate all instances")
        if (volumeExists) reasons.add("delete all volumes")
        val reasonText = reasons.joinToString(" and ")
        throw BadRequestException("You must $reasonText before deleting this project.")
    }
}
```

ภาพที่ 4.13 ฟังก์ชัน deleteProject

อธิบายภาพ 4.13

ฟังก์ชันลบโครงการ โดยยืนยันตัวตน และรับข้อมูลสิทธิ์ และโครงการของผู้เรียก ผ่านฟังก์ชัน authService.verifyToken() เริ่มต้นด้วยการกำหนด ID ของโครงการที่จะลบ หากเป็น Admin จะใช้ ID ที่ส่งมา แต่หากไม่ใช่จะใช้ ID ของโครงการที่ใช้งานอยู่ ตรวจสอบว่าโครงการมีอยู่จริงในฐานข้อมูล ตรวจสอบว่ามี Instance หรือ Volume ใช้งานอยู่ภายในโครงการหรือไม่ หากมีอยู่จะไม่ให้ลบและส่งกลับไปที่หน้าบ้าน

```

memberRepository.deleteAllByProjectId(project.id)
roleRepository.deleteAllByProjectId(project.id)
project.status = ProjectStatus.INACTIVE
when (project.provider) {
    Provider.OPENSTACK -> {
        openStack.deleteProject(project)
    }
    Provider.PROXMOX -> null
}
return ResponseEntity.ok(
    ApiResponse(
        "success",
        "project.removeProject.success",
    )
)
}

```

ภาพที่ 4.14 ฟังก์ชัน deleteProject (ต่อ)

อธิบายภาพที่ 4.14

ลบข้อมูลสมาชิก และ Role ทั้งหมดในโครงการออกจากฐานข้อมูล จากนั้นเรียกฟังก์ชันลบโครงการฝั่งผู้ให้บริการ Openstack และเปลี่ยนสถานะโครงการเป็น INACTIVE โดยจะเก็บข้อมูลโครงการไว้ในฐานข้อมูลเพื่อเก็บข้อมูลการใช้จ่ายไว้

```

override fun leaveProject(
    token: String,
    projectId: UUID
): ResponseEntity<ApiResponse<Void>> {
    val payload = authService.verifyTokenSafer(token)
    val project = projectRepository.findByIdAndStatus(projectId, ProjectStatus.ACTIVE)
        .orElseThrow { ItemNotFoundException("errors.projectNotFound") }
    if (project.owner.id == UUID.fromString(payload.id)) {
        throw IllegalArgumentException("errors.ownerCantLeaveProject")
    }
    val memberId = MemberId(
        projectId = project.id,
        userId = UUID.fromString(payload.id)
    )
    if (!memberRepository.existsById(memberId)) {
        throw ItemNotFoundException("Member not exist in this project")
    }
    when (project.provider) {
        Provider.OPENSTACK -> {
            if (project.projectSourceId != null) {
                openStack.removeMember(project.projectSourceId!!, payload.osUserId)
            }
        }
        Provider.PROXMOX -> null
    }
    memberRepository.deleteById(memberId)
    return ResponseEntity.ok().body(
        ApiResponse(
            "success",
            "project.leaveProject.success"
        )
    )
}

```

ภาพที่ 4.15 ฟังก์ชัน leaveProject

อธิบายภาพ 4.15

ฟังก์ชันออกจากโครงการ ยืนยันตัวตนและได้รับข้อมูลผู้เรียก ผ่านฟังก์ชัน `authService.verifyTokenSafer()` เริ่มต้นด้วยการค้นหาว่า ID ของโครงการที่ขอมามีอยู่จริงและสถานะเป็น ACTIVE อยู่ จากนั้นตรวจสอบว่าผู้เรียกไม่ได้เป็นเจ้าของโครงการเอง หากเป็นจะต้องมอบสิทธิ์เจ้าของโครงการให้สมาชิกคนอื่นก่อน แล้วจึงเรียก API ใหม่ ตรวจสอบว่าเป็นสมาชิกในโครงการนี้จริง เรียกฟังก์ชันลบสมาชิกออกจากโครงการในฝั่งของ Openstack จากนั้นลบข้อมูลสมาชิกออกจากฐานข้อมูล

4.2.1.6 การเปลี่ยนเจ้าของโครงการ

```
@Transactional
override fun transferOwner(
    request: TransferProjectRequest,
    token: String
): ResponseEntity<ApiResponse<String>> {
    val payload = authService.verifyTokenSafer(token)
    val projectUUID = UUID.fromString(payload.projectId)
    val project = projectRepository.findByIdAndStatus(projectUUID,
        ProjectStatus.ACTIVE)
        .orElseThrow { EntityNotFoundException("Project not found") }
    val newOwner = memberRepository.findById(
        MemberId(
            projectId = projectUUID,
            userId = request.userId
        )
    ).orElseThrow { ItemNotFoundException("The user to transfer ownership to is not a member of
the project yet") }
    val updateProject = project.copy(owner = newOwner.user)
    projectRepository.save(updateProject)
    return ResponseEntity.ok(
        ApiResponse(
            "success",
            "Project owner has been transfer to ${newOwner.user.email}"
        )
    )
}
```

ภาพที่ 4.16 ฟังก์ชันเปลี่ยนเจ้าของโครงการ

อธิบายภาพ 4.16

ฟังก์ชันเปลี่ยนเจ้าของโครงการ ยืนยันตัวตนและได้รับข้อมูลผู้เรียก ผ่านฟังก์ชัน `authService.verifyTokenSafer()` เริ่มต้นด้วยการค้นหาว่า ID ของโครงการที่ขอมามีอยู่จริง และสถานะเป็น ACTIVE อยู่ ค้นหาว่า `userId` ของเจ้าของคนใหม่เป็นสมาชิกอยู่ในโครงการ จากนั้นเปลี่ยนข้อมูลเจ้าของโครงการเป็นผู้ใช้คนใหม่ บันทึกข้อมูลที่แก้ไขลงฐานข้อมูล

4.2.2 การแสดงผลและจัดการ Role ภายในโครงการ

4.2.2.1 การแสดง Role ทั้งหมดภายในโครงการ

```
override fun getRoles(token: String): ResponseEntity<ApiResponse<List<RoleListResponse?>>> {
    val payload = authService.verifyToken(token)
    if (!payload.isValid) throw BadRequestException("Token is invalid or expired")
    val projectId = payload.data.path("projectId")?.asText()
    require(!projectId.isNullOrBlank()) { "Missing 'project id' in token payload" }
    val projectUUID = UUID.fromString(projectId)
    val list = roleRepository.findAllByProjectId(projectUUID)
    return ResponseEntity.ok().body(
        ApiResponse(
            "success",
            "List all roles inside project $projectId",
            list
        )
    )
}
```

ภาพที่ 4.17 ฟังก์ชัน `getRoles`

อธิบายภาพที่ 4.17

ฟังก์ชันดึงข้อมูลรายชื่อ Role ภายในโครงการเพื่อส่งให้หน้าบ้าน ยืนยันตัวตนและได้รับข้อมูลผู้เรียก ผ่านฟังก์ชัน `authService.verifyToken()` โดยนำข้อมูล ID ของโครงการที่ใช้งานอยู่ มาค้นหา Role ที่มี `projectId` ตรงกันจากนั้นส่งกลับไปใน `Response`

4.2.2.2 การสร้าง Role

```

override fun createRole(
    request: CreateRoleRequest,
    token: String
): ResponseEntity<ApiResponse<String>> {
    val payload = authService.verifyTokenSafer(token)
    val projectUUID = UUID.fromString(payload.projectId)
    val project = projectRepository
        .findById(projectUUID)
        .orElseThrow { ItemNotFoundException("errors.projectNotFound") }
    val role = RoleEntity(
        project = project,
        name = request.name,
        permissions = request.permission
    )
    val savedRole = roleRepository.save(role)
    return ResponseEntity.ok().body(
        ApiResponse(
            "success",
            "project.role.successCreate",
            savedRole.name
        )
    )
}

```

ภาพที่ 4.18 ฟังก์ชัน createRole

อธิบายภาพที่ 4.18

ฟังก์ชันรับข้อมูล Role เพื่อสร้างข้อมูล Role ใหม่ลงบนฐานข้อมูล ยืนยันตัวตนและได้รับข้อมูลผู้เรียก ผ่านฟังก์ชัน authService.verifyTokenSafer() โดยนำข้อมูล ID ของโครงการที่ใช้งานอยู่มาค้นหาว่าโครงการนี้มีอยู่ในฐานข้อมูลหรือไม่ ถ้าไม่เจอจะส่งข้อผิดพลาดกลับ ถ้าเจอจะสร้าง Object ข้อมูลของ Role ใหม่ขึ้นมาและบันทึกลงบนฐานข้อมูล จากนั้นส่งผลลัพธ์สำเร็จกลับไป

4.2.2.3 การตรวจสอบสิทธิ์ของ Role

```
@Before("@annotation(requireProjectPermission)")
fun checkPermission(
    joinPoint: JoinPoint,
    requireProjectPermission: RequireProjectPermission
) {
    val args = joinPoint.args
    val method = (joinPoint.signature as MethodSignature).method
    val parameterAnnotations = method.parameterAnnotations
    var token: String? = null
    for (i in args.indices) {
        for (annotation in parameterAnnotations[i]) {
            if (annotation is RequestHeader && annotation.value == "Authorization") {
                token = args[i] as? String
                break
            }
        }
    }
}
```

ภาพที่ 4.19 การตรวจสอบสิทธิ์ภายในโครงการ

อธิบายภาพที่ 4.19

ทุกๆ API ที่มีการใช้งานเกี่ยวข้องกับโครงการจะถูกเรียกฟังก์ชัน checkPermission ก่อนทุกครั้ง เพื่อตรวจสอบสิทธิ์ของผู้เรียก โดยจะเขียน @requireProjectPermission ไว้ด้านบนของฟังก์ชันทั้งหมดเพื่อเรียกใช้งานฟังก์ชัน checkPermission ดังตัวอย่าง ภาพที่ 4.28 เริ่มต้นดึง Header ของ Request ที่ชื่อว่า Authorization ซึ่งจะได้รับ AccessToken ออกมา หากไม่พบจะแจ้งว่าไม่พบ Authorization ใน Header

```

if (token == null) {
    throw IllegalArgumentException("Authorization header is missing")
}

val payload = authService.verifyTokenSafer(token)
if (payload.role == AccountRole.ADMIN) return
val account = accountRepository.findById(UUID.fromString(payload.id))
    .orElseThrow { ItemNotFoundException("User account not found") }
val userUUID = UUID.fromString(payload.id)
val projectUUID = UUID.fromString(payload.projectId)
val isOwner = projectRepository.existsByIdAndOwnerId(projectUUID, userUUID)
val member = memberRepository.findByProjectIdAndUserId(projectUUID, userUUID)
    .orElseThrow { AccessDeniedException("project.noAccess") }
if (requireProjectPermission.ownerOnly) {
    if (!isOwner) throw AccessDeniedException("project.ownerOnly")
    return
}

```

ภาพที่ 4.20 การตรวจสอบสิทธิ์ภายในโครงการ (ต่อ)

อธิบายภาพที่ 4.20

เมื่อได้ token แล้วนำไปเรียกขอข้อมูลด้วยฟังก์ชัน `authService.VerifyTokenSafer(token)` โดยเริ่มด้วยการตรวจสอบว่าผู้เรียกมี Role เป็น Admin หรือไม่ (Role ของระบบทั้งหมด ไม่ใช่เพียงแค่โครงการใด) หากเป็น Admin จะอนุญาตให้ผ่านทั้งหมด หากเป็น User จะตรวจสอบตามลำดับดังนี้

1. เป็นผู้ใช้ที่มีจริงในระบบหรือไม่
2. เป็นเจ้าของโครงการที่กำลังใช้งานหรือไม่ หากใช่จะอนุญาตให้ผ่านทั้งหมด
3. เป็นสมาชิกภายในโครงการที่กำลังใช้งานหรือไม่
4. ตรวจสอบสิทธิ์ของ API ที่เรียกว่าตรงตามสิทธิ์ที่มีหรือไม่

หากไม่ผ่านการตรวจสอบจะถูกยกเลิกและแจ้งข้อผิดพลาดกลับไปว่า “ไม่มีสิทธิ์เข้าถึง”

```

@GetMapping("")
fun getMyProjects(
    @RequestHeader("Authorization") token: String
): ResponseEntity<ApiResponse<List<MyProjectList?>>> {
    return projectService.getMyProjects(token)
}

@RequireProjectPermission(resource = "project", action = "update")
@PatchMapping("")
fun updateProject(
    @RequestHeader("Authorization") token: String,
    @RequestBody request: UpdateProjectRequest
): ResponseEntity<ApiResponse<String>> {
    return projectService.updateProject(request, token)
}

@RequireProjectPermission(resource = "owner", action = "delete", ownerOnly = true)
@DeleteMapping(value = ["", "/{projectId}"])
fun deleteProject(
    @RequestHeader("Authorization") token: String,
    @PathVariable(required = false) projectId: String?
): ResponseEntity<ApiResponse<String>> {
    return projectService.deleteProject(token, projectId)
}

```

ภาพที่ 4.21 ตัวอย่างการใช้งาน RequireProjectPermission

อธิบายภาพที่ 4.21

การใส่ Annotation สำหรับทุกๆ API ที่มีการใช้งานเกี่ยวกับโครงการโดยมีข้อมูลที่ต้องการได้แก่ส่วนที่ต้องการเข้าถึง (resource) ประกอบด้วย cloud member project และ owner การกระทำ (action) ประกอบด้วย create read update และ delete และสุดท้ายเป็นการกระทำที่มีเพียงเจ้าของโครงการหรือไม่ที่ทำได้ (ownerOnly) ประกอบด้วย true หรือ false

4.2.2.4 การแก้ไขข้อมูลของ Role

```

override fun updateRole(request: UpdateRoleRequest, token: String):
ResponseEntity<ApiResponse<Void>> {
    val payload = authService.verifyToken(token)
    if (!payload.isValid) throw BadRequestException("Token is invalid or expired")
    val role = roleRepository
        .findById(request.roleId)
        .orElseThrow { ItemNotFoundException("Role not found") }
    val updatedRole = role.copy(
        name = request.roleName ?: role.name,
        permissions = request.permissions ?: role.permissions
    )
    roleRepository.save(updatedRole)
    return ResponseEntity.ok().body(
        ApiResponse(
            "success",
            "Role ${role.name} updated"
        )
    )
}

```

ภาพที่ 4.22 ฟังก์ชัน updateRole

อธิบายภาพที่ 4.22

ฟังก์ชันรับข้อมูลที่แก้ไขของ Role ที่มีอยู่แล้วประกอบด้วยชื่อ (roleName) สิทธิ์ (permissions) ยืนยันตัวตนและได้รับข้อมูลผู้เรียก ผ่านฟังก์ชัน authService.verifyToken() โดยนำข้อมูล ID ของโครงการที่ใช้งานอยู่มาค้นหาว่าโครงการนี้มีอยู่ในฐานข้อมูลหรือไม่ ถ้าไม่เจอจะส่งข้อผิดพลาดกลับ ถ้าเจอจะค้นหาว่าในฐานข้อมูลมี Role ที่ต้องการแก้ไขไหม หากไม่มีจะส่งข้อผิดพลาดกลับ หากมีจะแก้ไขข้อมูลใหม่และบันทึกลงฐานข้อมูล จากนั้นส่งผลลัพธ์สำเร็จกลับไป

4.2.2.5 การลบ Role

```
@Transactional
override fun deleteRole(
    roleId: Int,
    token: String
): ResponseEntity<ApiResponse<Void>> {
    val role = roleRepository.findById(roleId).orElseThrow {
        ItemNotFoundException("errors.roleNotFound")
    }
    if (role.isDefault) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(
            ApiResponse(
                "error",
                "Cannot delete default role"
            )
        )
    }
}
```

ภาพที่ 4.23 ฟังก์ชัน deleteRole

อธิบายภาพที่ 4.23

ฟังก์ชันรับ ID ของ Role ที่ต้องการลบเพื่อเรียกกลับข้อมูล Role ในฐานข้อมูล เริ่มด้วยการค้นหา Role ที่ต้องการลบก่อนว่าพบในฐานข้อมูลหรือไม่ หากไม่เจอจะส่งข้อผิดพลาดกลับหากเจอจะตรวจสอบ role.isDefault (เป็น Role เริ่มต้นที่สร้างมาพร้อมกับโครงการ) เป็นจริงหรือไม่ ถ้าเป็นจริงจะไม่อนุญาตให้ลบ

```

val members = memberRepository.findByRoleId(role.id) ?: emptyList()
if (members.isNotEmpty()) {
    val defaultRole = roleRepository.findByProjectIdAndIsDefaultTrue(role.project.id)
        .orElseThrow { ItemNotFoundException("Default role not found") }
    members.forEach { member ->
        member.role = defaultRole
        memberRepository.save(member)
    }
}
roleRepository.deleteById(role.id)
return ResponseEntity.ok().body(
    ApiResponse(
        "success",
        "Role ${role.name} deleted successfully"
    )
)
}

```

ภาพที่ 4.24 ฟังก์ชัน deleteRole (ต่อ)

อธิบายภาพที่ 4.24

หากเป็นเท็จจะค้นหาวสมาชิกในโครงการที่ใช้ Role นี้อยู่และเปลี่ยน Role พวกเขาทั้งหมดเป็น Default สุดท้ายจะลบข้อมูล Role ออกจากฐานข้อมูลและส่งผลลัพธ์สำเร็จกลับ

4.2.3 การแสดงผลและจัดการสมาชิกในโครงการ

4.2.3.1 การแสดงสมาชิกในโครงการ

```

override fun getMembers(
    token: String,
    filter: MemberFilter,
    pageable: Pageable
): ResponseEntity<ApiResponse<Page<MembersResponse>>> {
    val payload = authService.verifyToken(token)
    if (!payload.isValid) throw BadRequestException("Token is invalid or expired")
    val projectId = payload.data?.path("projectId")?.asText()
    require(!projectId.isNullOrBlank()) { "Missing 'project id' in token payload" }
    val projectUUID = UUID.fromString(projectId)
    val spec = MemberSpecifications.buildSpecification(projectUUID, filter)
    val newPage = MemberSortSanitizer(pageable)
    val members = memberRepository.findAll(spec, newPage).map { m ->
        MembersResponse(
            id = m.id.userId.toString(),
            email = m.user.email,
            phone = m.user.userProfile?.phone,
            roleId = m.role.id,
            roleName = m.role.name,
            invitedBy = m.invitedBy?.email,
            memberSince = m.memberSince.toString(),
            status = m.status
        )
    }
    return ResponseEntity.ok(
        ApiResponse(
            "success",
            "members list in project $projectId",
            members
        )
    )
}

```

ภาพที่ 4.25 ฟังก์ชัน getMembers

อธิบายภาพที่ 4.25

ฟังก์ชันดึงข้อมูลจากฐานข้อมูลส่งกลับไปให้หน้าบ้าน โดยรับข้อมูลฟิลเตอร์ และหน้าของข้อมูล นำไปทำ Specification (การค้นหาข้อมูลโดยมีฟิลเตอร์) นำฟิลเตอร์ที่ได้ไปค้นหาในฐานข้อมูล และส่งข้อมูลกลับ

4.2.3.2 การส่งคำเชิญเข้าร่วมโครงการ

```
@Transactional
override fun invite(
    request: InviteMemberRequest,
    token: String
): ResponseEntity<ApiResponse<String>> {
    val payload = authService.verifyTokenSafer(token)
    val projectUUID = UUID.fromString(payload.projectId)
    val inviterUUID = UUID.fromString(payload.id)
    val projectRef = entityManager.getReference(ProjectEntity::class.java, projectUUID)
    val inviterRef = entityManager.getReference(AccountEntity::class.java, inviterUUID)
    val role = roleRepository.findByIdAndProjectId(request.userRoleId, projectUUID)
    ?: throw ItemNotFoundException("project.inviteForm.roleNotFound")
    val user = accountRepository.findByEmail(request userEmail)
    ?: throw ItemNotFoundException("project.inviteForm.userNotFound")
    val inviteld = MemberId(
        projectId = projectUUID,
        userId = user.id
    )
}
```

ภาพที่ 4.26 ฟังก์ชัน invite

อธิบายภาพที่ 4.26

ฟังก์ชันสำหรับบันทึกคำเชิญลงในฐานข้อมูล เรียก authService.verifyTokenSafer(token) เพื่อนำ ID ของโครงการที่ใช้งานอยู่ และ ID ของผู้เชิญออกมาและดึง Reference ออกมาจากฐานข้อมูล ค้นหา Role ที่เลือกว่ามีจริงในฐานข้อมูลไหม หากไม่มีแจ้งข้อผิดพลาดกลับไป ค้นหาผู้ถูกเชิญจากในฐานข้อมูล จากนั้นสร้าง ID ของ Member จาก ID ของโครงการ และ ID ของสมาชิก

```

if (memberRepository.existsById(
    MemberId(
        projectId = projectUUID,
        userId = user.id
    )
)) {
    throw IllegalArgumentException("project.inviteForm.userInProject")
}

val invitation = MemberEntity(
    id = inviteld,
    user = user,
    project = projectRef,
    invitedBy = inviterRef,
    status = MemberStatus.PENDING,
    role = role
)

memberRepository.save(invitation)

notificationService.info(
    invitation.user.id,
    "Project Invitation",
    "${invitation.invitedBy?.name} has invite you to project ${invitation.project.name}"
)

return ResponseEntity
    .ok()
    .body(
        ApiResponse(
            "success",
            "project.invitation.inviteSuccess",
            request.userEmail
        )
    )
}

```

ภาพที่ 4.27 ฟังก์ชัน invite (ต่อ)

อธิบายภาพที่ 4.27

นำ ID ที่สร้างมาใหม่ไปค้นหาว่าพบผู้ใช้เป็นสมาชิกโครงการอยู่แล้วหรือไม่ ถ้าไม่พบสร้างข้อมูลสมาชิกใหม่ขึ้นมาโดยตั้ง MemberStatus (สถานะสมาชิก) ให้เป็น PENDING บันทึกลงฐานข้อมูล และแจ้งเตือนไปยังผู้ถูกเชิญว่า “คุณถูกเชิญเข้าร่วมโครงการ”

4.2.3.3 การจัดการคำเชิญที่รอการตอบรับ

```

override fun notifyInvite(
    token: String,
    request: NotifyInviteRequest
): ResponseEntity<ApiResponse<Void>> {
    val id = MemberId(
        request.userId,
        request.projectId
    )
    val invitation = memberRepository.findByIdAndStatus(id, MemberStatus.PENDING).orElseThrow {
        ItemNotFoundException("Invitation not existed")
    }
    notificationService.info(
        invitation.id.userId,
        "Project Invitation",
        "${invitation.invitedBy?.name} has invite you to project ${invitation.project.name}"
    )
    return ResponseEntity.ok(
        ApiResponse(
            "success",
            "Re-sent invitation notification"
        )
    )
}

```

ภาพที่ 4.28 ฟังก์ชัน notifyInvite

อธิบายภาพที่ 4.28

ค้นหาว่ามีสมาชิกอยู่ในโครงการหรือไม่ (สมาชิกที่สถานะเป็น PENDING) ถ้ามีส่งการแจ้งเตือนซ้ำอีกที่

```

override fun cancelInvite(
    token: String,
    projectId: UUID,
    userId: UUID
): ResponseEntity<ApiResponse<String>> {
    val id = MemberId(
        userId,
        projectId
    )
    val invitation = memberRepository.findByIdAndStatus(id, MemberStatus.PENDING)
        .orElseThrow { ItemNotFoundException("errors.inviteNotFound") }
    memberRepository.delete(invitation)
    return ResponseEntity.ok(
        ApiResponse(
            "success",
            "project.invitation.inviteCancel",
            invitation.user.email
        )
    )
}

```

ภาพที่ 4.29 ฟังก์ชัน cancelInvite

อธิบายภาพที่ 4.29

ฟังก์ชันยกเลิกคำเชิญโดยค้นหาข้อมูลสมาชิกในโครงการที่ถูกเชิญอยู่ (MemberStatus เป็น PENDING) ถ้าพบทำการลบข้อมูลทิ้ง

4.2.3.4 การตอบรับคำเชิญเข้าร่วมโครงการ

```
@Transactional
override fun respondToInvitation(
    request: RespondInvitationRequest,
    token: String
): ResponseEntity<ApiResponse<String>> {
    val payload = authService.verifyTokenSafer(token)
    val inviteld = MemberId(
        userId = UUID.fromString(payload.id),
        projectId = request.projectId
    )
    val invitation = memberRepository.findByIdAndStatus(inviteld, MemberStatus.PENDING)
        .orElseThrow { ItemNotFoundException("errors.inviteNotFound") }
    val project = projectRepository.findByIdAndStatus(request.projectId, ProjectStatus.ACTIVE)
        .orElse(null) ?: run {
            memberRepository.delete(invitation)
            throw IllegalArgumentException("project.inviteForm.projectNotFound")
        }
}
```

ภาพที่ 4.30 ฟังก์ชัน respondToInvitation

อธิบายภาพที่ 4.30

ฟังก์ชันตอบรับการเชิญโครงการ เริ่มต้นโดยการนำข้อมูล ID ผู้เรียก API จากฟังก์ชัน `authService.verifyTokenSafer(token)` ไปค้นหาข้อมูลการสมาชิกโครงการว่ามีสถานะถูกเชิญอยู่หรือไม่ (`MemberStatus.PENDING`) และค้นหาว่าโครงการมีอยู่บนฐานข้อมูลหากไม่มีโครงการที่เปิดอยู่ (`ProjectStatus.ACTIVE`) จะลบข้อมูลการเชิญทิ้ง (ป้องกันไม่ให้เข้าร่วมโครงการที่ปิดไปแล้ว)

```

if (request.accepted) {
    invitation.status = MemberStatus.JOINED
    if (project.provider == Provider.OPENSTACK) {
        if (project.projectSourceId.isNullOrBlank()) {
            throw IllegalArgumentException("errors.conflictData")
        }
        val osUser = payload.osUserId
        val osUserId: String = if (osUser == "" || osUser == "-") {
            openStack.createAccount(invitation.user.id.toString(), invitation.user.email)
        } else osUser
        authService.updateUserAttribute(token, "osUserId", osUserId)
        openStack.assignProject(project.projectSourceId!!, osUserId)
    }
    if (invitation.invitedBy != null) {
        notificationService.info(
            invitation.invitedBy.id,
            "Project Invitation",
            "${invitation.user.name} accepted your invitation to project ${invitation.project.name}",
        )
    }
    return ResponseEntity.ok(
        ApiResponse(
            status = "success",
            message = "Invitation accepted"
        )
    )
}

```

ภาพที่ 4.31 ฟังก์ชัน respondToInvitation (ต่อ)

อธิบายภาพที่ 4.31

หากตอบรับคำเชิญเข้าร่วมโครงการ อัปเดตค่าสถานะสมาชิกเป็นเข้าร่วมแล้ว (MemberStatus.JOINED) ตรวจสอบว่าเป็นโครงการของ Openstack หรือไม่ หากใช่ตรวจสอบว่าผู้ใช้มีบัญชี Openstack แล้วหรือยังโดยค้นหาในข้อมูล บน payload (ได้จาก authService.verifyTokenSafer(token)) หากมี osUserId (ID ของผู้ใช้นบน Openstack) จะนำมาใช้

หากไม่มีจะสร้างให้ และบันทึกลงบนข้อมูลผู้ใช้งาน authService จากนั้นเพิ่มผู้ใช้เป็นสมาชิกโครงการในฝั่งของ Openstack ด้วย เมื่อเสร็จสิ้น แจ้งเตือนผู้เชิญว่าผู้ใช้นี้ได้ตอบรับคำเชิญแล้ว

```
else {
    memberRepository.deleteById(invitation.id)
    if (invitation.invitedBy != null) {
        notificationService.info(
            invitation.invitedBy.id,
            "Project Invitation",
            "${invitation.user.name} declined your invitation to project ${invitation.project.name}"
        )
    }
    return ResponseEntity.ok(
        ApiResponse(
            status = "success",
            message = "project.invitation.declineSuccess"
        )
    )
}
```

ภาพที่ 4.32 ฟังก์ชัน respondToInvitation (ต่อ)

อธิบายภาพที่ 4.32

หากปฏิเสธคำเชิญลบข้อมูลสมาชิกออกจากข้อมูลสมาชิกในโครงการ และแจ้งเตือนไปยังผู้เชิญว่าผู้ใช้นี้ได้ปฏิเสธคำเชิญเข้าร่วมโครงการ

4.2.3.5 การเปลี่ยน Role ของสมาชิกในโครงการ

```
@Transactional
override fun changeMemberRole(
    request: ChangeRoleRequest,
    token: String
): ResponseEntity<ApiResponse<String>> {
    val payload = authService.verifyTokenSafer(token)
    val projectUUID = UUID.fromString(payload.projectId)
    val member = memberRepository.findByProjectIdAndUserId(projectUUID, request.userId)
        .orElseThrow { ItemNotFoundException("Member not found for user=${request.userId}") }
    val newRole = roleRepository.findByIdAndProjectId(request.roleId, projectUUID)
        ?: throw ItemNotFoundException("Role not found in this project")
    val updatedMember = member.copy(role = newRole)
    memberRepository.save(updatedMember)
    authService.updateUserAttribute(token, "projectRole", newRole.id, member.user.id.toString())
    return ResponseEntity.ok(
        ApiResponse(
            "success", "Member role updated successfully"
        )
    )
}
```

ภาพที่ 4.33 ฟังก์ชัน changeMemberRole

อธิบายภาพที่ 4.33

นำข้อมูล ID ของโครงการที่ใช้งานอยู่จากฟังก์ชัน authService.verifyTokenSafer(token) นำไปค้นหาสมาชิกที่ต้องการเปลี่ยน Role ว่ามีในโครงการหรือไม่ จากนั้นค้นหาว่า Role ใหม่ที่ต้องการเปลี่ยนมีอยู่ในโครงการหรือไม่ หากพบทั้งคู่จะอัปเดตค่าลงบนฐานข้อมูล และอัปเดตค่า projectRole ใหม่บน authService

4.2.3.6 การลบสมาชิกออกจากโครงการ

```

override fun removeMember(
    userId: UUID,
    token: String
): ResponseEntity<ApiResponse<String>> {
    val payload = authService.verifyTokenSafer(token)
    if (userId == UUID.fromString(payload.id)) {
        throw IllegalArgumentException("errors.removeYourSelf")
    }
    val project = projectRepository.findByIdAndStatus(UUID.fromString(payload.projectId),
        ProjectStatus.ACTIVE)
        .orElseThrow { ItemNotFoundException("errors.projectNotFound") }
    val memberId = MemberId(
        projectId = project.id,
        userId = userId
    )
    val member = memberRepository.findById(memberId).orElseThrow {
        ItemNotFoundException("project.invitation.memberNotExist")
    }
}

```

ภาพที่ 4.34 ฟังก์ชัน removeMember

อธิบายภาพที่ 4.34

ฟังก์ชันลบสมาชิกออกจากโครงการโดยรับ ID ของสมาชิกที่ต้องการลบก่อน จากนั้นตรวจสอบว่าตรงกับ ID ของผู้เรียกเองหรือไม่ ถ้าตรงจะไม่อนุญาตให้ลบ หากไม่ตรงจะตรวจสอบว่าโครงการที่ต้องการลบสมาชิกมีอยู่จริงในฐานข้อมูลหรือไม่ จากนั้นจะค้นหาข้อมูลสมาชิกว่ามีอยู่ในโครงการนี้หรือไม่ หากไม่มีจะส่งข้อผิดพลาดกลับไป

```

when (project.provider) {
    Provider.OPENSTACK -> {
        if (project.projectSourceId != null) {
            val osUserId = authService.getUserDetail(userId.toString()).osUserId
            openStack.removeMember(project.projectSourceId!!, osUserId)
        }
    }
    Provider.PROXMOX -> null
}
memberRepository.deleteById(memberId)
return ResponseEntity.ok().body(
    ApiResponse(
        "success",
        "project.invitation.removeSuccess",
        member.user.email
    )
)
}

```

ภาพที่ 4.35 ฟังก์ชัน removeMember(ต่อ)

อธิบายภาพที่ 4.35

หากเป็นโครงการของ Openstack ตรวจสอบว่ามี ID โครงการฝั่ง Openstack (projectSourceId) จากนั้นส่งไปลบสมาชิกออกจากโครงการฝั่ง Openstack ด้วยฟังก์ชัน openStack.removeMember(project.projectSourceId, osUserId) หากเป็น Proxmox ไม่ต้องทำอะไรเพิ่มเติม หลังจากเสร็จสิ้น ลบข้อมูลสมาชิกออกจากรฐานข้อมูล และส่งผลลัพธ์สำเร็จ

4.2.4 การแสดงผลค่าใช้จ่ายในโครงการ

4.2.4.1 การแสดงคลาวด์ที่มีการเก็บเงินอยู่ในโครงการ

```

override fun getCurrentUsage(token: String): ResponseEntity<ApiResponse<CurrentUsage>> {
    val payload = authService.verifyTokenSafer(token)
    val projectUUID = UUID.fromString(payload.projectId)
    val instanceUsages = instanceRepository.findByProjectId(projectUUID).map {
        val hourlyCost = it.plan?.totalCost ?: it.resourceUsage?.toAmount() ?: BigDecimal.ZERO
        val monthlyCost = hourlyCost * BigDecimal(config.hourInMonth)
        Usage(
            id = it.id,
            name = it.name,
            costPerHour = hourlyCost,
            costPerMonth = monthlyCost,
            lastCharge = it.lastChargedAt,
            nextCharge = it.expiresAt
        )
    }
    val volumeUsages = volumeRepository.findByProjectId(projectUUID).map {
        Usage(
            id = it.id,
            name = it.name,
            costPerHour = it.toAmount(),
            costPerMonth = it.toAmount() * BigDecimal(config.hourInMonth),
            lastCharge = it.lastChargedAt,
            nextCharge = it.expiresAt
        )
    }
}

```

ภาพที่ 4.36 ฟังก์ชัน getCurrentUsage

อธิบายภาพที่ 4.36

ฟังก์ชันสำหรับดึงข้อมูลคลาวด์ที่กำลังใช้งานอยู่ในโครงการนั้น พร้อมทั้งคำนวณค่าใช้จ่ายรวมโดยประมาณส่งให้หน้าบ้าน เริ่มต้นด้วยนำข้อมูล ID ของโครงการที่ใช้งานอยู่จาก authService.verifyTokenSafer(token) นำไปค้นหา Instance ที่ใช้งานอยู่ในฐานข้อมูล จากนั้นทำการคำนวณ ค่าใช้จ่ายรายชั่วโมงจาก ถ้าเป็น Instance ที่ใช้งานแบบแพ็คเกจจะใช้ราคาแพ็คเกจทันที ถ้าเป็นแบบกำหนดสเปคเอง จะนำไปคำนวณในฟังก์ชัน toAmount() ของ resourceUsage ถ้าเป็น

Volume จะนำไปเข้าฟังก์ชัน toAmount() ของ Volume คำนวณค่าใช้จ่ายรายเดือน โดยนำค่าใช้จ่ายรายชั่วโมงไปคูณจำนวนชั่วโมงใน 1 เดือน (บริษัทใช้ค่าเป็น 720 ชั่วโมง/เดือน)

```
val instanceCategory: UsageCategory? = if (instanceUsages.isNotEmpty()) {
    UsageCategory(
        list = instanceUsages,
        hourlyTotal = instanceUsages.sumOf { it.costPerHour },
        monthlyTotal = instanceUsages.sumOf { it.costPerMonth }
    )
} else null

val volumeCategory: UsageCategory? = if (volumeUsages.isNotEmpty()) {
    UsageCategory(
        list = volumeUsages,
        hourlyTotal = volumeUsages.sumOf { it.costPerHour },
        monthlyTotal = volumeUsages.sumOf { it.costPerMonth }
    )
} else null

val currentUsage = CurrentUsage(
    instance = instanceCategory,
    volume = volumeCategory,
    license = null,
    hourlyTotal = (instanceCategory?.hourlyTotal ?: BigDecimal.ZERO) +
        (volumeCategory?.hourlyTotal ?: BigDecimal.ZERO),
    monthlyTotal = (instanceCategory?.monthlyTotal ?: BigDecimal.ZERO) +
        (volumeCategory?.monthlyTotal ?: BigDecimal.ZERO)
)

return ResponseEntity.ok(
    ApiResponse(
        "success", "current resource usage of project ${projectUUID}",
        data = currentUsage
    )
)
}
```

ภาพที่ 4.37 ฟังก์ชัน getCurrentUsage (ต่อ)

อธิบายภาพที่ 4.37

นำข้อมูลจากส่วนบนมาแปลงเป็นรูปแบบให้ตรงกับ Response ที่ต้องการ โดยนำข้อมูลของทุกแถวมารวมกัน และแนบรายการข้อมูลทั้งหมดไปด้วย ทำเหมือนกันทั้ง Instance และ Volume นำข้อมูลไปใส่เป็น CurrentUsage นำค่าใช้จ่ายรายชั่วโมงรวมกัน และค่าใช้จ่ายรายเดือนรวมกัน จากนั้นส่งกลับให้หน้าบ้าน

4.2.4.2 การแสดงตารางสรุปค่าใช้จ่ายในโครงการ

```
override fun getProjectBilling(
    token: String,
    filter: ProjectBillingFilter,
    pageable: Pageable
): ResponseEntity<ApiResponse<Page<ProjectBillingResponse>>> {
    val payload = authService.verifyTokenSafer(token)
    val projectId = if (payload.role == AccountRole.ADMIN && filter.projectId != null) {
        filter.projectId
    } else UUID.fromString(payload.projectId)
```

ภาพที่ 4.38 ฟังก์ชัน getProjectBilling

อธิบายภาพที่ 4.38

ฟังก์ชันสำหรับดึงข้อมูลค่าใช้จ่ายในโครงการ และคำนวณผลลัพธ์ในรูปแบบที่ต้องการ เริ่มต้นด้วยการรับข้อมูลผู้ใช้จาก authService.verifyTokenSafer(token) ใช้ ID โครงการโดยเงื่อนไขดังนี้ ถ้าเป็น Admin และแนบ ID โครงการมากับ Request ด้วยจะใช้ ID นั้น ถ้าหากเป็นผู้ใช้ธรรมดา หรือไม่ได้แนบ ID จะใช้ ID โครงการที่ใช้งานอยู่เท่านั้น

```

val results = when (filter.groupBy) {
    BillingGroupBy.DAY -> {
        billingHistoryRepository.findBillingByDay(projectId, filter.from, filter.to, pageable)
    }
    BillingGroupBy.MONTH -> {
        billingHistoryRepository.findBillingByMonth(projectId, filter.from, filter.to, pageable)
    }
    BillingGroupBy.YEAR -> {
        billingHistoryRepository.findBillingByYear(projectId, filter.from, filter.to, pageable)
    }
}
return ResponseEntity.ok(
    ApiResponse(
        "success",
        "",
        results
    )
)
}

```

ภาพที่ 4.39 ฟังก์ชัน getProjectBilling (ต่อ)

อธิบายภาพที่ 4.39

เรียกฟังก์ชันค้นหาข้อมูลจากฐานข้อมูลตามการเลือกผลรวมรายวัน รายเดือน หรือรายปี พร้อมใส่ช่วงเวลาที่เลือกลงไปด้วย ในกรณีที่ไม่ได้เลือกช่วง จะได้ผลลัพธ์เป็นทุกช่วงเวลาตั้งแต่มีการเก็บเงินครั้งแรกของโครงการ

```

override fun getAdminBillingStats(
    token: String,
    filter: AdminBillingFilter,
): ResponseEntity<ApiResponse<List<ProviderBillingGroup>>> {
    val effectiveTo = filter.to ?: LocalDate.now()
    val effectiveFrom = filter.from ?: when (filter.groupBy) {
        BillingGroupBy.DAY -> effectiveTo
            .minusMonths(1)
            .plusDays(1)
        BillingGroupBy.MONTH -> effectiveTo
            .minusYears(1)
            .plusMonths(1)
            .with(TemporalAdjusters.firstDayOfMonth())
        BillingGroupBy.YEAR -> effectiveTo
            .minusYears(10)
            .plusYears(1)
            .with(TemporalAdjusters.firstDayOfYear())
    }
}

```

ภาพที่ 4.40 ฟังก์ชัน getAdminBillingStats

อธิบายภาพที่ 4.40

ฟังก์ชันดึงข้อมูลการใช้จ่ายภายในระบบทั้งหมดจากฐานข้อมูล และนำมาแปลงให้เป็นรูปแบบที่เหมาะสมในการทำ Visualization โดยกำหนดจุดสิ้นสุดของช่วงเวลา ถ้าไม่ได้ขอมาก็ใช้เวลาปัจจุบัน ส่วนจุดเริ่มต้นของข้อมูลหากไม่ได้ใส่มาถ้าเป็นกลุ่มรายวัน จะใช้เป็น 1 เดือนก่อนหน้า ถ้าเป็นรายเดือน จะเป็น 1 ปีก่อนหน้า ถ้าเป็นรายปีจะเป็น 10 ปีก่อนหน้า

```

val providers = filter.provider?.let { listOf(it) } ?: Provider.entries.toList()
val data = providers.map { pv ->
    val results = when (filter.groupBy) {
        BillingGroupBy.DAY -> fillMissingDays(pv, effectiveFrom, effectiveTo)
        BillingGroupBy.MONTH -> fillMissingMonths(pv, effectiveFrom, effectiveTo)
        BillingGroupBy.YEAR -> fillMissingYears(pv, effectiveFrom, effectiveTo)
    }
    ProviderBillingGroup(
        provider = pv.toString(),
        data = results
    )
}
return ResponseEntity.ok(
    ApiResponse(
        "success",
        "",
        data
    )
)
}

```

ภาพที่ 4.41 ฟังก์ชัน getAdminBillingStats (ต่อ)

อธิบายภาพที่ 4.41

กำหนดผู้ให้บริการ ถ้าไม่ได้มีใน Request จะใช้ผู้ให้บริการที่มีทั้งหมด (Openstack, Proxmox) นำไปค้นหาในฐานข้อมูลตามกลุ่มที่ต้องการ และใส่ช่วงเวลาของข้อมูลเข้าไปด้วย สุดท้ายส่งผลลัพธ์กลับไปให้หน้าบ้าน

4.2.5 การเติมเครดิต

```

const createToken = (cardData: CardTokenParam): Promise<OmiseCreateResponseSuccessful>
=> {
  return new Promise((resolve, reject) => {
    if (!omise) {
      reject(new Error("Omise not initialized"))
      return;
    }
    const card = {
      name: cardData.holderName,
      number: `${cardData.number}`,
      expiration_month: cardData.expirationMonth,
      expiration_year: cardData.expirationYear,
      security_code: cardData.securityCode
    };
    omise.createToken(
      "card",
      card,
      (statusCode, response) => {
        if (statusCode !== 200 || response.object !== "token") return reject(response);
        resolve(response);
      }
    )
  })
}

```

ภาพที่ 4.42 ฟังก์ชัน createToken

อธิบายภาพที่ 4.42

ฟังก์ชันสำหรับสร้าง Token เพื่อใช้ในการสร้างการเรียกเก็บเงินจาก Omise มีไว้เพื่อใช้แปลงข้อมูลบัตรเครดิตให้เป็น Token เพื่อไม่ให้มีการส่งข้อมูลละเอียดอ่อนผ่านระบบของเรา โดยจะส่งจากหน้าบ้านไปหาเซิร์ฟเวอร์ของ Omise เมื่อได้รับ Token แล้วจึงนำไปใช้งานกับ API ของหลังบ้าน โดยเริ่มต้นจะตรวจสอบก่อนว่าโหนด Library ของ Omise แล้วหรือยัง ถ้ายังไม่โหนดจะแจ้งเตือนหากโหนดแล้วจะสร้างข้อมูลบัตรเครดิตขึ้นมาแล้วส่งไปที่เซิร์ฟเวอร์ของ Omise ผ่านฟังก์ชัน omise.createToken()

```

const createCharge = async (tokenOrSourceId: string): Promise<PaymentResponse> => {
  try {
    const method: Payment = topupForm.getFieldValue("method");
    const topupAmount = topupForm.getFieldValue("topupAmount");
    const wantTax = topupForm.getFieldValue("wantTax");
    const methodEnum: PaymentMethod = method == "promptpay" ? "PROMPTPAY" :
"CREDIT_CARD";
    const res = await
apiClient.post<ApiResponse<PaymentResponse>>("/api/payments/charge", {
      token: tokenOrSourceId,
      amount: topupAmount * 100,
      currency: "THB",
      method: methodEnum,
      wantTax
    });
    if (res.data.status === 'error' || !res.data.data) {
      throw new Error("Charge creation failed");
    }
    return res.data.data;
  } catch (error) {
    console.error("Error creating charge:", error);
    throw error;
  }
};

```

ภาพที่ 4.43 ฟังก์ชัน createCharge ฝั่งหน้าบ้าน

อธิบายภาพที่ 4.43

ฟังก์ชันสร้างการเรียกเก็บเงินจาก Omise รับค่าช่องทางการจ่าย (method) จำนวนเงินที่เต็ม (topupAmount) ต้องการใบกำกับภาษีไหม (wantTax) แปลงช่องทางการโอนให้เป็น Enum จากนั้นส่งไปสร้างที่หลังบ้านผ่าน API POST “/api/payments/charge” พร้อมแนบ Token ของบัตร และข้อมูลอื่นๆ หากมีข้อผิดพลาดแสดงการแจ้งเตือน “ไม่สามารถสร้างการเรียกเก็บเงินได้” หากสำเร็จ return ข้อมูลสำหรับพาไปจ่ายเงินเช่น URL ของหน้าเว็บบัตรเครดิต หรือ URL ของ QR CODE สำหรับสแกนจ่าย

```

@Transactional
override fun createCharge(
    token: String,
    request: PaymentRequest
): ResponseEntity<ApiResponse<PaymentResponse>> {
    val payload = authService.verifyToken(token)
    val userId = UUID.fromString(payload.data.path("sub")?.asText())
    val userRef = entityManager.getReference(UserEntity::class.java, userId)
    val charge = when (request.method) {
        PaymentMethod.PROMPTPAY -> handlePromptPayCharge(request.amount, request.token)
        PaymentMethod.CREDIT_CARD -> handleCreditCardFlow(token, request)
        else -> throw BadRequestException("")
    }
    val txn = paymentHistoryRepository.save(
        PaymentHistoryEntity(
            amount = request.amount.divide(BigDecimal(100)),
            direction = PaymentDirection.IN,
            user = userRef,
            method = request.method,
            status = PaymentStatus.PENDING,
            chargeId = charge.id,
            wantTax = request.wantTax,
        )
    )
    return ResponseEntity.ok(
        ApiResponse(
            "success",
            "Created charge successfully",
            buildPaymentResponse(charge, txn, restTemplate)
        )
    )
}

```

ภาพที่ 4.44 ฟังก์ชัน createCharge

อธิบายภาพที่ 4.44

ฟังก์ชันสร้างการเรียกเก็บเงินโดยรับข้อมูล ID ของผู้เรียกผ่าน `authService.verifyToken(token)` หากผู้ใช้ต้องการจ่ายผ่านช่องทางพร้อมเพย์ (Promptpay) จะเรียกฟังก์ชัน `handlePromptpayCharge()` โดยส่งจำนวนเงิน และ Token ที่ผู้ใช้ได้มาจาก Omise ไป หากต้องการจ่ายผ่านช่องทางบัตรเครดิต จะเรียกฟังก์ชัน `handleCreditCardFlow()` โดยส่ง Token ยืนยันตัวตน และ Request ทั้งหมดที่เรียกประกอบด้วยจำนวนเงิน และ Token ที่ได้จาก Omise โดยทั้ง 2 ฟังก์ชันจะทำหน้าที่ติดต่อกับ Omise หลังจากทำงานเสร็จจะกลับมาสร้างประวัติการเติมเงินในฐานข้อมูลโดยตั้งสถานะเริ่มต้นเป็น กำลังรอดำเนินการ (`PaymentStatus.PENDING`) เมื่อเสร็จสิ้นจะส่งข้อมูลการเรียกเก็บเงินกลับไป

```
private fun handlePromptPayCharge(amount: BigDecimal, source: String): Charge {
    val requestCharge: Request<Charge> = Charge.CreateRequestBuilder()
        .source(source)
        .amount(amount.multiply(BigDecimal("1.07")).setScale(0, RoundingMode.HALF_UP).toLong())
        .currency("THB")
        .build();
    return client.sendRequest(requestCharge)
}
```

ภาพที่ 4.45 ฟังก์ชัน `handlePromptpayChage`

อธิบายภาพที่ 4.45

ฟังก์ชันสำหรับดำเนินการสร้างการชำระเงินสำหรับช่องทางพร้อมเพย์ โดยสิ่งที่ต้องแนบไปด้วยในการส่งไปที่ Omise คือ source (เป็นสิ่งเดียวกันกับ Token แต่ช่องทางพร้อมเพย์จะเรียกว่า Source) จำนวนเงินโดยทำการคำนวณภาษี 7% และสกุลเงินซึ่งในระบบจะใช้หน่วยเป็น ไทยบาทเสมอ (THB) ทำการ return ผลลัพธ์จาก Omise กลับไปให้ฟังก์ชัน `createCharge()`

```
private fun handleCreditCardCharge(amount: BigDecimal, customerId: String, cardId: String? = null):
Charge {
    val charge = Charge.CreateRequestBuilder()
        .customer(customerId)
        .amount(amount.multiply(BigDecimal("1.07")).setScale(0, RoundingMode.HALF_UP).toLong())
        .currency("THB")
        .capture(true)
    if (!cardId.isNullOrEmpty()) {
        charge.card(cardId)
    }
    return client.sendRequest(charge.build())
}
```

ภาพที่ 4.46 ฟังก์ชัน handleCreditCardCharge

อธิบายภาพที่ 4.46

ฟังก์ชันสำหรับดำเนินการสร้างการชำระเงินสำหรับช่องทางบัตรเครดิต โดยสิ่งที่ต้องแนบประกอบไปด้วย ID ของลูกค้า (ID ผู้ใช้ฝั่ง Omise จะสร้างโดยอัตโนมัติเมื่อมีการชำระเงินครั้งแรก) จำนวนเงินโดยทำการคำนวณภาษี 7% และสกุลเงินซึ่งในระบบจะใช้หน่วยเป็น ไทยบาทเสมอ (THB) เพิ่มเงื่อนไขเพิ่มเติม หากมี ID ของบัตรเครดิตแนบมาด้วยหมายความว่ามีการเลือกบัตรที่ใช้ในการจ่ายจะกำหนดบัตรที่จะจ่ายเข้าไปในคำขอด้วย (โดยปกติจะใช้บัตรเริ่มต้นในการจ่ายหากไม่ได้กำหนด) ทำการ return ผลลัพธ์จาก Omise กลับไปให้ฟังก์ชัน createCharge()

```

const handlePayment = useCallback(async () => {
  try {
    setCardLoading(true);
    setLoading(true);
    let token: string = "";
    if (cards.length > 0) {
      await savedCardForm.validateFields();
      token = savedCardForm.getFieldValue("preferCard");
    } else {
      const values = await form.validateFields();
      const { id } = await createToken({
        holderName: values.holderName,
        number: values.number,
        expirationMonth: values.expirationMonth,
        expirationYear: values.expirationYear,
        securityCode: values.securityCode
      });
      token = id;
    }
    const response = await createCharge(token);
    if (response.status === "failed" && 'failureCode' in response) {
      const failureCode = response.failureCode || t('balance.topup.failed.unknown')
      notify.error({
        message: t('error'),
        description: t(`errors.${failureCode}`)
      })
    }
    return
  }
}

```

ภาพที่ 4.47 ฟังก์ชันเติมเงินสำหรับบัตรเครดิต

อธิบายภาพที่ 4.47

ฟังก์ชัน handlePayment สำหรับบัตรเครดิตเริ่มต้นด้วยการเปิดสถานะกำลังโหลด กำหนด Token ของบัตรที่จะใช้หากไม่เคยมีบัตรมาก่อนหน้าเติมเงินจะมีฟอร์มให้กรอกรายละเอียดบัตร และนำข้อมูลของบัตรไปสร้าง Token ผ่านฟังก์ชัน createToken หากมีอยู่แล้วจะมีรายการบัตรให้เลือก ซึ่งมี Token อยู่แล้ว นำไปใช้ต่อโดยการเรียกฟังก์ชัน createCharge(token)

ตรวจสอบว่าผลลัพธ์มีข้อผิดพลาดจาก Omise หรือไม่เช่น เงินไม่พอ บัตรถูกระงับ หรืออื่นๆ หากมีจะแจ้งเตือนข้อผิดพลาดนั้น และยกเลิกการทำงาน

```

    if ('authorizeUri' in response) {
        window.location.href = response.authorizeUri
    }

    const createdAt = new Date(response.createdAt);
    const expiration = new Date(createdAt.getTime() + 5 * 60 * 1000);
    setExpirationTime(expiration);
    await startPolling(response.id, expiration);
    closeTopupModal();
} catch (error: any) {
    if (error.errorFields) {
        return;
    }
    notify.error({ message: t('balance.topup.failed.unknown') });
} finally {
    setCardLoading(false);
    setLoading(false);
}
}, [cards.length])

```

ภาพที่ 4.48 ฟังก์ชันเติมเงินสำหรับบัตรเครดิต (ต่อ)

อธิบายภาพที่ 4.48

หากไม่มีข้อผิดพลาดจะตรวจสอบว่ามี URI สำหรับเปลี่ยนหน้าหรือไม่ (ในกรณีที่ต้องไปทำรายการต่อบนหน้าเว็บของบัตรเครดิต) หากมีจะพาไป จากนั้นกำหนดวันที่สร้าง และวันหมดอายุของการทำรายการ (5 นาทีถัดไป) หลังจากนั้นเรียกฟังก์ชัน startPolling หลังทำงานเสร็จปิดหน้าต่างเติมเงินลง สุดท้ายปิดสถานะกำลังโหลด

```

const handlePayment = async () => {
  try {
    setLoading(true);
    const { id } = await createSource();
    const response = await createCharge(id);
    if (response.status === "failed" && 'failureCode' in response) {
      const failureCode = response.failureCode || t('balance.topup.failed.unknown')
      notify.error({
        message: t('error'),
        description: t(`errors.${failureCode}`)
      })
    }
    return
  }
}

```

ภาพที่ 4.49 ฟังก์ชันเติมเงินสำหรับพร้อมเพย์

อธิบายภาพที่ 4.49

ฟังก์ชัน handlePayment สำหรับพร้อมเพย์ เริ่มต้นด้วยการเปิดสถานะกำลังโหลด ทำการสร้าง Token โดยของพร้อมเพย์ Omise จะเรียกว่า ID ของ Source นำไปเรียกฟังก์ชัน createCharge() ตรวจสอบว่าผลลัพธ์มีข้อผิดพลาดจาก Omise หรือไม่เช่น Qr Code หมดเวลา หรือธนาคารปฏิเสธ หากมีจะแจ้งเตือนข้อผิดพลาดนั้น และยกเลิกการทำงาน

```

    if ('qrCodeUri' in response) {
        setQrCodeUri(response.qrCodeUri);
        setChargeId(response.id);
        setAmount(response.amount);
        closeTopupModal();
    } else if ('authorizeUri' in response) {
        window.location.href = response.authorizeUri
    }

    const createdAt = new Date(response.createdAt);
    const expiration = new Date(createdAt.getTime() + 5 * 60 * 1000);
    setExpirationTime(expiration);
    await startPolling(response.id, expiration);
    setQrCodeUri("");
    closeTopupModal();
} catch (error: any) {
    notify.error({
        message: "Error",
        description: "Payment failed, please try again"
    })
} finally {
    setLoading(false);
}
};

```

ภาพที่ 4.50 ฟังก์ชันเติมเงินสำหรับพร้อมเพย์ (ต่อ)

อธิบายภาพที่ 4.50

ตรวจสอบผลลัพธ์ว่ามี URI ของ Qr Code สำหรับจ่ายเงินหรือไม่ หากมีจะเก็บไว้ที่ตัวแปร QrCodeUri ChargeId และ Amount ตามลำดับเพื่อใช้แสดงให้ผู้ใช้สแกน กำหนดวันสร้างรายการ และวันหมดอายุ (5 นาทีถัดไป) ทำการเรียกฟังก์ชัน startPolling หลังจากจบการทำงานล้างค่า Qr Code ออกและปิดหน้าต่างเติมเงินลง และสุดท้ายปิดสถานะกำลังโหลด

```

const startPolling = (id: number, expiration: Date): Promise<PaymentStatus> => {
  return new Promise((resolve, reject) => {
    const intervalId = setInterval(async () => {
      try {
        const res = await
apiClient.get<ApiResponse<PaymentStatus>>("/api/payments/charge", {
          params: { id }
        });
        if (res.data.status === "error") {
          clearInterval(intervalId);
          notify.error({ message: "Failed to get payment detail" });
          reject(new Error("Failed to get payment detail"));
          return;
        }
        if (expiration.getTime() < Date.now()) {
          clearInterval(intervalId);
          notify.error({ message: "Timeout" });
          reject(new Error("Timeout"));
          return;
        }
      }
    });
  });
}

```

ภาพที่ 4.51 ฟังก์ชัน startPolling

อธิบายภาพที่ 4.51

ฟังก์ชันสำหรับขอสถานะเติมเงิน จะขอสถานะการเติมเงินซ้ำ ๆ จนกว่าการเติมเงินเกิดข้อผิดพลาดหรือสำเร็จเท่านั้น โดยกำหนด Interval ให้มีการทำงานคือเรียก API GET “/api/payments/charge” โดยใส่ param เป็น ID ของการเรียกเก็บเงินที่ต้องการสถานะไป หากมีข้อผิดพลาด หรือเวลาการดำเนินการหมดจะหยุด Interval แจ้งเตือนข้อผิดพลาด และส่งข้อผิดพลาดกลับไปหาฟังก์ชัน handlePayment

```

const status = res.data.data;
if (status === 'SUCCESS') {
  clearInterval(intervalId);
  fetchCredit();
  notify.success({
    message: t('success'),
    description: t('balance.topup.result.success')
  })
  setTimeout(() => resolve(status), 1000);
} else if (status === 'FAILED') {
  clearInterval(intervalId);
  notify.error({
    message: t('error'),
    description: t('balance.topup.result.failed')
  })
  setTimeout(() => resolve(status), 1500);
}
} catch (err) {
  clearInterval(intervalId);
  reject(err);
}
}, 1000);
});
};

```

ภาพที่ 4.52 ฟังก์ชัน startPolling (ต่อ)

อธิบายภาพที่ 4.52

ตรวจสอบสถานะการชำระเงินแบบวนซ้ำตามช่วงเวลา โดยระบบจะส่งคำขอตรวจสอบสถานะไปยังเซิร์ฟเวอร์ทุก 1 วินาที ภายในฟังก์ชัน startPolling(). หากเซิร์ฟเวอร์ตอบกลับด้วยสถานะ SUCCESS จะทำการยุติการตรวจสอบทันที จากนั้นเรียกใช้ฟังก์ชัน fetchCredit() เพื่ออัปเดตยอดเครดิต และแสดงข้อความแจ้งเตือนการทำรายการสำเร็จ เมื่อแสดงข้อความครบแล้วจะรอประมาณ 1 วินาทีเพื่อให้ผู้ใช้สามารถสังเกตข้อความได้ชัดเจนก่อนสิ้นสุดกระบวนการ ในกรณีที่ได้รับสถานะ FAILED ระบบจะยุติการตรวจสอบเช่นเดียวกัน แล้วแสดงข้อความแจ้งเตือนข้อผิดพลาดพร้อมช่วงเวลา 1.5 วินาทีเพื่อให้ผู้ใช้รับทราบก่อนปิดการทำงาน แต่หากสถานะยังคงเป็น PENDING ระบบจะไม่หยุดการทำงาน และจะดำเนินการตรวจสอบซ้ำในรอบถัดไปทุก 1 วินาทีจนกว่าจะได้รับ

ผลเป็น SUCCESS หรือ FAILED. หากเกิดข้อผิดพลาดระหว่างกระบวนการ ระบบจะยุติการตรวจสอบและส่งคืนข้อผิดพลาดเพื่อใช้ในการจัดการต่อไป

```

override fun getCharge(
    token: String,
    paymentId: Long
): ResponseEntity<ApiResponse<PaymentStatus>> {
    val payload = authService.verifyToken(token)
    val userId = payload.data.path("sub")?.asText()
    val userUUID = UUID.fromString(userId)
    val payment = paymentHistoryRepository
        .findById(paymentId)
        .orElseThrow { IllegalArgumentException("Payment id not found") }
    if (payment.user.accountId != userUUID) {
        throw AccessDeniedException("You can't access this")
    }

    return ResponseEntity.ok(ApiResponse(
        "success",
        "",
        payment.status
    ))
}

```

ภาพที่ 4.53 ฟังก์ชันดึงข้อมูลสถานะของการเติมเงิน

อธิบายภาพที่ 4.53

ฟังก์ชัน getCharge จะถูกเรียกโดยฟังก์ชัน startPolling ของฝั่งหน้าบ้านเพื่อขอสถานะของการเติมเงิน เริ่มต้นด้วยการขอ ID ผู้เรียกจาก authService.verifyToken(token) นำไปค้นหาในฐานข้อมูล โดยตรวจสอบด้วยว่าผู้ขอเป็นเจ้าของข้อมูลนี้หรือไม่ หากใช่จะส่งเพียงสถานะกลับไปให้เท่านั้นประกอบด้วย “SUCCESS” “PENDING” และ “FAILED”

```

@Transactional
override fun webhookEvent(request: Map<String, Any>): ResponseEntity<Void> {
    val eventId = request["id"] as? String ?: return ResponseEntity.badRequest().build()
    val omiseEvent = verifyAndGetEvent(eventId)
        ?: return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build()
    val key = omiseEvent.key
    if (key == "charge.complete" || key == "charge.create") {
        try {
            processChargeEvent(omiseEvent)
        } catch (ex: Exception) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build()
        }
    }
    return ResponseEntity.ok().build()
}

```

ภาพที่ 4.54 ฟังก์ชัน Webhook รับข้อมูล Omise

อธิบายภาพที่ 4.54

ฟังก์ชัน `webhookEvent` จะทำงานเมื่อ Omise ส่งผลลัพธ์การเติมเงินของผู้ใช้มาหาหลังบ้านของระบบ โดยส่งมาในรูปแบบ Event เริ่มต้นด้วยการนำ `eventId` ไปตรวจสอบกับฟังก์ชัน `verifyAndGetEvent` ว่าเป็นของจริงหรือไม่หากเป็นของจริงจะมี `key` อยู่ภายใน event เพื่อใช้บอกสถานะของการเติมเงินโดยจะทำงานเฉพาะเมื่อ `key` เป็น `charge.complete` หรือ `charge.create` เท่านั้นโดยเรียกฟังก์ชัน `processChargeEvent` เพื่อดำเนินการต่อ

```

private fun verifyAndGetEvent(eventId: String): Event<Charge>? {
    return try {
        val request = Event.GetRequestBuilder(eventId).build()
        client.sendRequest(request) as Event<Charge>
    } catch (ex: Exception) {
        null
    }
}

```

ภาพที่ 4.55 ฟังก์ชัน `verifyAndGetEvent`

อธิบายภาพที่ 4.55

ฟังก์ชันที่ใช้สำหรับตรวจสอบว่า Event เป็นของจริงโดยการนำ `eventId` ไปเรียกขอข้อมูลที่ Omise หากได้รับข้อมูลกลับมาแสดงว่ามีจริง

```

@Transactional
private fun processChargeEvent(event: Event<Charge>) {
    val charge = event.data
    ?.throw IllegalArgumentException("Event data is not a Charge")
    val chargeId = charge.id
    val transaction = paymentHistoryRepository.findByChargeId(chargeId)
        .orElseThrow { IllegalArgumentException("No payment matching for charge id: $chargeId") }
    if (transaction.status != PaymentStatus.PENDING) {
        return
    }
    when (charge.status) {
        ChargeStatus.Successful -> {
            try {
                transaction.status = PaymentStatus.SUCCESS
                val user = transaction.user
                user.credit += transaction.amount
                user.totalSpend += transaction.amount
                user.thisMonthSpend += transaction.amount
                creditTxnRepo.save(CreditTransactionEntity(
                    type = TransactionType.TOPUP,
                    amount = transaction.amount,
                    direction = PaymentDirection.IN,
                    user = user,
                ))
            } catch (ex: DataIntegrityViolationException) {
                println("Duplicate credit transaction prevented: $chargeId")
                throw ex
            }
        }
    }
}

```

ภาพที่ 4.56 ฟังก์ชัน processChargeEvent

อธิบายภาพที่ 4.56

ฟังก์ชันสำหรับดำเนินการตามผลลัพธ์การเติมเงินที่ Omise ส่งมาเริ่มต้นด้วยการตรวจสอบก่อนว่าใน Event มีข้อมูลของการเรียกเก็บเงิน จากนั้นนำ ID ของการเรียกเก็บเงินไปค้นหาในฐานข้อมูล ตรวจสอบว่าสถานะของธุรกรรมไม่เป็น กำลังรอดำเนินการเท่านั้น (PENDING) จากนั้นตรวจสอบสถานะการเติมเงินถ้าเป็น Successful เปลี่ยนสถานะธุรกรรมในฐานข้อมูลให้เป็น SUCCESS และทำการเพิ่มเครดิตตามจำนวนเงินที่เติมให้ผู้ใช้คนนั้น ทำการอัปเดตข้อมูลการเติมเงินต่างๆ

```
ChargeStatus.Failed -> {
    transaction.status = PaymentStatus.FAILED
    transaction.failureCode = charge.failureCode
}
ChargeStatus.Expired -> {
    transaction.status = PaymentStatus.EXPIRED
}
else -> {
    return
}
}
paymentHistoryRepository.save(transaction)
}
```

ภาพที่ 4.57 ฟังก์ชัน processChargeEvent (ต่อ)

อธิบายภาพที่ 4.57

หากสถานะการเติมเงินเป็น Failed เปลี่ยนสถานะธุรกรรมในฐานข้อมูลเป็น FAILED พร้อมเก็บรหัสข้อผิดพลาด (failureCode) เอาไว้สำหรับการแสดงผล หากสถานะการเติมเงินเป็น Expired เปลี่ยนสถานะธุรกรรมในฐานข้อมูลเป็น EXPIRED หากเป็นสถานะการเติมเงินอื่นจะไม่ดำเนินการใด ๆ

```

override fun getCards(token: String): ResponseEntity<ApiResponse<List<CardDTO>>> {
    val payload = authService.verifyToken(token)
    val customerId = payload.data.path("omiseCustomerId").asText()
    if (customerId.isNullOrBlank()) {
        return ResponseEntity.ok(ApiResponse(
            "success",
            "empty card list",
            emptyList()
        ))
    }
    val customerRequest = Customer.GetRequestBuilder(customerId).build()
    val customer = client.sendRequest(customerRequest)
    val defaultCardId = customer.defaultCard
    val cardRequest: Request<ScopedList<Card>> = Card.ListRequestBuilder(customerId).build()
    val response = client.sendRequest(cardRequest)
    return ResponseEntity.ok(ApiResponse(
        "success",
        "card list",
        response.data.map { it.toCardDTO(defaultCardId) }
    ))
}

```

ภาพที่ 4.58 ฟังก์ชันขอข้อมูลบัตรเครดิตจาก Omise

อธิบายภาพที่ 4.58

ฟังก์ชันสำหรับขอข้อมูลบัตรเครดิตจาก Omise เพื่อส่งไปให้หน้าบ้าน เริ่มต้นด้วยการรับข้อมูลผู้เรียกจาก authService.verifyToken(token) นำ ID ลูกค้าบน Omise (omiseCustomerId) มาเรียกขอบัตรทั้งหมด หากไม่มี ID แสดงว่ายังไม่เคยเพิ่มบัตรมาก่อนจึงส่ง list ว่างกลับไป

```

override fun createCustomer(token: String, cardToken: String): String {
    val payload = authService.verifyToken(token)
    val customerEmail = payload.data.path("email").asText()
    val request: Request<Customer> = Customer.CreateRequestBuilder()
        .email(customerEmail)
        .card(cardToken)
        .build()
    val response = client.sendRequest(request)
    authService.updateUserAttribute(token, "omiseCustomerId", response.id)
    return response.id
}

```

ภาพที่ 4.59 ฟังก์ชันสร้างลูกค้าบน Omise

อธิบายภาพที่ 4.49

ฟังก์ชัน createCustomer ใช้เพื่อสร้างบัญชีให้ผู้ใช้บนฝั่ง Omise เพื่อใช้สำหรับเก็บข้อมูลบัตรเครดิต เนื่องจากข้อมูลละเอียดก่อนจะไม่ถูกเก็บไว้ที่ฐานข้อมูลเรา โดยฟังก์ชันนี้จะทำงานครั้งแรกเมื่อเติมเงินด้วยบัตรเครดิตครั้งแรก เริ่มต้นด้วยการรับข้อมูลผู้เรียก API ผ่าน authService.verifyToken() โดยนำ Email ของผู้เข้ามาเพื่อใช้สร้างบัญชีลูกค้าพร้อมกับ Token ของบัตรเครดิตใบแรกที่ใช้งานส่งไปที่เซิร์ฟเวอร์ของ Omise

```

override fun addCard(token: String, cardToken: String): ResponseEntity<ApiResponse<Void>> {
    val payload = authService.verifyToken(token)
    val customerId = payload.data.path("omiseCustomerId").asText()
    val request: Request<Customer> = Customer.UpdateRequestBuilder(customerId)
        .card(cardToken)
        .build()
    client.sendRequest(request)
    return ResponseEntity.ok(ApiResponse(
        "success",
        "Add card successfully"
    ))
}

```

ภาพที่ 4.60 ฟังก์ชันเพิ่มบัตรเครดิต

อธิบายภาพที่ 4.60

ฟังก์ชัน addCard ทำหน้าที่เพิ่มบัตรเครดิตใหม่เข้าไปในบัญชีลูกค้าใน Omise เริ่มต้นด้วยการรับข้อมูลผู้เรียกจาก authService.verifyToken(token) จากนั้นนำ ID ลูกค้าบน Omise

(omiseCustomerId) มาเรียกอัปเดตข้อมูลลูกค้า โดยการเพิ่ม Token บัตรเครดิตใหม่เข้าไป จากนั้นส่งคำขอไปที่เซิร์ฟเวอร์ Omise

```
override fun destroyCard(token: String, cardId: String): ResponseEntity<ApiResponse<Void>> {
    val payload = authService.verifyToken(token)
    val customerId = payload.data.path("omiseCustomerId").asText()
    val request = Card.DeleteRequestBuilder(customerId, cardId).build()
    client.sendRequest(request)
    return ResponseEntity.ok(ApiResponse(
        "success",
        "Deleted card detail from this customer account"
    ))
}
```

ภาพที่ 4.61 ฟังก์ชันลบบัตรเครดิต

อธิบายภาพที่ 4.61

ฟังก์ชัน destroyCard เป็นการลบข้อมูลบัตรเครดิตออกจากเซิร์ฟเวอร์ของ Omise เริ่มต้นด้วยการรับข้อมูลผู้เรียกผ่าน authService.verifyToken(token) นำ ID ลูกค้าบน Omise (omiseCustomerId) มาเรียกลบข้อมูลโดยการแนบ ID ของบัตรที่ต้องการลบจากนั้นส่งไปขอที่เซิร์ฟเวอร์ของ Omise

4.2.6 การเก็บค่าบริการคลาวด์ในระบบ

4.2.6.1 การเก็บค่าบริการตามเวลา

```
@Transactional
@Scheduled(cron = "0 0 0 * * *")
fun processCharging() {
    val now = LocalDateTime.now()
    val resources = instanceRepository.findAll() + volumeRepository.findAll()
    val userSummary = mutableMapOf<String, MutableList<BillingSummary>>()
    resources.forEach { resource ->
        try {
            val userProfile = resource.project.owner.userProfile
            if (userProfile != null) {
                val expiresAt = resource.expiresAt ?: return@forEach
                val duration = Duration.between(now, expiresAt)
                val minutesUntilExpire = duration.toMinutes()
                val creditAmount = userProfile.credit
```

ภาพที่ 4.62 ฟังก์ชันเก็บค่าใช้จ่ายตามเวลา

อธิบายภาพที่ 4.62

ฟังก์ชัน processCharging ถูกกำหนด crontab ให้ทำงานเวลาเที่ยงคืนของทุกวัน เริ่มต้นด้วยการเก็บเวลาปัจจุบัน จากนั้นดึงข้อมูล Instance และ Volume ทั้งหมดจากฐานข้อมูลมา ต่อมาสร้าง List สำหรับเก็บว่าต้องส่ง Email อะไรไปหาผู้ใช้บ้างเพื่อส่งต่องานเสร็จครั้งเดียว จากนั้นเริ่มต้นการวนลูปข้อมูล Instance และ Volume ทั้งหมด (Resource) โดยดึงข้อมูลผู้ใช้เจ้าของ Resource นั้นมารอไว้ หากวันหมดอายุไม่ได้ถูกกำหนดจะข้ามไป หาระยะเวลาที่เหลือจนกว่าจะหมดอายุ และแปลงค่าเป็นนาทีเก็บไว้ กำหนดตัวแปรจำนวนเงินที่เหลือของเจ้าของ Resource นี้

```

when {
    minutesUntilExpire <= 0 -> {
        val amount = resource.amount * BigDecimal(config.cycleHour)
        if (creditAmount < amount) {
            billingHelper.deleteResource(now, resource)
            notificationService.send(
                NotificationType.BILLING,
                target = resource.project.owner.id,
                title = "Failed to perform charging resource",
                message = "Not enough credit for ${resource.resourceType} '${resource.name}'
${Formatter.formatAmount(amount)} terminated",
                link = "/balance"
            )
            userSummary.getOrPut(resource.project.owner.email) { mutableListOf() }
                .add(BillingSummary(BillMailType.DELETE, resource))
        } else {
            billingService.charge(now, resource, amount)
            notificationService.send(
                NotificationType.BILLING,
                target = resource.project.owner.id,
                title = "Resource charging notify",
                message = "${resource.resourceType} '${resource.name}' was charged for
${Formatter.formatAmount(amount)}",
                link = "/billing"
            )
        }
    }
}

```

ภาพที่ 4.63 ฟังก์ชันเก็บค่าใช้จ่ายตามเวลา (ต่อ)

อธิบายภาพที่ 4.63

ถ้าจำนวนนาฬิกาที่เหลือน้อยกว่าเท่ากับ 0 (คลาวด์หมดอายุแล้ว) คำนวณราคาของคลาวด์ โดยการนำราคาคูณกับจำนวนชั่วโมง หากจำนวนเครดิตที่มีน้อยกว่าราคาที่ต้องจ่ายจะทำการลบคลาวด์ นั้นทันทีพร้อมส่งการแจ้งเตือนไปยังเจ้าของ และทำการเพิ่มข้อมูลที่จะแจ้งเตือนไปที่ userSummary ที่กำหนดไว้ก่อนหน้านี้ โดยใช้ Email ของเจ้าของคลาวด์ ประเภทของ Mail

BillingMailType.DELETE และข้อมูลคลาวด์ลงไป หากมีเครดิตพอจะดำเนินการหักเครดิตผ่าน billingService.charge() จากนั้นทำการแจ้งเตือน และเพิ่มข้อมูลในการส่ง Mail ต่อไป

```
else -> {
    val costFor7Days = resource.amount * BigDecimal(HOURS_IN_7_DAYS)
    if (creditAmount <= costFor7Days) {
        val oneDayAgoUuid = Uuids.startOf(now.minusDays(3)
            .atZone(ZoneOffset.UTC).toInstant().toEpochMilli())
        val notify = notificationRepository.findRecentBillingNotifications(
            resource.project.owner.id,
            oneDayAgoUuid,
            NotificationType.BILLING
        )
        if (notify.isEmpty()) {
            val hoursLeft = creditAmount.divide(resource.amount, 2,
                RoundingMode.HALF_UP)
            val daysLeft = hoursLeft.divide(BigDecimal(24), 1, RoundingMode.HALF_UP)
            notificationService.send(
                NotificationType.BILLING,
                target = resource.project.owner.id,
                title = "Low credit",
                message = "${resource.resourceType} '${resource.name}' has about
                $daysLeft days of credit remaining",
                link = "/balance"
            )
        }
        userSummary.getOrPut(resource.project.owner.email) { mutableListOf() }
            .add(BillingSummary(BillMailType.WARN, resource))
    }
}
```

ภาพที่ 4.64 ฟังก์ชันเก็บค่าใช้จ่ายตามเวลา (ต่อ)

อธิบายภาพที่ 4.64

กรณีที่จำนวนนาฬิกาที่เหลือมากกว่า 0 (ยังไม่หมดอายุ) ทำการคำนวณค่าใช้จ่ายสำหรับอีก 7 วัน ว่ามีเครดิตเหลือเพียงพอสำหรับใช้จ่ายไหม หากมีไม่พอค้นหาว่าภายใน 3 วันนี้เคยแจ้งเตือนไปแล้วหรือยัง

หากยังจะส่งการแจ้งเตือนไปบอกว่าเครดิตที่มีตอนนี้ไม่เพียงพอสำหรับค่าใช้จ่ายอีก 7 วัน และเพิ่มข้อมูลในการส่ง Mail ต่อไป

```

    }
  }
} catch (e: Exception) {
    e.printStackTrace()
}
}
userSummary.forEach {
    billingService.sendUserBillingSummaryMail(it.key, it.value)
}
}

```

ภาพที่ 4.65 ฟังก์ชันเก็บค่าใช้จ่ายตามเวลา (ต่อ)

อธิบายภาพที่ 4.65

หากเกิดข้อผิดพลาดจะแสดงข้อผิดพลาดบน Console ของระบบ จากนั้นดำเนินการส่ง Mail ไปยังผู้ใช้ทีละคนจากการบันทึกข้อมูลไว้ผู้ใช้ละ 1 Mail ผ่านฟังก์ชัน

billingService.sendUserBillingSummaryMail()

```

@Transactional
override fun charge(now: LocalDateTime, resource: BillableResource, amount: BigDecimal) {
    val owner = resource.project.owner.userProfile
    if (owner != null) {
        val nextExpires = now.plus(config.cycleHour.toLong(), ChronoUnit.HOURS)
        val leftCredit = owner.credit - amount
        if (leftCredit < BigDecimal.ZERO) {
            throw NotEnoughCredit("${leftCredit.abs()} credit required")
        }
    }
}

```

ภาพที่ 4.66 ฟังก์ชัน charge

อธิบายภาพที่ 4.66

ฟังก์ชันสำหรับหักเครดิตผู้ใช้ในการคิดค่าบริการ Resource ต่างๆ เริ่มโดยเก็บข้อมูลเจ้าของ Resource ไว้ที่ตัวแปร Owner กำหนดเวลาหมดอายุครั้งต่อไป คำนวณเครดิตที่เหลือหลังจากจ่ายแล้ว ถ้าน้อยกว่า 0 แจ้งเตือนว่าเครดิตไม่เพียงพอและยกเลิกการทำงาน

```

owner.credit = leftCredit
userRepository.save(owner)
resource.expiresAt = nextExpires
resource.lastChargedAt = now
val periodStart = now.toLocalDate()
val billingHistory = billingHistoryRepository
    .findByProjectIdAndPeriod(resource.project.id, periodStart)
    .orElseGet {
        BillingHistoryEntity(
            project = resource.project,
            period = periodStart,
            amount = BigDecimal.ZERO,
        )
    }
billingHistory.amount += amount
val billing = billingHistoryRepository.save(billingHistory)
creditTxnRepo.save(
    CreditTransactionEntity(
        resource = ResourceName(
            resourceType = resource.resourceType,
            name = resource.name,
        ),
        type = TransactionType.CHARGE,
        amount = amount,
        direction = PaymentDirection.OUT,
        user = resource.project.owner.userProfile!!,
        billing = billing,
    )
)
}
}

```

ภาพที่ 4.67 ฟังก์ชัน charge (ต่อ)

อธิบายภาพที่ 4.67

อัปเดตเครดิตเป็นหลังจากจ่ายแล้วจากนั้นบันทึกข้อมูล อัปเดตข้อมูลเวลาหมดอายุครั้งต่อไป และเวลาจ่ายครั้งล่าสุดเป็นปัจจุบัน ทำการค้นหารอบการเก็บเงินว่าวันนี้มีแล้วหรือยัง

ถ้ามีจะใช้ข้อมูลเดิมถ้าไม่มีจะสร้างใหม่ นำจำนวนเครดิตที่หักไปบวกเพิ่มในรอบการเก็บเงินที่จ่ายไป แล้วจากนั้นบันทึกข้อมูล ทำการสร้างข้อมูลธุรกรรมเครดิตจากการใช้งานเครดิตครั้งนี้ลงในฐานข้อมูล โดยระบุรายละเอียดว่าเป็นการหักเครดิตในการจ่ายค่าบริการคลาวด์

```
@Transactional
override fun cancel(
    now: LocalDateTime,
    resource: BillableResource
) {
    val expiresAt = resource.expiresAt
    val duration = Duration.between(now, expiresAt)
    val minutesUntilExpire = duration.toMinutes()
    val cycleMinute = config.cycleHour * 60
    val owner = resource.project.owner.userProfile
```

ภาพที่ 4.68 ฟังก์ชันยกเลิกคลาวด์

อธิบายภาพที่ 4.68

ฟังก์ชันในการยกเลิกคลาวด์ มีไว้เพื่อคืนเครดิต หรือเก็บเครดิตเพิ่มเติมเนื่องจากเวลาที่ซื้อคลาวด์อาจไม่ตรงกับเวลาที่ระบบตัดรอบ เริ่มต้นด้วยการเก็บค่าที่ใช้จนบอ่ยไว้ที่ตัวแปร ประกอบด้วยเวลาหมดอายุ ระยะเวลาที่เหลือ นาที่ที่เหลือ คำนวณจำนวนนาที่ใน 1 ชั่วโมง และข้อมูลเจ้าของคลาวด์

```
if (owner != null) {
    when {
        minutesUntilExpire < 0 -> {
            val overMinutes = minutesUntilExpire.absoluteValue
            val chargeAmount = resource.amount
                .multiply(BigDecimal(overMinutes))
                .divide(BigDecimal(60), 4, RoundingMode.HALF_UP)
            owner.credit -= chargeAmount
```

ภาพที่ 4.69 ฟังก์ชันยกเลิกคลาวด์ (ต่อ)

อธิบายภาพที่ 4.69

หากจำนวนนาที่ที่เหลือจนกว่าจะหมดอายุน้อยกว่า 0 (ใช้งานเกินเวลาที่จ่าย) ต้องทำการหักเครดิตเพิ่มเติม นำจำนวนนาที่ที่เหลือทำให้เป็นค่าบวก ไปคำนวณราคา (หาร 60 ให้เป็นชั่วโมง คูณราคาต่อชั่วโมง) หักเครดิตผู้ใช้ออกโดยไม่สนว่าเป็นค่าติดลบหรือไม่

```

creditTxnRepo.save(
    CreditTransactionEntity(
        resource = ResourceName(
            resourceType = resource.resourceType,
            name = resource.name,
            minute = overMinutes
        ),
        type = TransactionType.CHARGE_OVERDUE,
        amount = chargeAmount,
        direction = PaymentDirection.OUT,
        user = resource.project.owner.userProfile!!,
    )
)

```

ภาพที่ 4.70 ฟังก์ชันยกเลิกคลาวด์ (ต่อ)

อธิบายภาพที่ 4.70

ทำการสร้างข้อมูลธุรกรรมเครดิตจากการใช้งานเครดิตครั้งนี้ลงในฐานข้อมูล โดยระบุรายละเอียดว่าชื่อ ประเภทของคลาวด์ ระยะเวลาที่ใช้เกินมา พร้อมระบุประเภทด้วยว่าเป็นการหักเครดิตในการจ่ายค่าบริการคลาวด์เกินกำหนด

```

val periodStart = now.toLocalDate()

val billingHistory = billingHistoryRepository
    .findByProjectIdAndPeriod(resource.project.id, periodStart)
    .orElseGet {
        BillingHistoryEntity(
            project = resource.project,
            period = periodStart,
            amount = BigDecimal.ZERO
        )
    }

billingHistory.amount += chargeAmount
billingHistoryRepository.save(billingHistory)
}

minutesUntilExpire > 0 -> {
    val refundAmount = resource.amount
        .multiply(BigDecimal(minutesUntilExpire))
        .divide(BigDecimal(60), 4, RoundingMode.HALF_UP)

    owner.credit += refundAmount
}

```

ภาพที่ 4.71 ฟังก์ชันยกเลิกคลาวด์ (ต่อ)

อธิบายภาพที่ 4.71

ทำการค้นหาลอการเก็บเงินว่ามีข้อมูลของวันนี้มีแล้วหรือยัง ถ้ามีจะใช้ข้อมูลเดิมไปบวกจำนวนเงินเพิ่มเข้าไปถ้าไม่มีจะสร้างใหม่ นำจำนวนเครดิตที่หักไปบวกเพิ่มในลอการเก็บเงินที่จ่ายไปแล้วจากนั้นบันทึกข้อมูล หากจำนวนนาที่ที่เหลือมากกว่า 0 ต้องคืนเครดิตตามเวลาที่เหลือ คำนวณราคาคลาวด์ (นาที่ที่เหลือหาร 60 คูณราคาต่อชั่วโมง) อัปเดตจำนวนเครดิตให้กับเจ้าของคลาวด์นี้

```

if (resource.project.owner.userProfile != null) {
    creditTxnRepo.save(
        CreditTransactionEntity(
            resource = ResourceName(
                resourceType = resource.resourceType,
                name = resource.name,
                minute = minutesUntilExpire
            ),
            type = TransactionType.REFUND,
            amount = refundAmount,
            direction = PaymentDirection.IN,
            user = resource.project.owner.userProfile!!,
        )
    )
}

val periodStart = resource.lastChargedAt.toLocalDate().withDayOfMonth(1)
val billingHistory = billingHistoryRepository
    .findByProjectIdAndPeriod(resource.project.id, periodStart)
    .orElseGet {
        BillingHistoryEntity(
            project = resource.project,
            period = periodStart,
            amount = BigDecimal.ZERO
        )
    }
billingHistory.amount -= refundAmount
billingHistoryRepository.save(billingHistory)
}
}

```

ภาพที่ 4.72 ฟังก์ชันยกเลิกคลาวด์ (ต่อ)

อธิบายภาพที่ 4.72

ทำการคำนวณการเก็บเงินของวันที่เก็บเงินครั้งล่าสุด ถ้ามีจะใช้ข้อมูลเดิมถ้าไม่มีจะสร้างใหม่ นำจำนวนเครดิตที่หักไปบวกกลับออกในรอบการเก็บเงินที่จ่ายไปแล้วจากนั้นบันทึกข้อมูลทำการสร้างข้อมูลธุรกรรมเครดิตจากการคืนเครดิตครั้งนี้ลงในฐานข้อมูล โดยระบุรายละเอียดว่าเป็นการได้รับเครดิตคืนจากการยกเลิกคลาวด์ก่อนกำหนด

```

fun VolumeEntity.toAmount(): BigDecimal {
    val pricePerGB = resourceCost.volumePrice
    val volumeSize = BigDecimal(size)
    return if (isOsVolume) {
        val planDiskSize = BigDecimal(instance?.plan?.disk ?: 0)
        val totalSize = maxOf(volumeSize.subtract(planDiskSize), BigDecimal.ZERO)
        pricePerGB.multiply(totalSize)
    } else {
        pricePerGB.multiply(volumeSize)
    }
}

```

ภาพที่ 4.73 ฟังก์ชันคำนวณราคาของ Volume

อธิบายภาพที่ 4.73

ฟังก์ชัน toAmount() ใช้ในการคำนวณราคาการใช้งาน Volume ต่อชั่วโมง โดยอ้างอิงจากราคาต่อ 1 GB และขนาดพื้นที่จัดเก็บของ Volume ที่กำหนด ในกรณีที่ Volume นั้นเป็น Volume ของระบบปฏิบัติการ (isOsVolume เป็นจริง) จะใช้ตัวแปร planDiskSize เพื่ออ้างอิงขนาดพื้นที่ที่รวมอยู่ในราคาของ Instance อยู่แล้ว หากผู้ใช้กำหนดขนาด Volume มากกว่าพื้นที่ที่แพ็คเกจมีให้ ระบบจะคิดราคาเฉพาะส่วนที่เกินจาก planDiskSize เท่านั้น โดยนำความจุส่วนที่เกินมาคูณกับราคาต่อ GB เพื่อให้ได้ราคาต่อชั่วโมง แต่หาก Volume ไม่ใช่ Volume ของระบบปฏิบัติการ (isOsVolume เป็นเท็จ) จะนำขนาดพื้นที่ของ Volume ทั้งหมดคูณกับราคาต่อ GB โดยตรงเพื่อคำนวณราคาต่อชั่วโมงของ Volume นั้น

```

fun InstanceResourceUsageEntity.toAmount(): BigDecimal {
    val cpuCost = BigDecimal(this.cpu).multiply(this.resourceCost.cpuPrice)
    val memoryCost = BigDecimal(this.memory).multiply(this.resourceCost.memoryPrice)
    return cpuCost.add(memoryCost)
}

```

ภาพที่ 4.74 ฟังก์ชันคำนวณราคาของ Instance แบบ Custom

อธิบายภาพที่ 4.74

ฟังก์ชัน toAmount() ใช้คำนวณราคาต่อชั่วโมงของ Instance แบบ Custom โดยนำราคา CPU ต่อ Core คูณจำนวนที่เลือก บวกกับราคา Memory ต่อ GB คูณขนาดที่เลือก ส่วนขนาดพื้นที่เก็บข้อมูลจะคำนวณโดยเรียกใช้ฟังก์ชัน toAmount() ของ Volume

4.2.6.2 การเปลี่ยนแพ็คเกจ Instance

```
@Transactional
override fun changeInstancePlan(resource: BillableResource, newPlan: PlanEntity) {
    val now = LocalDateTime.now()
    val ownerProfile = resource.project.owner.userProfile
    val instance = resource as InstanceEntity
    val osVolume = volumeRepository.findByIdAndOsVolumelsTrue(instance.id)
        .orElseThrow {
            ItemNotFoundException(
                "OS volume not found"
            )
        }
    if (ownerProfile != null) {
        if (osVolume.size < newPlan.disk) {
            changeVolumeSize(osVolume, newPlan.disk)
        } else if (osVolume.size > newPlan.disk) {
            throw IllegalArgumentException("Cannot downgrade plan to a smaller disk size")
        }
        cancel(now, instance)
        instance.expiresAt = now.plus(config.cycleHour.toLong(), ChronoUnit.HOURS)
        instance.lastChargedAt = now
        instance.plan = newPlan
        charge(now, instance, newPlan.totalCost * BigDecimal(config.cycleHour))
    }
}
```

ภาพที่ 4.75 ฟังก์ชันสำหรับเปลี่ยนแพ็คเกจ Instance

อธิบายภาพที่ 4.75

ฟังก์ชัน `changeInstancePlan` เริ่มต้นด้วยการค้นหา `osVolume` ก่อน (Volume ที่เก็บระบบปฏิบัติการ) ตรวจสอบว่าแพ็คเกจใหม่ที่เลือกมีขนาด Volume น้อยกว่าปัจจุบันหรือไม่ หากน้อยลงจะไม่อนุญาตให้เปลี่ยน หากมากขึ้นจะไปเรียกฟังก์ชัน `changeVolumeSize()` หลังจากนั้นทำการยกเลิกการเก็บเงินเพื่อคืนเงินเวลาส่วนที่เหลือของแพ็คเกจปัจจุบัน ทำการอัปเดตเวลาหมดอายุเวลาที่เก็บเงินล่าสุด และข้อมูลแพ็คเกจใหม่ลงไป จากนั้นเรียกฟังก์ชัน `charge()` เพื่อคิดค่าบริการใหม่

4.2.6.3 การเปลี่ยนสเปคของ Instance แบบ Custom

```
@Transactional
override fun changeInstanceUsage(
    resource: BillableResource,
    newUsage: InstanceResourceUsageEntity
) {
    val now = LocalDateTime.now()
    val ownerProfile = resource.project.owner.userProfile
    val instance = resource as InstanceEntity
    if (ownerProfile != null) {
        cancel(now, resource)
        instance.expiresAt = now.plus(config.cycleHour.toLong(), ChronoUnit.HOURS)
        instance.lastChargedAt = now
        instance.resourceUsage = newUsage
        charge(now, instance, newUsage.toAmount() * BigDecimal(config.cycleHour))
    }
}
```

ภาพที่ 4.76 ฟังก์ชันเปลี่ยนสเปคของ Instance แบบ Custom

อธิบายภาพที่ 4.76

ฟังก์ชัน `changeInstanceUsage` เริ่มต้นด้วยการยกเลิกการเก็บเงินเพื่อคืนเงินเวลาส่วนที่เหลือของสเปคปัจจุบัน อัปเดตเวลาหมดอายุ เวลาเก็บเงินล่าสุด และสเปคใหม่ลงไป จากนั้นเรียกเก็บเงินใหม่อีกครั้ง

4.2.6.4 การเปลี่ยนขนาด Volume

```

@Transactional
override fun changeVolumeSize(
    resource: BillableResource,
    newSize: Int
) {
    val now = LocalDateTime.now()
    val ownerProfile = resource.project.owner.userProfile
    if (ownerProfile != null) {
        val newCycleCost = resource.amount.multiply(BigDecimal(config.cycleHour))
        if (ownerProfile.credit < newCycleCost)
            throw IllegalStateException("Insufficient credit to change volume size for at least
            ${config.cycleHour} hours")
        cancel(now, resource)
        val volume = resource as VolumeEntity
        volume.expiresAt = now.plus(config.cycleHour.toLong(), ChronoUnit.HOURS)
        volume.lastChargedAt = now
        volume.size = newSize
        charge(now, volume, resource.amount * BigDecimal(config.cycleHour))
    }
}

```

ภาพที่ 4.77 ฟังก์ชันเปลี่ยนขนาดพื้นที่เก็บข้อมูล Volume

อธิบายภาพที่ 4.77

ฟังก์ชัน `changeVolumeSize` เริ่มต้นด้วยการคำนวณราคาใหม่และตรวจสอบเครดิตที่มีว่าเพียงพอสำหรับ 1 รอบการคิดค่าบริการใหม่หากไม่พอจะส่งข้อผิดพลาดกลับ หากพอจะทำการยกเลิกการเก็บเงินเพื่อคืนเงินเวลาส่วนที่เหลือของเดิมก่อนจากนั้นอัปเดตเวลาหมดอายุ เวลาเก็บเงินล่าสุด และขนาดพื้นที่เก็บข้อมูลใหม่ลงไป ทำการเรียกฟังก์ชันคิดค่าบริการ

```

override fun getAllTaxInvoice(keycloakId: String): List<MyTaxList> {
    val uuid = UUID.fromString(keycloakId)
    return taxInvoiceRepository.findAllByUser(uuid)
}

```

ภาพที่ 4.78 ฟังก์ชันส่งข้อมูลผู้เสียภาษี

อธิบายภาพที่ 4.78

ฟังก์ชัน `getAllTaxInvoice` จะค้นหาที่อยู่ผู้เสียภาษีของผู้ใช้ที่ต้องการแล้ว return กลับไป

```

fun getMainTax(): ResponseEntity<ApiResponse<String>?> {
    val authentication = SecurityContextHolder.getContext().authentication
    val token = authentication.principal as Jwt
    val userId = UUID.fromString(token.getClaim<String>("sub"))
    val data = userRepository.findMainTaxByUserId(userId)
    return ResponseEntity.ok().body(
        ApiResponse(
            "success",
            "MainTax",
            data?.id
        )
    )
}

```

ภาพที่ 4.79 ฟังก์ชันส่งข้อมูลที่อยู่ภาษีหลัก

อธิบายภาพที่ 4.79

ฟังก์ชัน `getMainTax` จะค้นหาที่อยู่ผู้เสียภาษีหลักที่ผู้ใช้ตั้งไว้จากฐานข้อมูลแล้วส่ง return กลับไปให้

```

override fun deleteTax(id: String): ResponseEntity<ApiResponse<String>> {
    val authentication = SecurityContextHolder.getContext().authentication
    val jwt = authentication.principal as Jwt
    val currentUserId = UUID.fromString(jwt.subject)
    val user = userRepository.findByAccount_Id(currentUserId).orElseThrow {
        ItemNotFoundException("User not found")
    }
    val taxInvoice = taxInvoiceRepository.findByIdOrNull(UUID.fromString(id))
    ?: return ResponseEntity.ok(ApiResponse("fail", "No Tax Invoice"))
    if(user.taxInvoice == taxInvoice){
        val updatedUser = user.copy(taxInvoice = null)
        userRepository.save(updatedUser)
    }
    val response = taxInvoiceRepository.deleteById(UUID.fromString(id))
    return ResponseEntity
        .ok()
        .body(ApiResponse(
            "success",
            "profile.tax.deleteSuccess",
        ))
}

```

ภาพที่ 4.80 ฟังก์ชันลบที่อยู่ผู้เสียภาษี

อธิบายภาพที่ 4.80

ฟังก์ชันดึงข้อมูลเบื้องต้นของผู้เรียกจาก JWT Token ID ผู้เรียกไปค้นหาว่ามีผู้ใช้อยู่จริง ค้นหาที่อยู่ภาษีว่ามีอยู่จริง ถ้าเป็นที่อยู่หลักจะลบออกจากข้อมูลผู้ใช้อีก่อน สุดท้ายลบข้อมูลออกจากฐานข้อมูล

```

@Transactional
override fun chooseTax(id: String): ResponseEntity<ApiResponse<String>> {
    val authentication = SecurityContextHolder.getContext().authentication
    val jwt = authentication.principal as Jwt
    val currentUserId = UUID.fromString(jwt.subject)
    val account = userRepository.findByAccount_Id(currentUserId).orElseThrow {
        ItemNotFoundException("User not found") }
    if (account == null) {
        return ResponseEntity
            .internalServerError()
            .body(ApiResponse(
                "fail",
                "No User",
            ))
    }
    val taxInvoice = taxInvoiceRepository.findByIdOrNull(UUID.fromString(id))
    ?: return ResponseEntity.internalServerError().body(ApiResponse("fail", "No Tax Invoice"))
    val updatedTax = account.copy(taxInvoice = taxInvoice)
    entityManager.merge(updatedTax)
    entityManager.flush()
    userRepository.save(updatedTax)
    return ResponseEntity
        .ok()
        .body(ApiResponse(
            "success",
            "updated tax successfully",
        ))
}

```

ภาพที่ 4.81 ฟังก์ชันกำหนดที่อยู่เริ่มต้น

อธิบายภาพที่ 4.81

ฟังก์ชันกำหนดที่อยู่เริ่มต้นดึงข้อมูลเบื้องต้นของผู้เรียกจาก JWT Token ID ของผู้ใช้ไปค้นหาว่ามีอยู่จริง ค้นหาที่อยู่ในฐานข้อมูลว่ามีอยู่จริงถ้ามีจริงแก้ไขข้อมูลผู้ใช้ที่เรียกให้เลือกที่อยู่เริ่มต้นเป็น ID ที่ส่งมา ทำการบันทึกข้อมูล

@Transactional

```
override fun updateTaxInvoice(request: TaxInvoiceUpdateRequest, keycloakId: String):
UserProfileResponse {
    val account = accountRepository.findByIdOrNull(UUID.fromString(keycloakId))
    if (account == null){
        return UserProfileResponse(success = false, message = "Invalid user ID in token")
    }
    val taxInvoice = taxInvoiceRepository.findByIdOrNull(UUID.fromString(request.taxInvoiceId))
    if(taxInvoice == null){
        return UserProfileResponse(success = false, message = "It's not exist anymore")
    }
    val updatedTaxInvoice = taxInvoice.copy(
        companyName = request.companyName ?: taxInvoice.companyName,
        branchName = request.branchName ?: taxInvoice.branchName,
        branchNumber = request.branchNumber ?: taxInvoice.branchNumber,
        taxId = request.taxId ?: taxInvoice.taxId,
        address = request.address ?: taxInvoice.address,
        province = request.province ?: taxInvoice.province,
        district = request.district ?: taxInvoice.district,
        subDistrict = request.subDistrict ?: taxInvoice.subDistrict,
        postId = request.postId ?: taxInvoice.postId
    )
    entityManager.merge(updatedTaxInvoice)
    return UserProfileResponse(success = true, message = "Tax Invoice updated successfully")
}
```

ภาพที่ 4.82 ฟังก์ชันแก้ไขที่อยู่ผู้เสียภาษี

อธิบายภาพที่ 4.82

ฟังก์ชัน updateTaxInvoice เริ่มต้นด้วยการค้นหาผู้ว่ามีในฐานข้อมูลหรือไม่ จากนั้นค้นหาที่อยู่ที่ต้องการแก้ไขว่ามีจริงหรือไม่ หากมีจริงบันทึกข้อมูลใหม่ลงในฐานข้อมูล

@Transactional

```

override fun createTaxInvoice(request: TaxInvoiceRequest, keycloakId: String):
UserProfileResponse {
    val account = accountRepository.findByIdOrNull(UUID.fromString(keycloakId))
    if (account == null){
        return UserProfileResponse(success = false, message = "Invalid user ID in token")
    }
    val taxInvoice = TaxInvoiceEntity(
        account = account,
        companyName = request.companyName,
        branchName = request.branchName,
        branchNumber = request.branchNumber!!,
        taxId = request.taxId,
        address = request.address,
        province = request.province,
        district = request.district,
        subDistrict = request.subDistrict,
        postId = request.postId
    )
    val saved = taxInvoiceRepository.saveAndFlush(taxInvoice)
    return UserProfileResponse(success = true, message = "profile.tax.createSuccess")
}

```

ภาพที่ 4.83 ฟังก์ชันสร้างที่อยู่ผู้ใช้เสียภาษี

อธิบายภาพที่ 4.83

ฟังก์ชัน createTaxInvoice เริ่มต้นด้วยการค้นหาว่าผู้ใช้ที่เรียกมีจริงไหม ถ้ามีจริงจะสร้างข้อมูลที่อยู่ใหม่ที่ต้องการให้ จากนั้นบันทึกข้อมูลลงในฐานข้อมูล

บทที่ 5

ผลการดำเนินโครงการ

5.1 ผลการดำเนินงาน

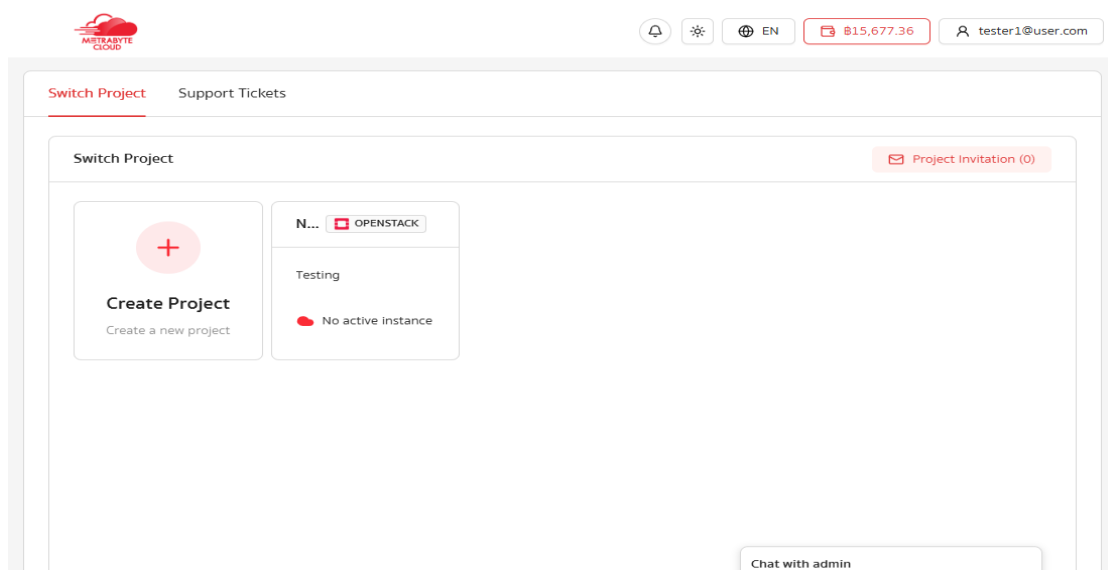
ผู้พัฒนาได้พัฒนา Web Application แพลตฟอร์มบริหารจัดการโครงการสำหรับคลาวด์แบบบริการตนเองเสร็จสมบูรณ์ โดยรองรับการทำงานทั้งฝั่งผู้ทั่วไปและผู้ดูแลระบบ ในฝั่งของผู้ใช้ทั่วไป สามารถบริหารจัดการโครงการและสมาชิกได้ด้วยตนเอง พร้อมทั้งมีระบบเติมเงินเป็นเครดิตเพื่อใช้สำหรับค่าใช้จ่ายคลาวด์ และสามารถติดตามสรุปผลค่าใช้จ่ายได้ทั้งแบบรายวัน รายเดือน และรายปี ในขณะที่ฝั่งผู้ดูแลระบบ สามารถจัดการโครงการของผู้ใช้ทั้งหมด สามารถติดตามการใช้จ่ายของแต่ละโครงการในรูปแบบกราฟ โดยจำแนกตามผู้ให้บริการ Openstack และ Proxmox นอกจากนี้ ยังสามารถตรวจสอบประวัติการเติมเงินและการใช้เครดิต พร้อมทั้งจัดอันดับผู้ใช้ที่มีการเติมเงินและการใช้งานสูงสุดตามช่วงเวลาที่ได้เลือกได้

5.2 การแสดงผลของ Web Application

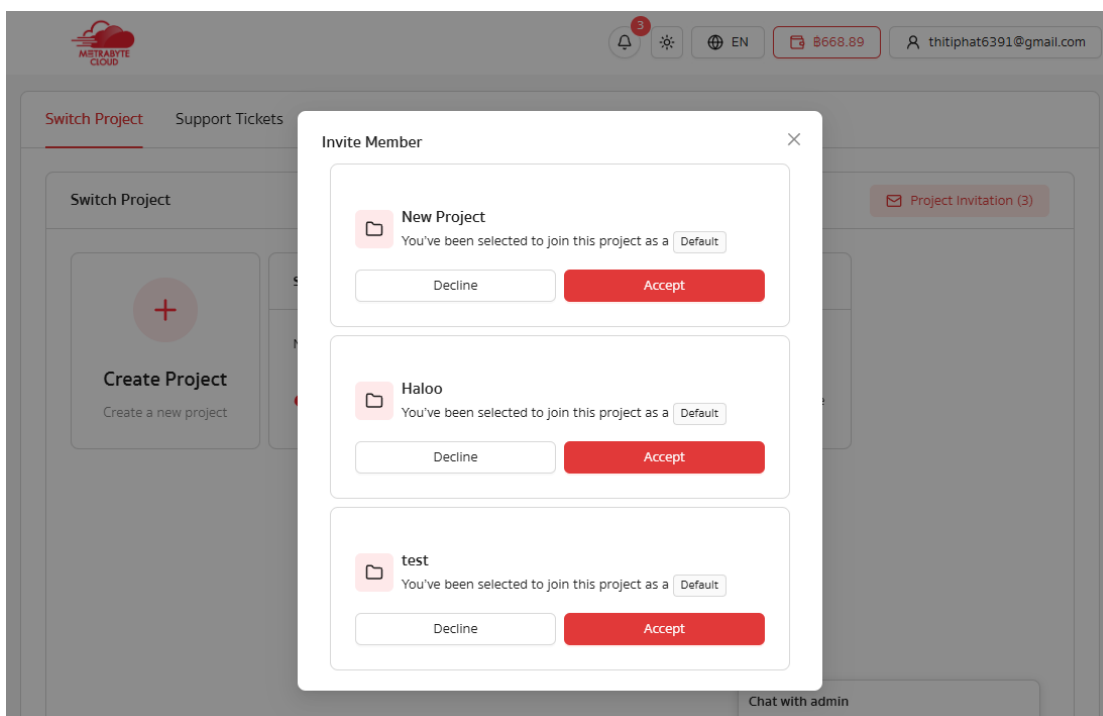
5.2.1 ในส่วนของผู้ใช้งานทั่วไป

5.2.1.1 หน้าเลือกโครงการ

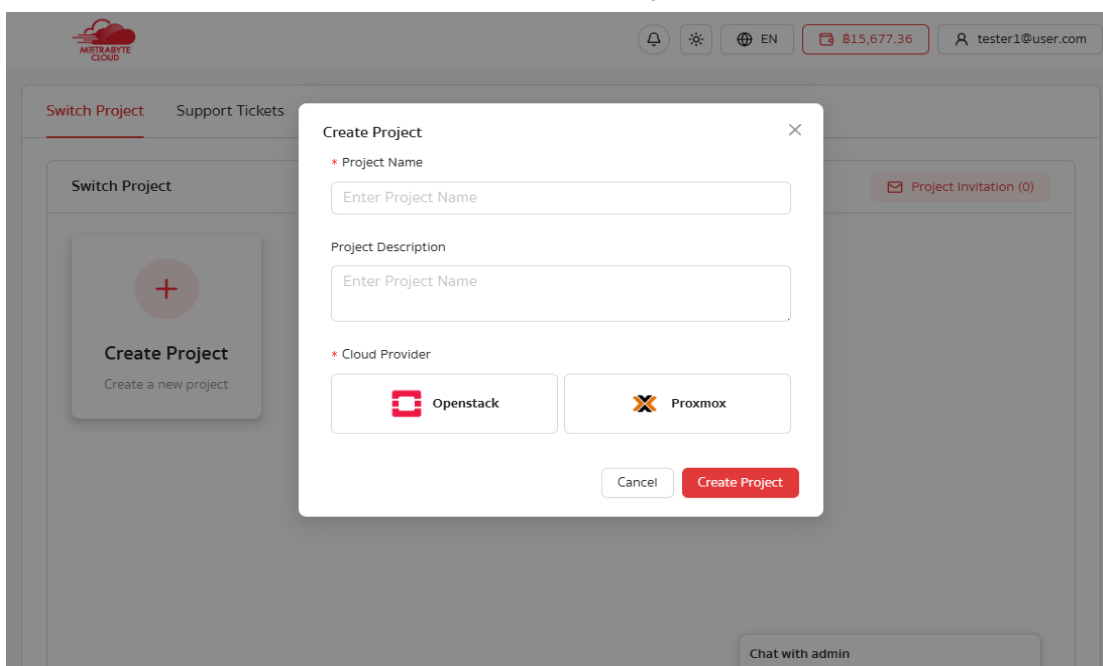
เมื่อเข้าสู่ระบบมาแล้วจะแสดงหน้าเลือกโครงการเป็นหน้าแรกดังภาพที่ 5.1 ให้ผู้ใช้เลือกโครงการ หรือหากต้องการตรวจสอบค่าเชิญเข้าโครงการกดปุ่ม Project Invitation เพื่อเปิด Modal ค่าเชิญสามารถตอบรับ หรือปฏิเสธค่าเชิญได้ดังภาพ 5.2 หากต้องการสร้างโครงการให้กด Create Project จะเปิด Modal ให้กรอกรายละเอียดโครงการดังภาพที่ 5.3



ภาพที่ 5.1 หน้าเลือกโครงการ



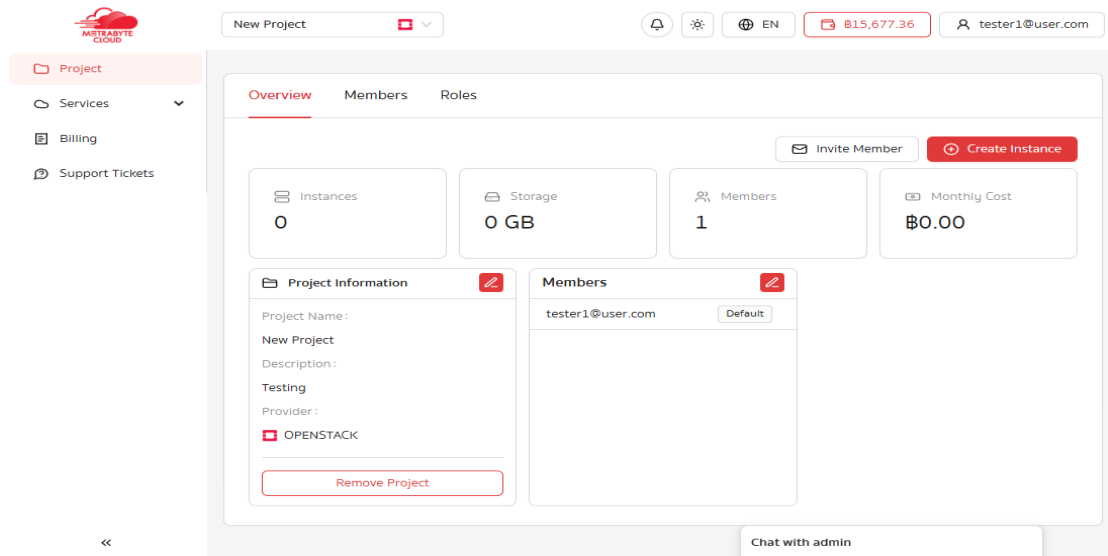
ภาพที่ 5.2 Modal ตอบรับคำเชิญเข้าร่วมโครงการ



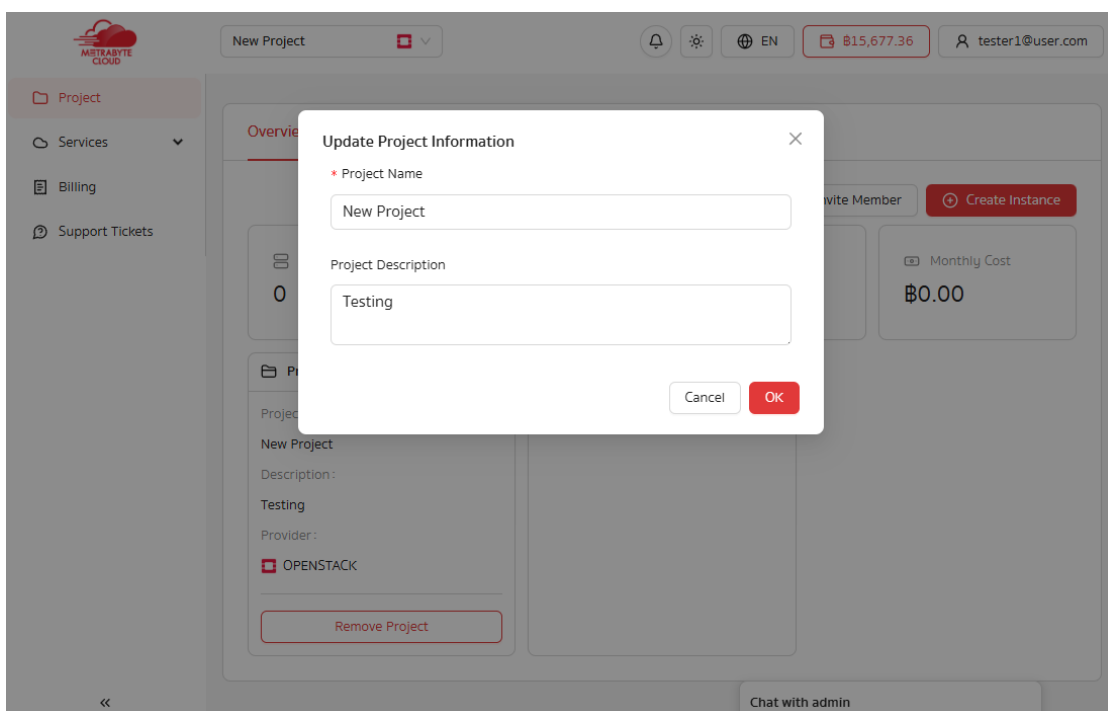
ภาพที่ 5.3 Modal สร้างโครงการ

5.2.1.2 หน้าจัดการโครงการ

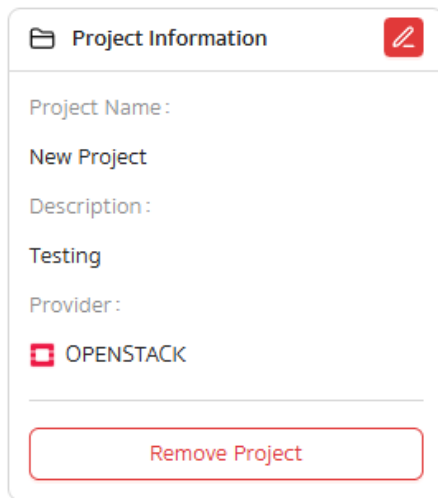
เมื่อเลือกโครงการจากหน้าแรกจะแสดงหน้าแรกของโครงการดังภาพที่ 5.4 แสดงข้อมูลเบื้องต้นต่างๆ ภายในโครงการให้ผู้ใช้สามารถแก้ไขข้อมูลโครงการได้เมื่อกดปุ่มดินสอ บริเวณ Project Information จะเปิด Modal ดังภาพที่ 5.5 ให้แก้ไขข้อมูลได้ และในส่วนเดียวกันจะมีปุ่มสำหรับบอกร้างโครงการ หรือลบโครงการหากเป็นเจ้าของโครงการดังภาพที่ 5.6-5.7



ภาพที่ 5.4 หน้าแรกของโครงการ



ภาพที่ 5.5 Modal แก้ไขข้อมูลโครงการ



Project Information

Project Name :

New Project

Description :

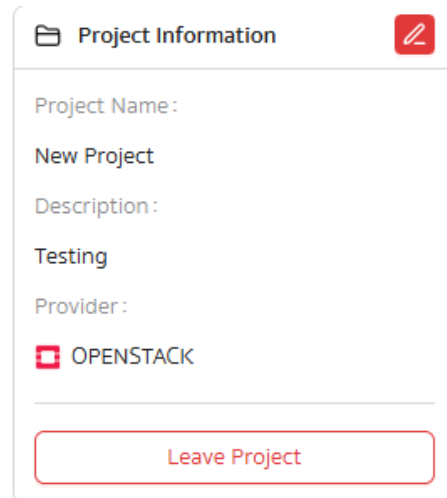
Testing

Provider :

 OPENSTACK

Remove Project

ภาพที่ 5.6 ปุ่มลบโครงการ



Project Information

Project Name :

New Project

Description :

Testing

Provider :

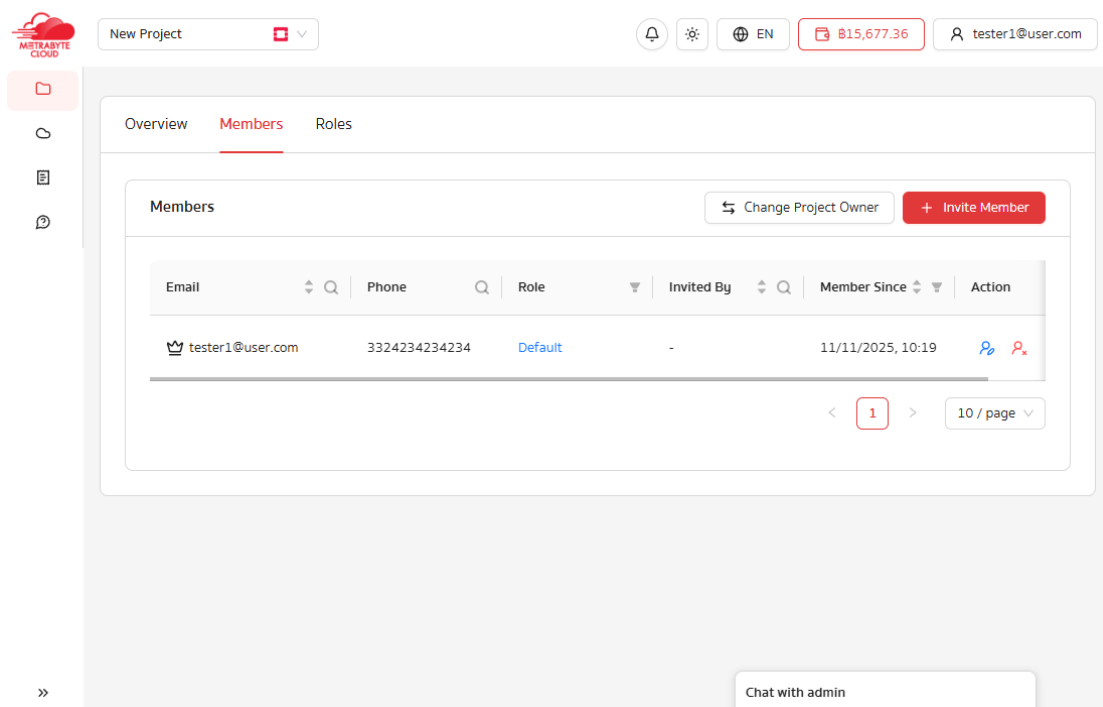
 OPENSTACK

Leave Project

ภาพที่ 5.7 ปุ่มออกจากโครงการ

5.2.1.3 หน้าจัดการสมาชิกในโครงการ

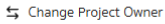
แสดงตารางสมาชิกทั้งหมดในโครงการดังภาพที่ 5.8 โดยในหน้านี้ผู้ใช้สามารถเปลี่ยน Role ของสมาชิกได้เมื่อกดปุ่มแก้ไขที่สมาชิกที่ต้องการจะแสดง Modal ดังภาพที่ 5.9 สามารถลบสมาชิกออกจากโครงการดังภาพที่ 5.10 และสามารถส่งคำเชิญเข้าร่วมโครงการได้ดังภาพที่ 5.11 หากเป็นเจ้าของโครงการจะสามารถเปลี่ยนเจ้าของโครงการได้ดังภาพที่ 5.12 สมาชิกที่รอการตอบรับคำเชิญจะแสดงเมนูพิเศษสำหรับส่งการแจ้งเตือนซ้ำ และยกเลิกคำเชิญได้ดังภาพที่ 5.13



Metabyte Cloud

New Project 

   EN  \$15,677.36  tester1@user.com

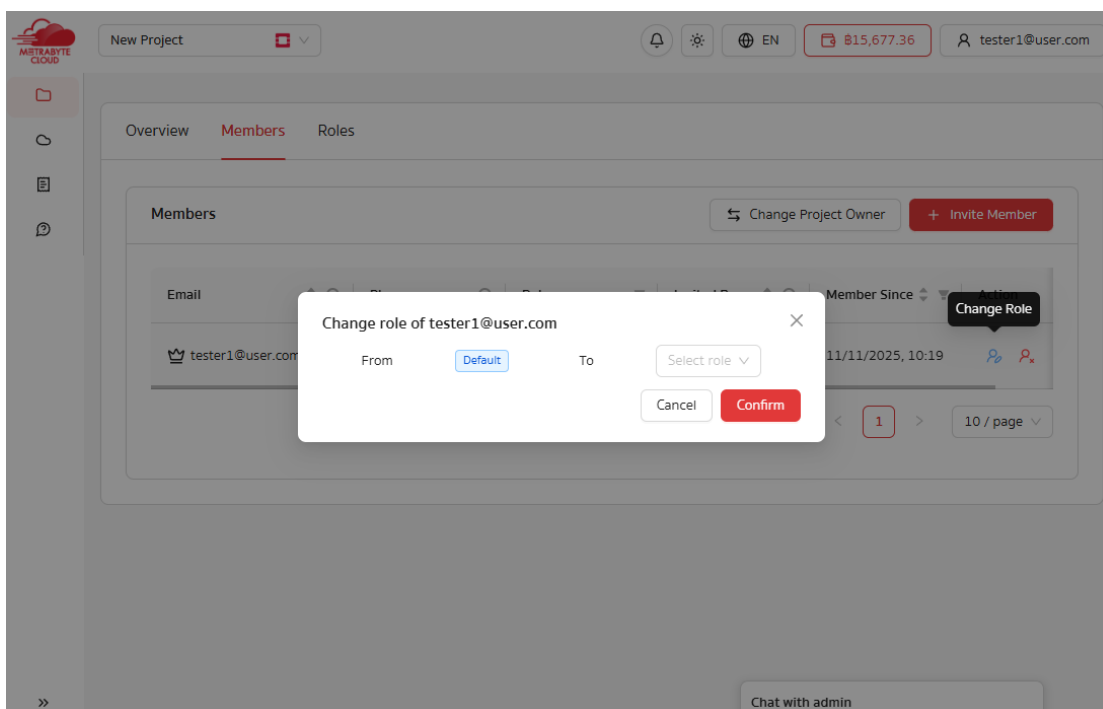
Members  

Email	Phone	Role	Invited By	Member Since	Action
 tester1@user.com	3324234234234	Default	-	11/11/2025, 10:19	 

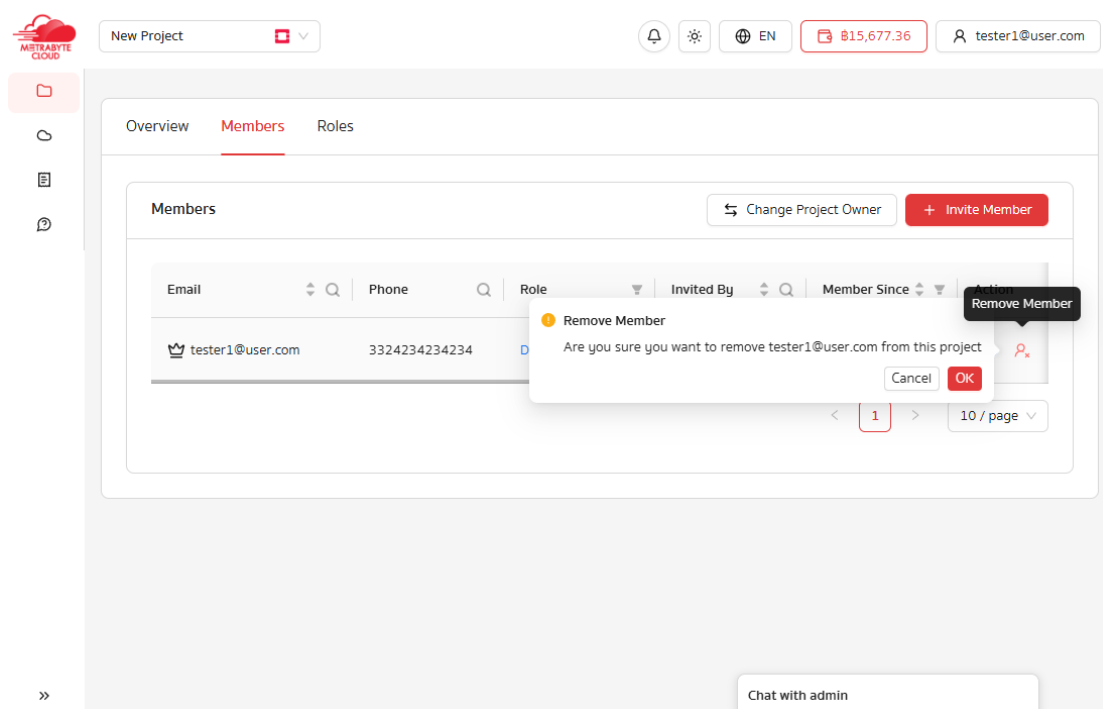
< 1 > 10 / page

Chat with admin

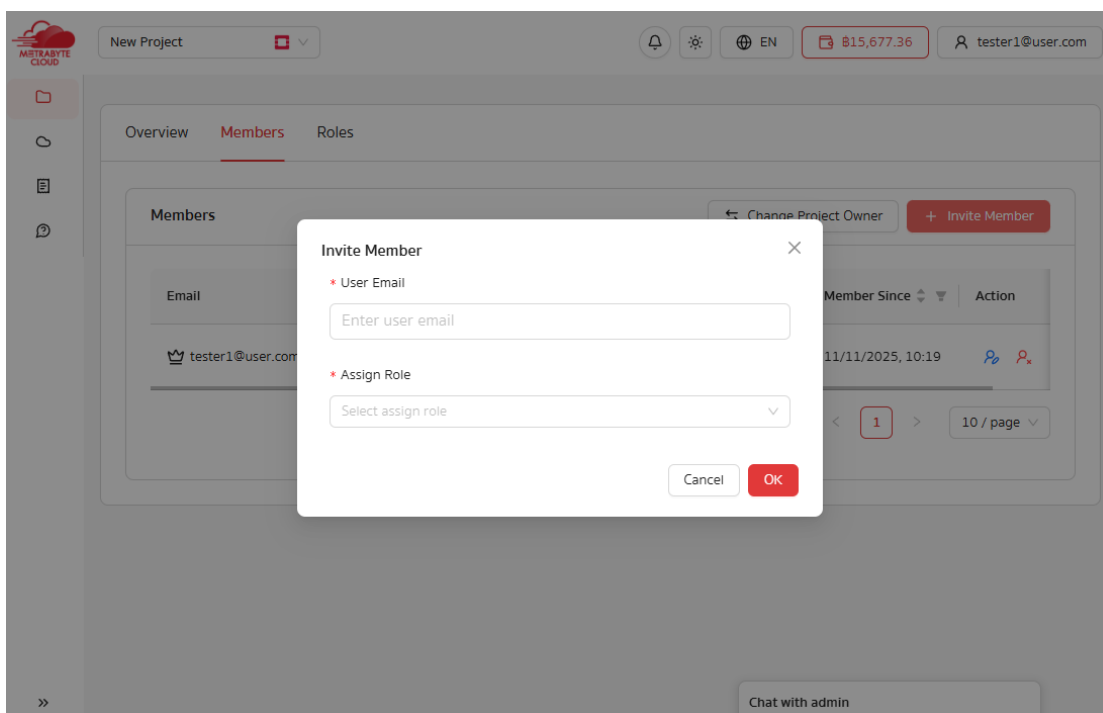
ภาพที่ 5.8 หน้าจัดการสมาชิกในโครงการ



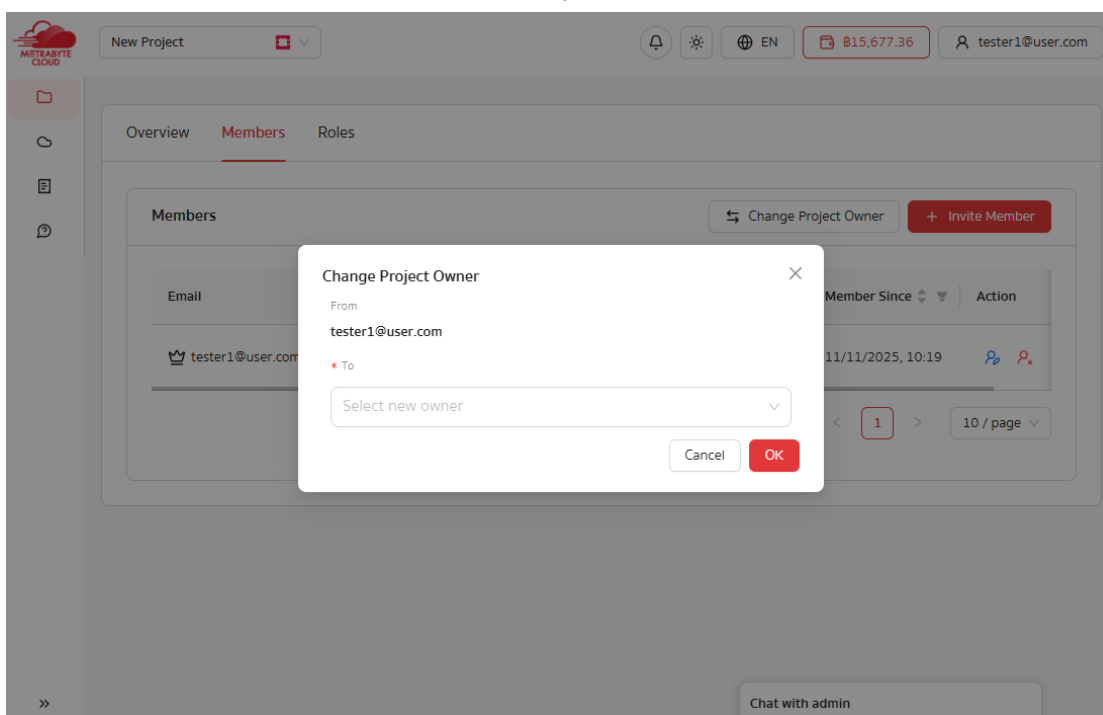
ภาพที่ 5.9 Modal เปลี่ยน Role สมาชิก



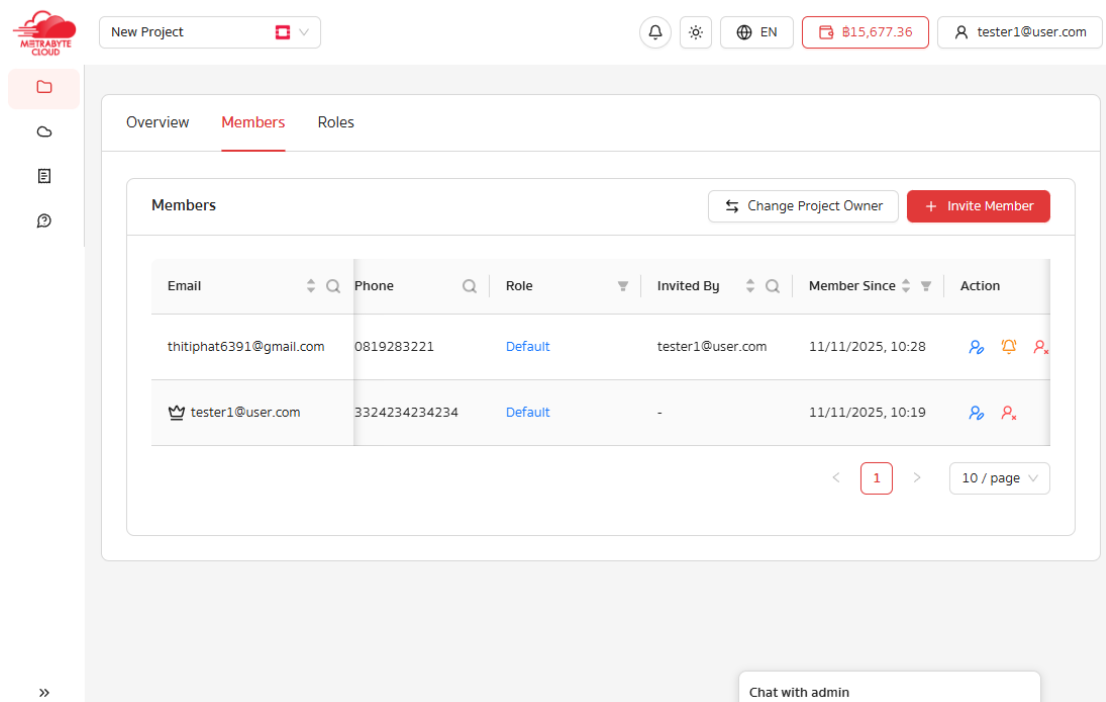
ภาพที่ 5.10 ยืนยันการลบสมาชิกออกจากโครงการ



ภาพที่ 5.11 Modal เชิญสมาชิกเข้าร่วมโครงการ



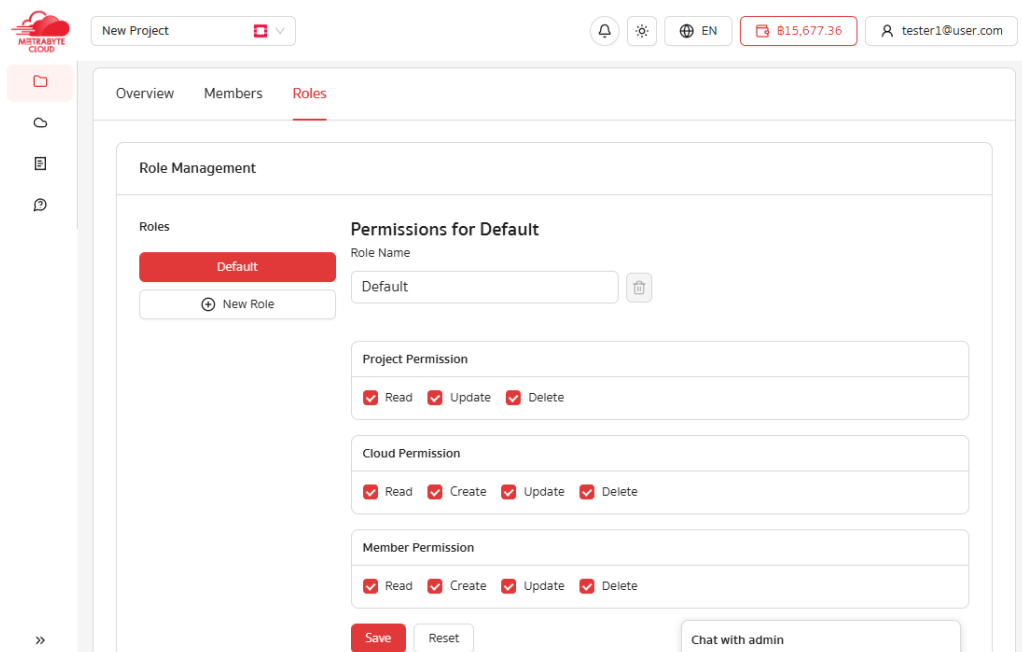
ภาพที่ 5.12 Modal เปลี่ยนเจ้าของโครงการ



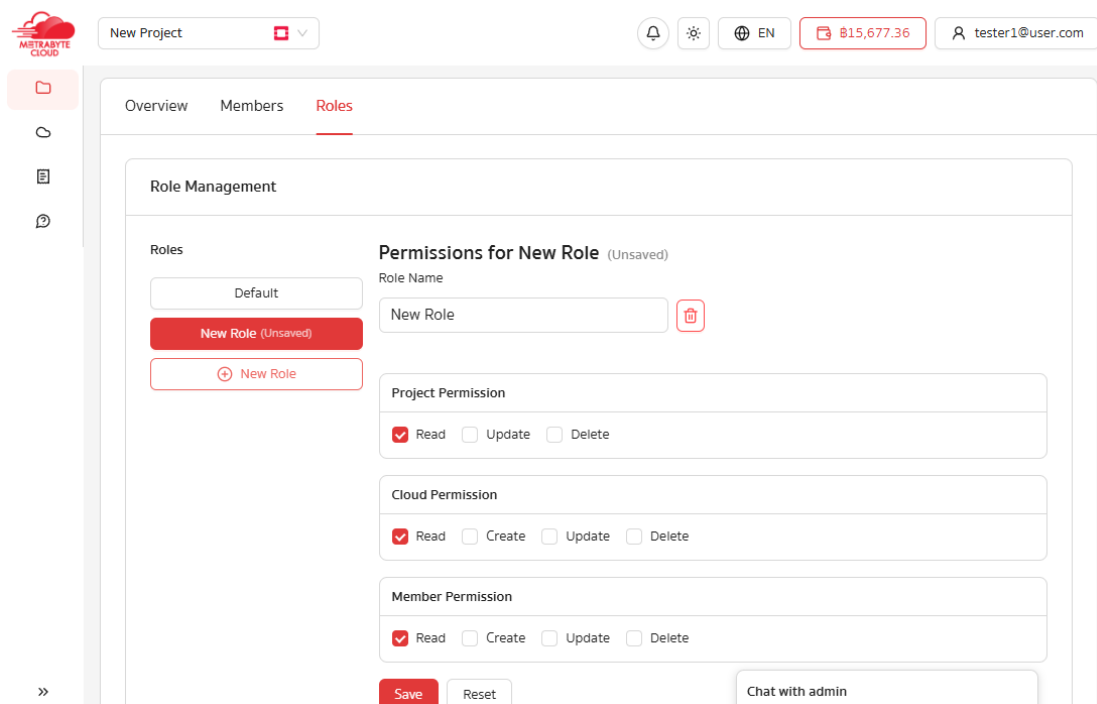
ภาพที่ 5.13 สมาชิกที่รอการตอบรับคำเชิญ

5.2.1.4 หน้าจัดการ Role ในโครงการ

จะแสดง Role ทั้งหมดในโครงการดังภาพที่ 5.14 ผู้ใช้สามารถกำหนดสิทธิ์และชื่อของ Role ที่กำลังเลือกอยู่ได้ดังภาพที่ 5.14 เมื่อกด New Role จะสร้าง Role ใหม่ขึ้นมาพร้อมให้กรอกชื่อ และตั้งสิทธิ์ที่อนุญาตให้เข้าถึงได้ดังภาพที่ 5.15 หากต้องการลบสามารถกดปุ่มถึงขยะด้านข้างช่องกรอกชื่อของ Role ได้ดังภาพที่ 5.16

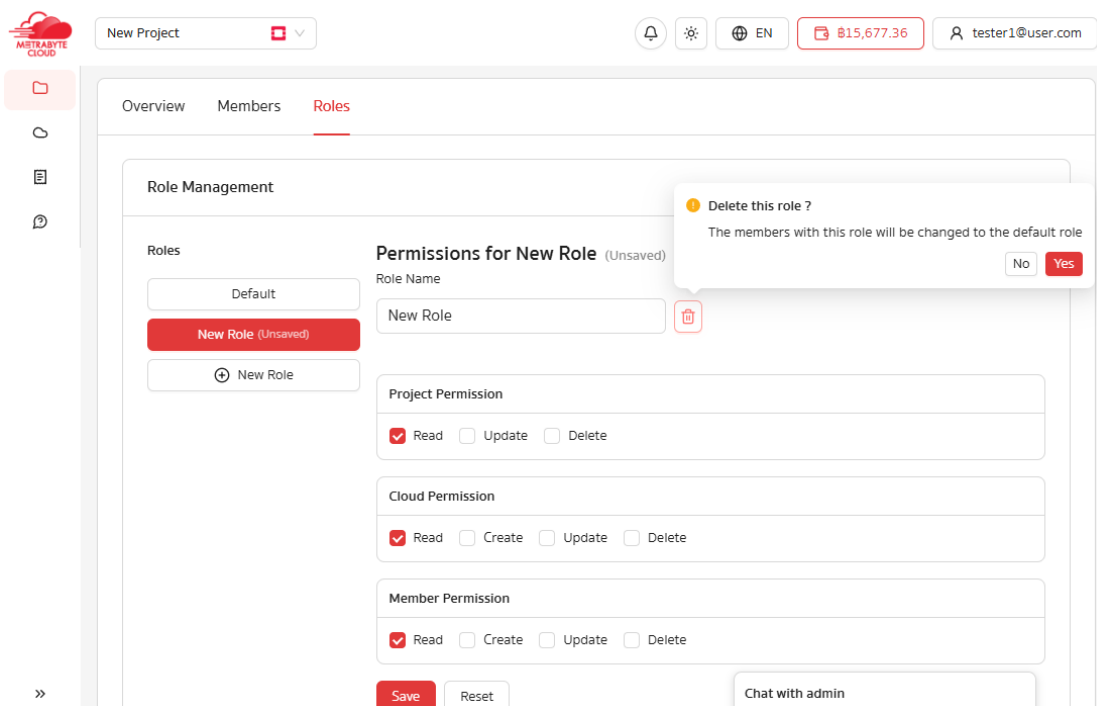


ภาพที่ 5.14 หน้าจัดการ Role ในโครงการ



The screenshot shows the 'Role Management' section of the Metabyte Cloud interface. The 'Roles' list on the left includes 'Default', 'New Role (Unsaved)', and a '+ New Role' button. The 'Permissions for New Role (Unsaved)' form is active, showing 'Role Name' as 'New Role'. Below this, there are three permission sections: 'Project Permission' (Read checked, Update and Delete unchecked), 'Cloud Permission' (Read checked, Create, Update, and Delete unchecked), and 'Member Permission' (Read checked, Create, Update, and Delete unchecked). At the bottom, there are 'Save' and 'Reset' buttons, and a 'Chat with admin' button.

ภาพที่ 5.15 หน้าการสร้าง Role ใหม่



This screenshot shows the same 'Role Management' interface as the previous one, but with a confirmation dialog box overlaid. The dialog box has a yellow warning icon and the text 'Delete this role ?' followed by 'The members with this role will be changed to the default role'. At the bottom of the dialog are 'No' and 'Yes' buttons. The background form is partially obscured by the dialog.

ภาพที่ 5.16 ยืนยันการลบ Role

5.2.1.5 หน้าดูค่าใช้จ่ายในโครงการ

ประกอบด้วยส่วนของค่าใช้จ่ายรวม แสดงคลาวด์ปัจจุบันที่มีการเก็บค่าบริการอยู่ รวมถึงมีตารางแสดงค่าใช้จ่ายของโครงการทั้งหมดตั้งแต่สร้างโครงการมาดังภาพที่ 5.17 โดยในภาพที่ 5.18 ผู้ใช้สามารถปรับแต่งดูเป็นแบบรายวัน รายเดือน หรือรายปี และสามารถเลือกช่วงเวลาที่ต้องการได้ด้วย

The screenshot shows the Mitrabyte Cloud interface. On the left is a sidebar with 'Project', 'Services', 'Billing' (highlighted), and 'Support Tickets'. The main content area is titled 'Billing' and shows 'Monthly Cost' as ฿108.00 /Month and 'Hourly Cost' as ฿0.15 /Hour. Below this is the 'Instance Usage' section, showing a 'Monthly Total' of ฿36.00 /Month and 'Hourly Total' of ฿0.05 /Hour. A table lists instances with columns: Name, Last Charge, Next Charge, Baht/Month, and Baht/Hour. One instance 'oho' is shown with a last charge 11 hours ago and a next charge in 13 hours. At the bottom, there is a 'Volume Usage' section and a 'Group By' dropdown set to 'Month'.

ภาพที่ 5.17 หน้าดูค่าใช้จ่ายในโครงการปัจจุบัน

This screenshot shows the same Mitrabyte Cloud interface but with the 'Group By' dropdown set to 'Month'. Below the 'Instance Usage' table, there is a 'Select Date' section with 'Start Date' and 'End Date' pickers. A table shows historical usage with columns 'date' and 'Total (Baht)'. The data rows are for November 2025 (Total: 39.60) and October 2025 (Total: 7.48). The sidebar and top navigation remain the same.

ภาพที่ 5.18 ตารางค่าใช้จ่ายทั้งหมดตั้งแต่สร้างโครงการ

5.2.1.6 หน้าการเงินภายในบัญชีผู้ใช้

ภาพที่ 5.19-5.20 แสดงตารางประวัติการเติมเครดิต ตารางธุรกรรมเครดิต รวมถึงเมนูสำหรับเติมเครดิตเข้าบัญชี

Balance

≈ ฿15,677.36
Actual: ฿15,677.3649

Payment History

ID	Name	Method	Status	Created	Amount (Baht)	Action
6	Credit Top-Up For Cloud Service Fee	Credit Card	Success	29/10/2025, 18:29	+ 50.00	
3	Credit Top-Up For Cloud Service Fee	Credit Card	Success	29/10/2025, 17:30	+ 1,000.00	

Chat with admin

ภาพที่ 5.19 หน้าการเงินของบัญชีผู้ใช้

Credit Transaction

ID	Name	Create Date	Amount (Baht)
547	Charging INSTANCE 'oho'	11/11/2025, 00:00	- 1.20
514	Charging INSTANCE 'oho'	10/11/2025, 00:00	- 1.20
479	Charging INSTANCE 'oho'	09/11/2025, 00:00	- 1.20
446	Charging INSTANCE 'oho'	08/11/2025, 00:00	- 1.20
413	Charging INSTANCE 'oho'		- 1.20

Chat with admin

ภาพที่ 5.20 ตารางธุรกรรมเครดิต

5.2.1.7 Modal เติมเครดิต

เมื่อเปิด Modal เติมเครดิตจะมีหลัก ๆ 3 ขั้นตอนคือ ภาพที่ 5.21 เลือกจำนวนเงิน ภาพที่ 5.22 เลือกช่องทางการชำระเงิน (ถ้าตีถูกต้องการใบกำกับภาษีจะมีให้เลือกที่อยู่ผู้เสียภาษีดังภาพที่ 5.23) และดำเนินการชำระเงิน

ภาพที่ 5.21 Modal เติมเครดิต

ภาพที่ 5.22 Modal เลือกช่องทางการจ่ายเงิน

Choose payment method

* Enter amount

฿ 50

Receipt [Quotation](#)

Service Amount	50.00
VAT 7%	3.50
GRAND TOTAL	53.50

☒ **Require Tax Receipt**

+ Add New Tax Address

☒ **Headquarters** asdasd
*3123
12312 ลาวร้องไห้ ไร่เศษชัยชาญ อ่างทอง 14110

☐ **Individual** test asd
*3123

ภาพที่ 5.23 เลือกที่อยู่ผู้เสียภาษี

5.2.1.8 Modal การชำระเงินด้วยพร้อมเพย์

ช่องทางพร้อมเพย์สามารถชำระได้ทันที ไม่ต้องกรอกข้อมูลอะไรเพิ่มเติมดังภาพที่ 5.24 เมื่อกดแล้วจะเปลี่ยน Modal เป็นหน้าต่างสำหรับสแกนจ่ายเงิน สามารถกดโหลดรูปภาพ Qr Code หรือยกเลิกการจ่ายได้ดังภาพที่ 5.25

Choose payment method

฿ 50

Receipt [Quotation](#)

Service Amount	50.00
VAT 7%	3.50
GRAND TOTAL	53.50

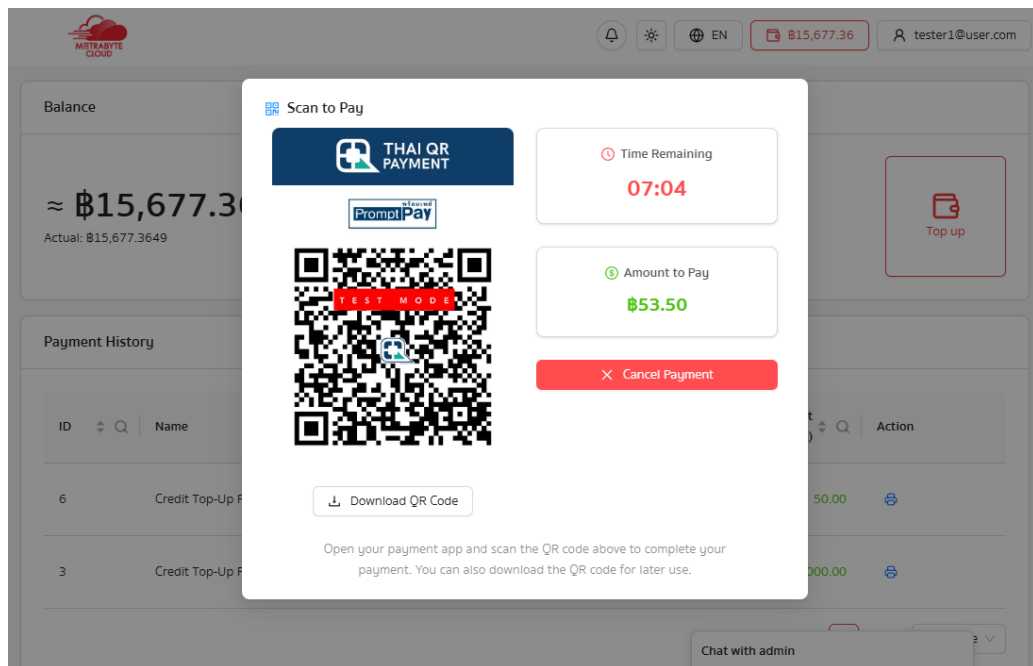
☐ **Require Tax Receipt**

Promptpay
Pay instantly with your mobile banking app

Credit Card
Pay securely with Visa, MasterCard, or other cards

Pay with Promptpay

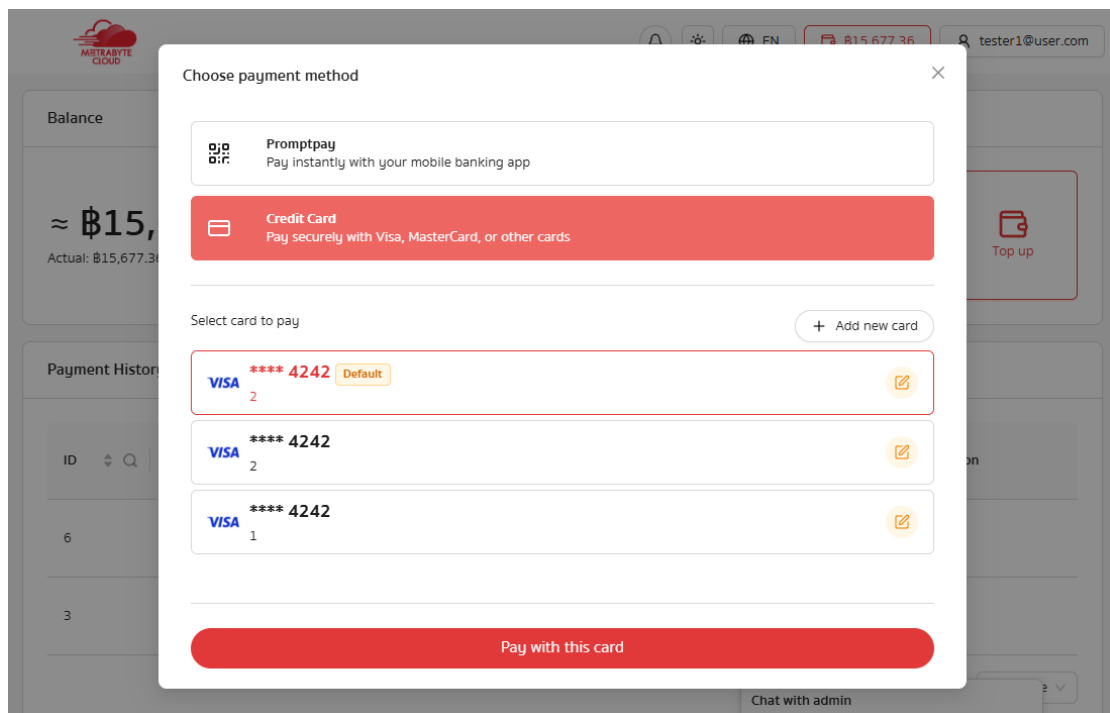
ภาพที่ 5.24 เลือกจ่ายด้วยพร้อมเพย์



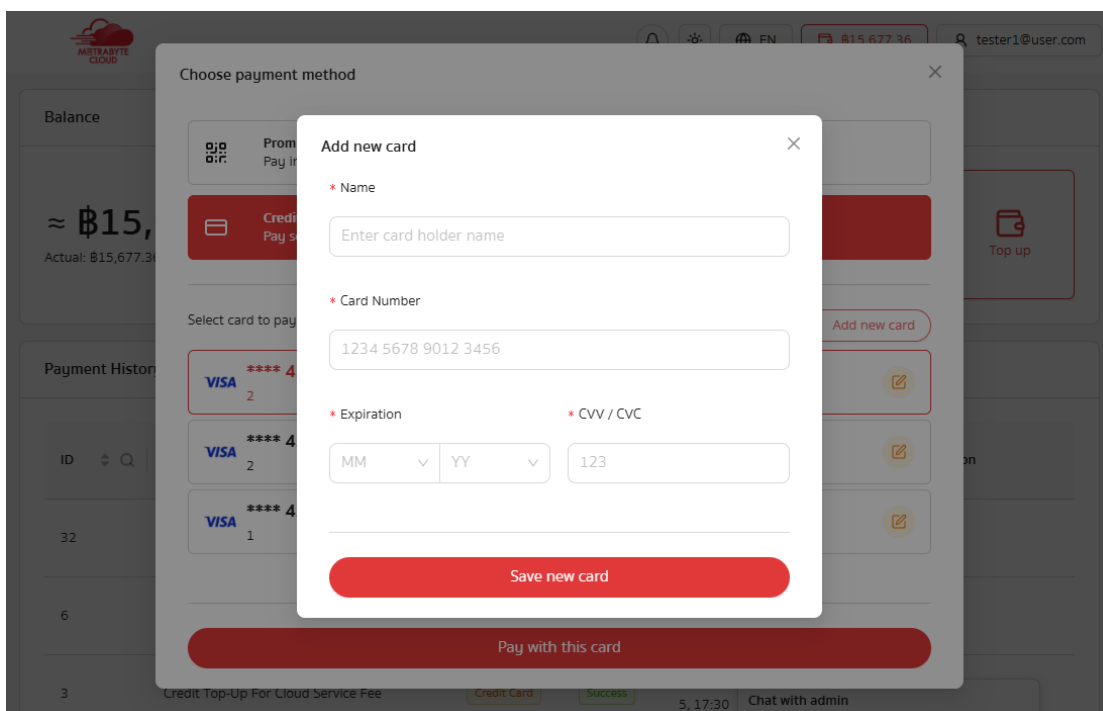
ภาพที่ 5.25 Modal สแกนจ่ายพร้อมเพย์

5.2.1.9 Modal การชำระเงินด้วยบัตรเครดิต

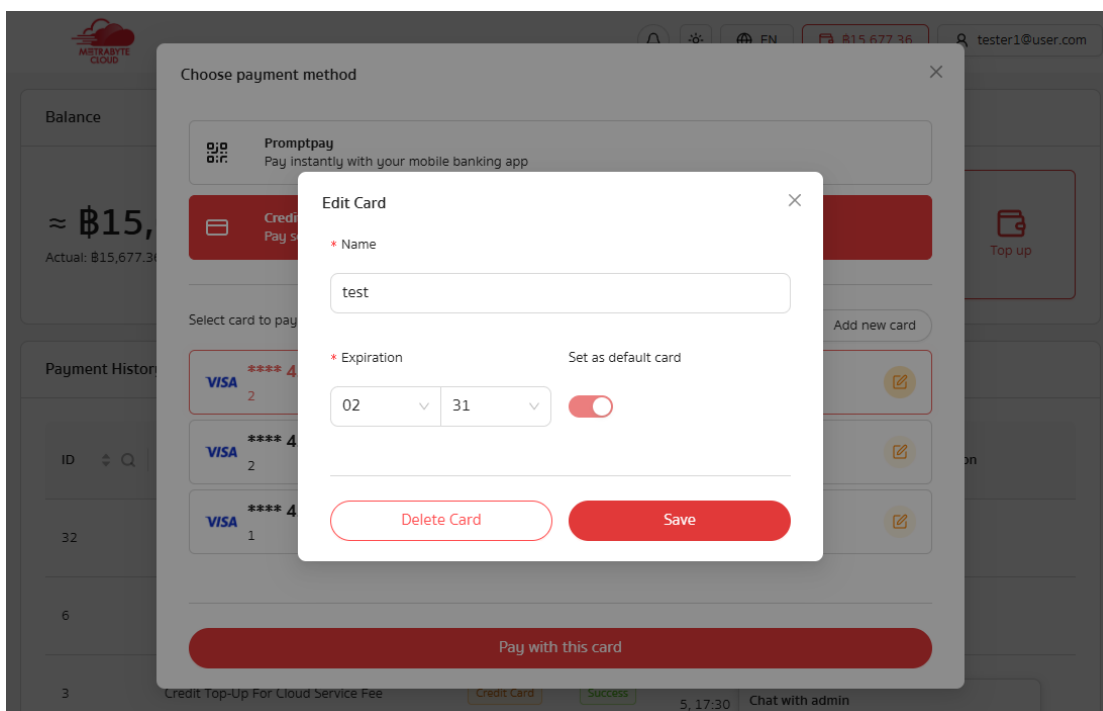
เมื่อเลือกช่องทางบัตรเครดิตจะต้องเลือกบัตรที่ต้องการใช้จ่าย โดยสามารถกดเพิ่มบัตร แก้ไขบัตร หรือลบบัตรได้ดังภาพที่ 5.26-5.28



ภาพที่ 5.26 เลือกจ่ายด้วยบัตรเครดิต



ภาพที่ 5.27 Modal เพิ่มบัตรเครดิตใหม่



ภาพที่ 5.28 Modal แก้ไขบัตรเครดิตเก่า

5.2.1.10 การจัดการที่อยู่ผู้เสียภาษี

ผู้ใช้สามารถจัดการที่อยู่ผู้เสียภาษีเพื่อใช้ในการพิมพ์ใบเสร็จ ใบเสนอราคาได้ โดยในภาพที่ 5.29 จะมีให้เลือกประเภทของที่อยู่ว่าเป็นของบุคคลธรรมดา หรือบริษัท เมื่อกรอกข้อมูลเสร็จกด Save ดังภาพที่ 5.30 เพื่อบันทึกข้อมูลที่อยู่ผู้เสียภาษี

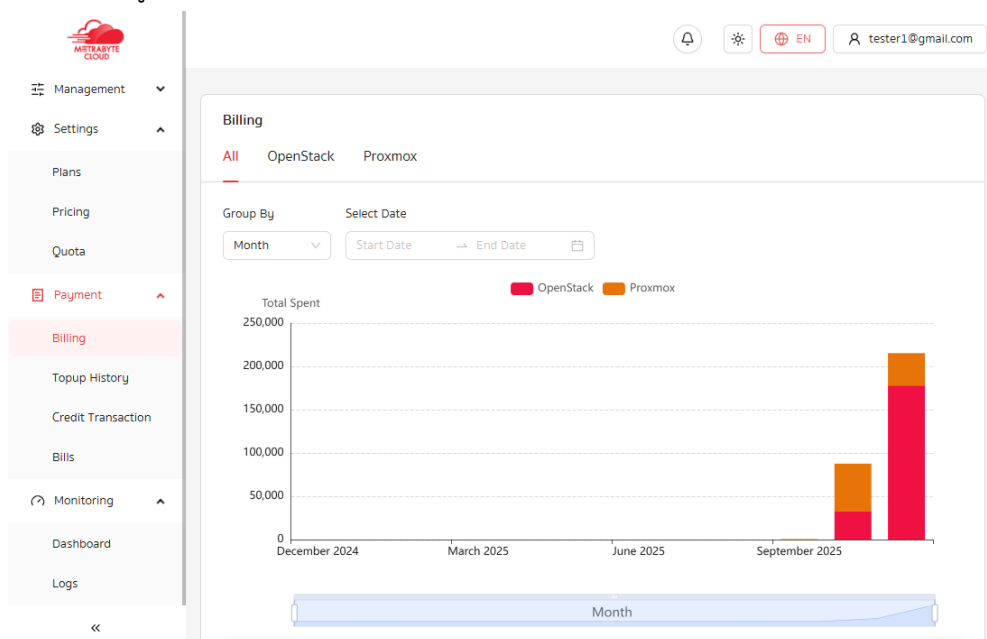
ภาพที่ 5.29 Modal สร้างที่อยู่ผู้เสียภาษี

ภาพที่ 5.30 Modal บันทึกที่อยู่ผู้เสียภาษี

5.2.2 ในส่วนของผู้ดูแลระบบ

5.2.2.1 หน้าแสดงการใช้งานเครดิตของโครงการทั้งหมดในระบบ

ผู้ดูแลระบบสามารถดูกราฟการใช้จ่ายทั้งหมดในระบบได้แบ่งตามผู้ให้บริการ Openstack และ Proxmox สามารถเลือกจัดกลุ่มรายวัน รายเดือน หรือรายปีรวมถึงเลือกช่วงเวลาของข้อมูลได้ดังภาพที่ 5.31 ส่วนด้านล่างในภาพที่ 5.32 จะแสดงข้อมูลในรูปแบบตาราง โดยแยกตามโครงการ เมื่อกดดูจะพาไปหน้าค่าใช้จ่ายของโครงการนั้นดังภาพที่ 5.33

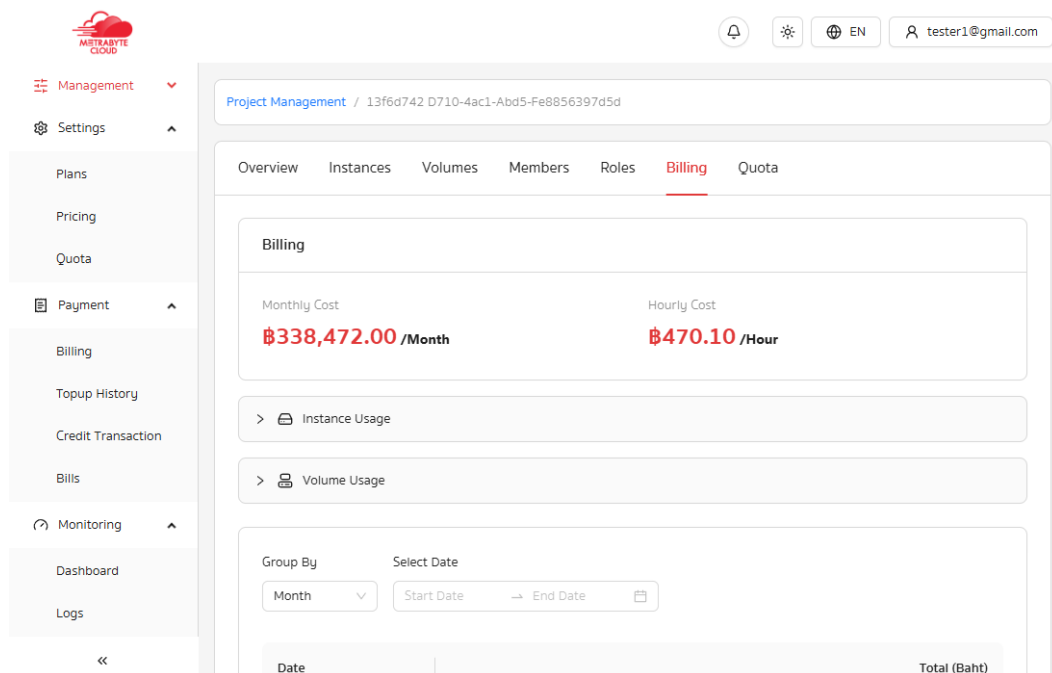


ภาพที่ 5.31 หน้ารวมการใช้จ่ายทั้งระบบ

The screenshot shows the 'Billing' section of the Metabyte Cloud interface, displaying a table of project billing details. The left sidebar is the same as in the previous image. The main content area shows a table with columns: Project, Date, Total (Baht), and Action. The table lists three projects with their respective IDs, dates, and total costs in Baht. The 'Action' column contains green circular icons with a right-pointing arrow.

Project	Date	Total (Baht)	Action
AI อังกฤษ 47be4a69-a9ea-4f42-92c8-73b274173865	November 2025	0.02	👉
f 13f6d742-d710-4ac1-abd5-fe8856397d5d	November 2025	26.40	👉
f 13f6d742-d710-4ac1-abd5-fe8856397d5d	October 2025	2.40	👉
FOR MC DO NOT DELETE 22adab61-a464-41bb-aca9-a87193259089	November 2025	528.00	👉

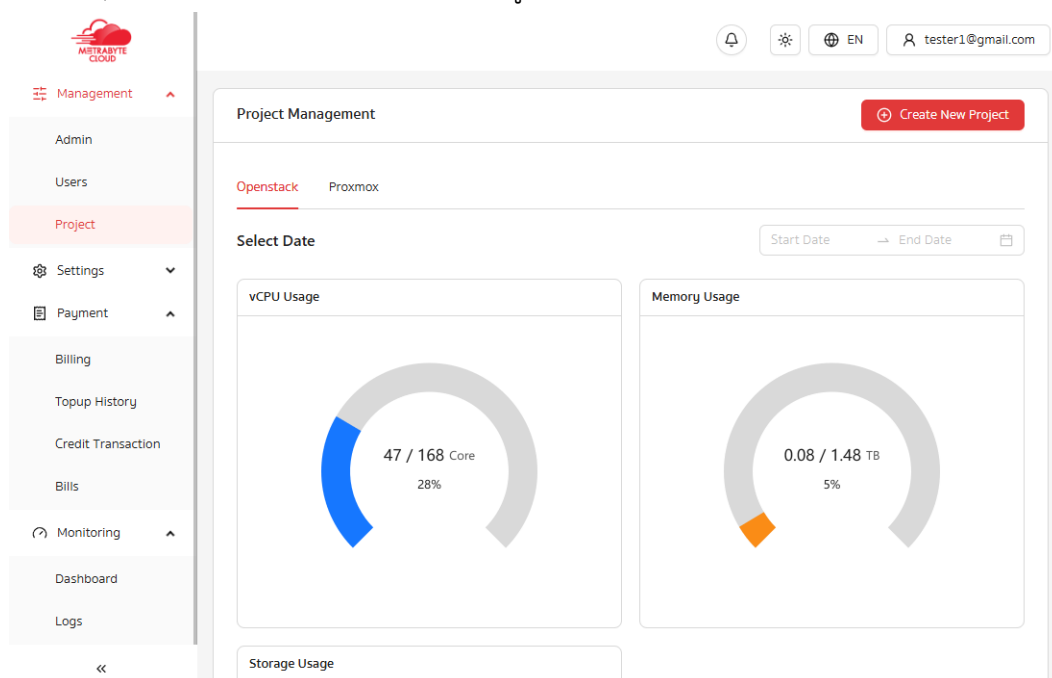
ภาพที่ 5.32 ตารางค่าใช้จ่ายจัดกลุ่มตามโครงการ



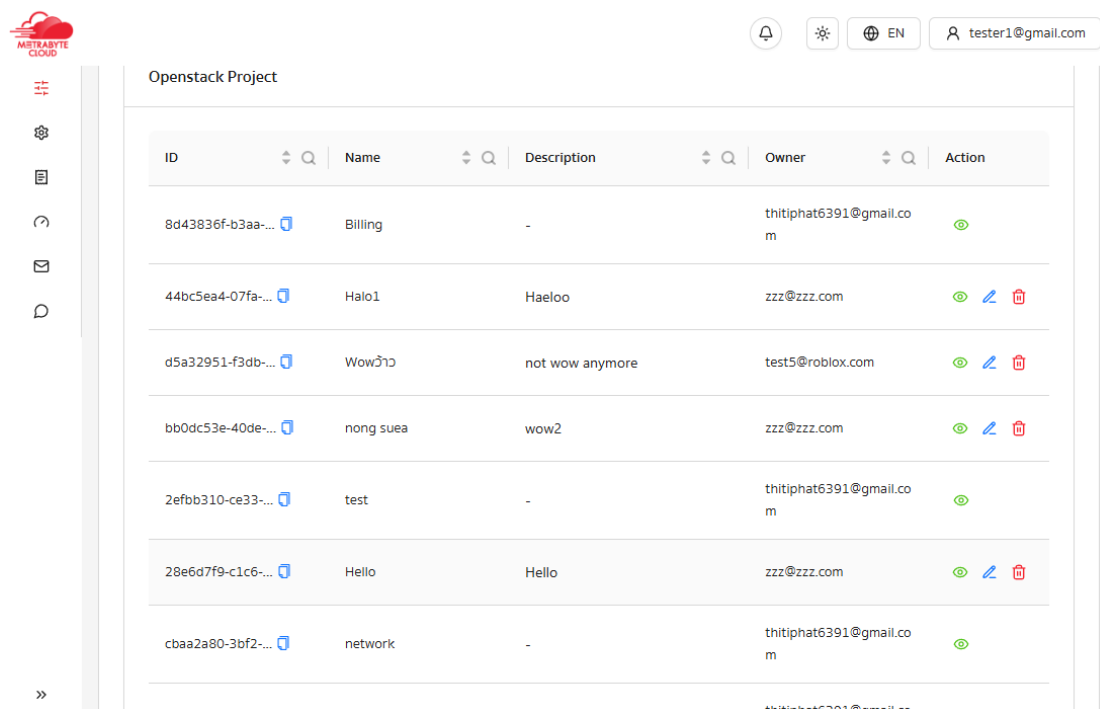
ภาพที่ 5.33 หน้าดูค่าใช้จ่ายในโครงการ

5.2.2.2 หน้าจัดการโครงการในระบบ

ภายในหน้านี้จะมีสถิติการใช้งานทรัพยากรของคลาวด์ทั้งหมดในระบบ โดยแยกตามผู้ให้บริการ Openstack และ Proxmox สามารถเลือกช่วงเวลาของข้อมูลได้ดังภาพที่ 5.34 ส่วนด้านล่างจะเป็นตารางโครงการทั้งหมดในระบบดังภาพที่ 5.35 ผู้ดูแลระบบสามารถเข้าไปจัดการแก้ไขต่าง ๆ ในโครงการ รวมถึงสร้างโครงการให้ผู้ใช้ได้ดังภาพที่ 5.36-5.37



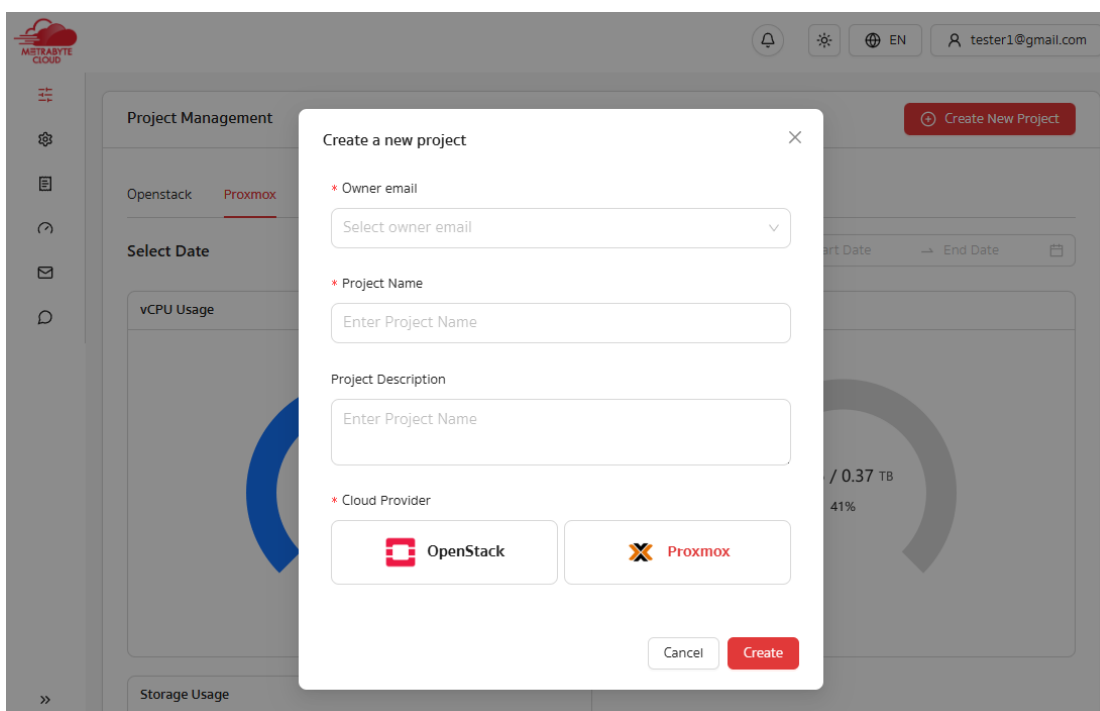
ภาพที่ 5.34 หน้าจัดการโครงการในระบบ



Openstack Project

ID	Name	Description	Owner	Action
8d43836f-b3aa-...	Billing	-	thitiphat6391@gmail.com	
44bc5ea4-07fa-...	Halo1	Haeloo	zzz@zzz.com	
d5a32951-f3db-...	Wowว่าว	not wow anymore	test5@roblox.com	
bb0dc53e-40de-...	nong suea	wow2	zzz@zzz.com	
2efb310-ce33-...	test	-	thitiphat6391@gmail.com	
28e6d7f9-c1c6-...	Hello	Hello	zzz@zzz.com	
cbaa2a80-3bf2-...	network	-	thitiphat6391@gmail.com	

ภาพที่ 5.35 ตารางโครงการในระบบ



Project Management

Openstack Proxmox

Select Date

vCPU Usage

Storage Usage

Create a new project

* Owner email

Select owner email

* Project Name

Enter Project Name

Project Description

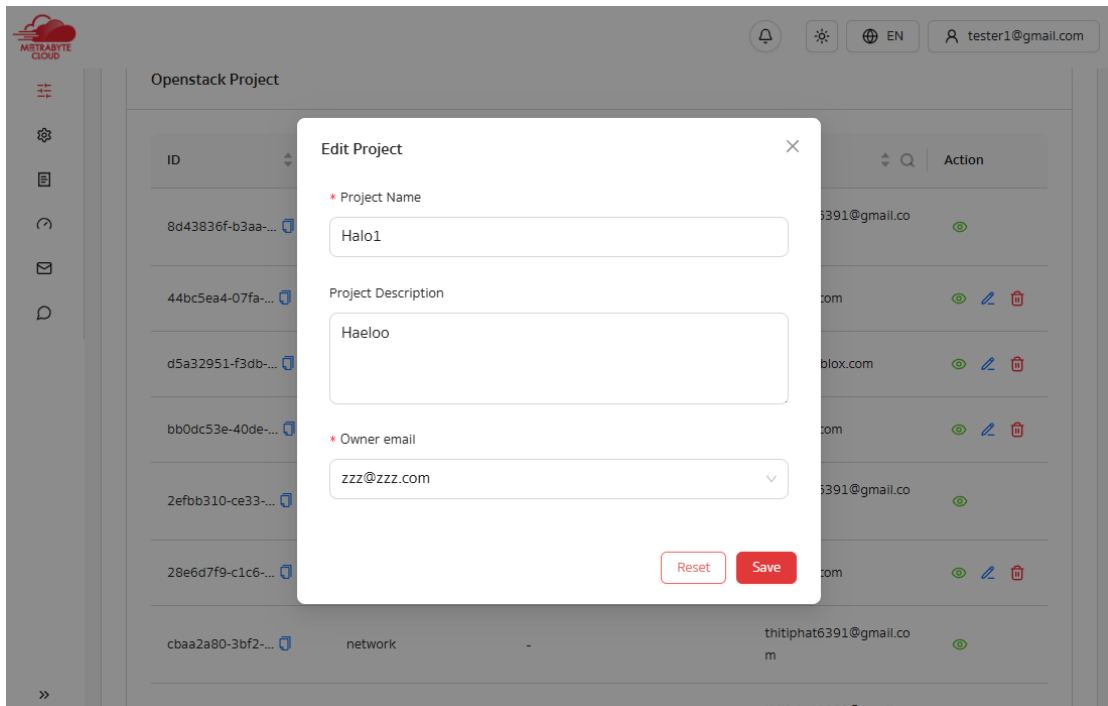
Enter Project Name

* Cloud Provider

OpenStack Proxmox

Cancel Create

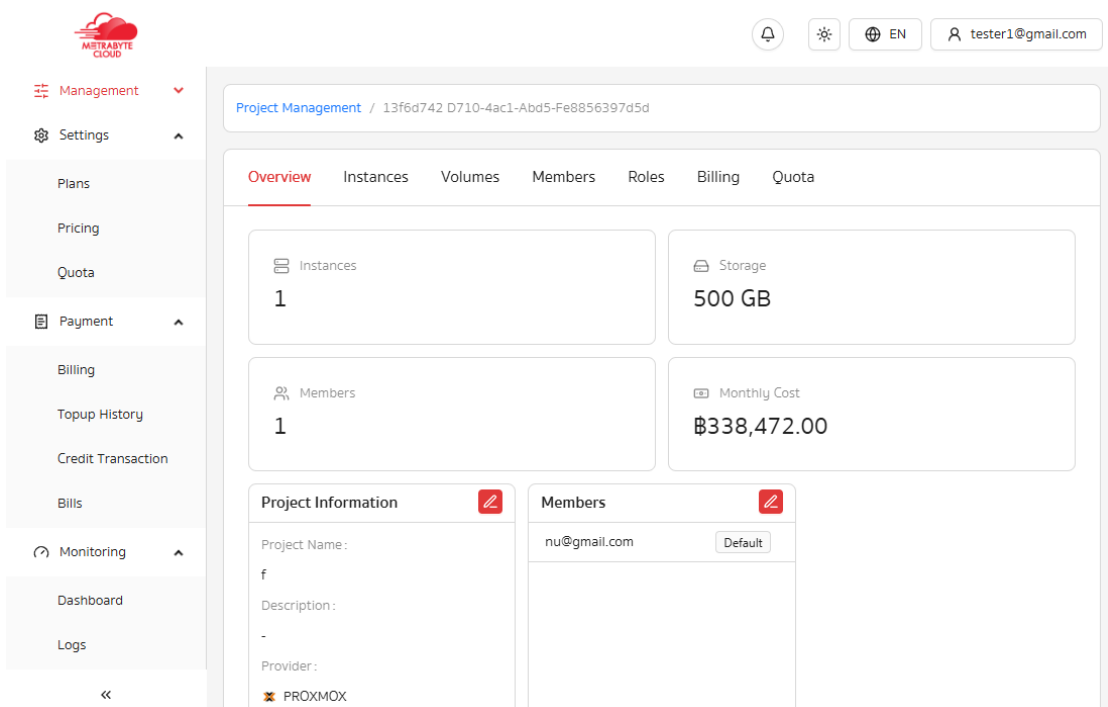
ภาพที่ 5.36 Modal สร้างโครงการให้ผู้ใช้



ภาพที่ 5.37 Modal แก้ไขโครงการของผู้ใช้

5.2.2.3 หน้าสำหรับผู้ดูแลในการจัดการโครงการของผู้ใช้

หน้าสำหรับผู้ดูแลเมื่อเข้าไปในโครงการของผู้ใช้ในระบบ โดยผู้ดูแลมีสิทธิ์สามารถเพิ่ม แก้ไข ลบได้เทียบเท่าเจ้าของโครงการดังภาพที่ 5.38-5.40 เพิ่มเติมคือผู้ดูแลระบบสามารถเพิ่มสมาชิกโครงการได้โดยไม่ต้องตอบรับคำเชิญดังภาพที่ 5.41



ภาพที่ 5.38 หน้าผู้ดูแลเข้าสู่โครงการ

Project Management / 13f6d742 D710-4ac1-Abd5-Fe8856397d5d

Overview Instances Volumes **Members** Roles Billing Quota

Member Management [Change Project Owner](#) [+ Add Member](#)

Email	Phone	Role	Invite By	Member Since
nu@gmail.com	-	Default	-	20/10/2025, 11:2

< 1 > 10 / page

ภาพที่ 5.39 หน้าจัดการสมาชิกในโครงการ

Project Management / 13f6d742 D710-4ac1-Abd5-Fe8856397d5d

Overview Instances Volumes Members **Roles** Billing Quota

Role Management

Roles

[Default](#) [+ New Role](#)

Permissions for Default

Role Name

Default

Project Permission

☒ Read ☒ Update ☒ Delete

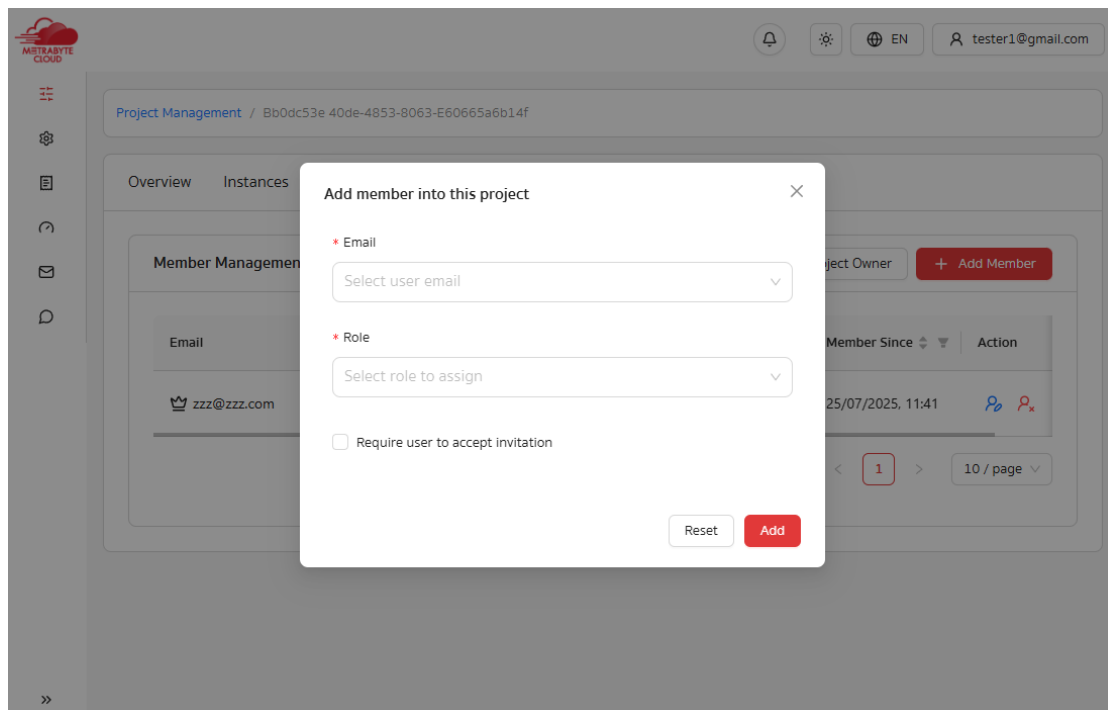
Cloud Permission

☒ Read ☒ Create ☒ Update ☒ Delete

Member Permission

☒ Read ☒ Create ☒ Update ☒ Delete

ภาพที่ 5.40 หน้าจัดการ Role ของโครงการ



ภาพที่ 5.41 Modal เพิ่มสมาชิกโครงการ

5.2.2.4 หน้าประวัติการเติมเครดิต

ภาพที่ 5.42 หน้าแสดงอันดับผู้ใช้ที่เติมมากที่สุดในระบบ สามารถเลือกช่วงเวลาที่ต้องการได้ และด้านล่างจะแสดงตารางของประวัติการเติมเครดิตทั้งหมดในระบบ

Top credit topup user

Rank	User	Amount (Baht)
1.	tester1 user	฿1,100.00
2.	Thitiphath Kunrong	฿673.00
3.	Thitiphath Kunrong	฿200.00
4.	glmv ot8iy	฿50.00

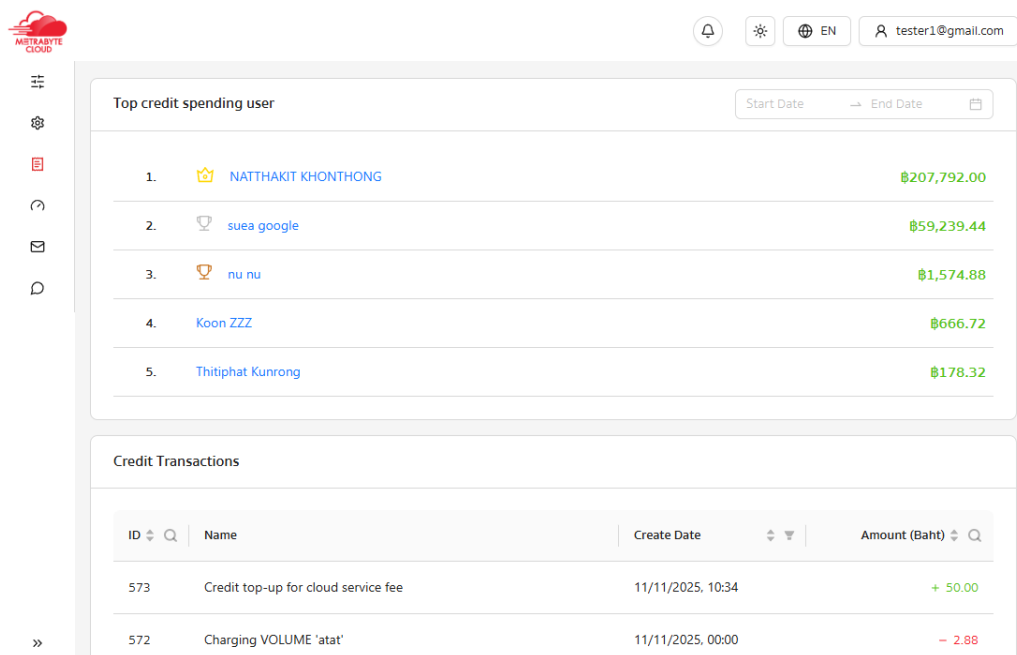
Topup History

ID	Email	Name	Method	Status	Create Date	Amount (Baht)
33	tester1@user.com	Credit Top-Up For Cloud Service Fee	Credit Card	Success	11/11/2025, 10:34	+ 50.00
32	tester1@user.com	Credit Top-Up For Cloud	PromotPay	Cancelled	11/11/2025,	+ 50.00

ภาพที่ 5.42 หน้าประวัติการเติมเครดิต

5.2.2.5 หน้าแสดงธุรกรรมเครดิต

ภาพที่ 5.43 หน้าแสดงอันดับผู้ใช้ที่ใช้จ่ายเครดิตมากที่สุดในระบบ สามารถเลือกช่วงเวลาที่ต้องการได้ และด้านล่างจะแสดงตารางธุรกรรมเครดิตทั้งหมดในระบบว่ามีการเติมเครดิต ใช้เครดิตอย่างไร



The screenshot shows the Mitrabyte Cloud interface. At the top, there's a header with the logo, a bell icon, a settings icon, a language selector set to 'EN', and a user profile icon with the email 'tester1@gmail.com'. Below the header, there's a sidebar with icons for home, menu, reports, settings, and a magnifying glass. The main content area is divided into two sections. The first section is titled 'Top credit spending user' and has a date range selector 'Start Date' to 'End Date'. It contains a table with 5 rows of users and their credit spending amounts. The second section is titled 'Credit Transactions' and contains a table with 2 rows of transactions, including ID, Name, Create Date, and Amount (Baht).

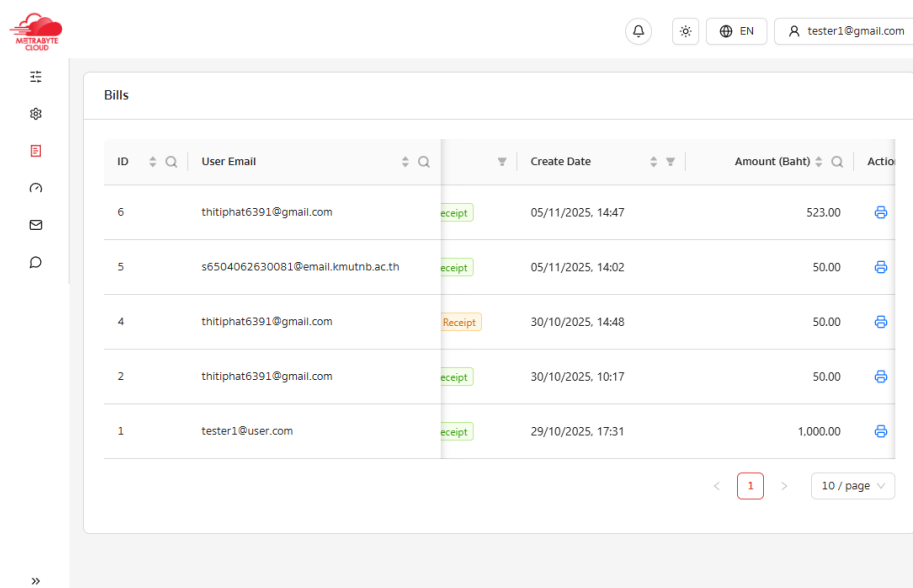
Top credit spending user		Start Date	End Date
1.	NATTHAKIT KHONTHONG		฿207,792.00
2.	suea google		฿59,239.44
3.	nu nu		฿1,574.88
4.	Koon ZZZ		฿666.72
5.	Thitiphat Kunrong		฿178.32

ID	Name	Create Date	Amount (Baht)
573	Credit top-up for cloud service fee	11/11/2025, 10:34	+ 50.00
572	Charging VOLUME 'atat'	11/11/2025, 00:00	- 2.88

ภาพที่ 5.43 หน้าธุรกรรมเครดิต

5.2.2.6 หน้าแสดงใบเสร็จต่างๆในระบบ

ภาพที่ 5.44 หน้าแสดงตารางใบเสร็จทั้งหมดที่มีการสั่งพิมพ์ในระบบ โดยผู้ดูแลสามารถสั่งพิมพ์ใบเสร็จได้เหมือนกับผู้ใช้เองเพื่อตรวจสอบความถูกต้องได้



The screenshot shows the 'Bills' section in the Mitrabyte Cloud interface. It features a table with columns for ID, User Email, Create Date, Amount (Baht), and Action. The table contains 5 rows of bills. At the bottom, there's a pagination control showing '1' out of 10 pages.

ID	User Email	Create Date	Amount (Baht)	Action
6	thitiphat6391@gmail.com	05/11/2025, 14:47	523.00	Receipt
5	s6504062630081@email.kmutnb.ac.th	05/11/2025, 14:02	50.00	Receipt
4	thitiphat6391@gmail.com	30/10/2025, 14:48	50.00	Receipt
2	thitiphat6391@gmail.com	30/10/2025, 10:17	50.00	Receipt
1	tester1@user.com	29/10/2025, 17:31	1,000.00	Receipt

ภาพที่ 5.44 หน้าดูใบเสร็จต่างๆ

บทที่ 6

บทสรุปและแนวทางในการพัฒนาต่อ

6.1 สรุปผลการดำเนินงาน

ได้พัฒนา Web Application สำหรับระบบจัดการโครงการให้สามารถใช้งานได้อย่างครบถ้วน ผู้ใช้สามารถสร้างและบริหารโครงการ รวมถึงจัดการสมาชิกภายในโครงการของ Openstack และ Proxmox บน Web Application ได้ด้วยตนเอง โดยระบบรองรับการสร้าง Role เพื่อกำหนดสิทธิ์การเข้าถึงของสมาชิกแต่ละคน และสามารถตรวจสอบค่าใช้จ่ายของโครงการได้อย่างเป็นระบบ

ในส่วนของการคิดค่าบริการ ระบบได้เชื่อมต่อเข้ากับ Omise ในการชำระเงินเพื่อให้ผู้ใช้สามารถเติมเครดิตและนำไปใช้เป็นค่าใช้จ่ายภายในระบบ โดยจะเก็บค่าบริการตามช่วงเวลาที่กำหนด และดำเนินการหักเครดิตโดยอัตโนมัติ พร้อมแจ้งเตือนเมื่อเครดิตใกล้หมด เพื่อให้ผู้ใช้สามารถเติมเครดิตได้อย่างเหมาะสมและต่อเนื่อง

สำหรับผู้ดูแลระบบ ระบบมีส่วนกลางสำหรับตรวจสอบและจัดการโครงการของผู้ใช้ทั้งหมด พร้อมทั้งมีระบบแสดงข้อมูลค่าใช้จ่ายในรูปแบบกราฟ โดยสามารถเลือกดูค่าใช้จ่ายเฉพาะบริการ เช่น OpenStack หรือ Proxmox หรือดูค่าใช้จ่ายรวมทั้งหมดของระบบได้ นอกจากนี้ยังสามารถสรุปค่าใช้จ่ายตามช่วงเวลาที่ต้องการ เช่น รายวัน รายเดือน หรือรายปี เพื่อช่วยให้ผู้ดูแลระบบสามารถวิเคราะห์ต้นทุนและการใช้งานทรัพยากรได้อย่างมีประสิทธิภาพ

6.2 ปัญหาที่พบระหว่างการดำเนินงาน

6.2.1 Omise ไม่มีตัวอย่างการใช้งานสำหรับ React และไม่มีตัวอย่างโค้ดฝั่ง Backend ด้วยภาษา Kotlin โดยตัวอย่างที่มีเป็นของ Node.js และ Java และยังเป็นเวอร์ชันที่ไม่ตรงกับปัจจุบัน

6.2.2 เบราร์เซอร์บล็อกการเปิดหน้าต่างพิมพ์ใบเสร็จ เนื่องจากการสร้างเอกสาร PDF ใช้เวลานานทำให้ระบบเปิด Popup ซ้ำจนเบราร์เซอร์มองว่าไม่ได้ถูกเปิดโดยผู้ใช้

6.2.3 ข้อมูลในฐานข้อมูลกับข้อมูลจากฝั่งผู้ให้บริการคลาวด์ Openstack หรือ Proxmox ไม่ตรงกัน ในกรณีที่เกิดปัญหาในการทำงานขึ้น

6.3 แนวทางการแก้ไขปัญหา

6.3.1 ฝั่ง Frontend ใช้วิธีโหลด omise.js ด้วย useEffect และเรียกใช้งานแบบ Manual ฝั่ง Backend นำโค้ดตัวอย่าง Java มาแปลงเป็น Kotlin

6.3.2 เปิดแท็บสำหรับพิมพ์ใบเสร็จทันทีเมื่อผู้ใช้กดพิมพ์ แล้วค่อยโหลดและแสดงข้อมูลภายหลัง เพื่อป้องกันไม่ให้เบราร์เซอร์บล็อก Popup

6.3.3 ทำการตรวจสอบความถูกต้องของฝั่งผู้ให้บริการคลาวด์ก่อนดำเนินการในฐานข้อมูล

6.4 แนวทางในการพัฒนาต่อ

6.4.1 ปรับปรุงประสิทธิภาพ และ Logic การ Query ให้ลดจำนวนครั้งการดึงข้อมูลจากฐานข้อมูลเมื่อมีการใช้ข้อมูลซ้ำ ๆ

6.4.2 เก็บข้อมูลการใช้จ่ายในโครงการให้ละเอียดยิ่งขึ้น ช่วยให้สามารถแจกแจงรายละเอียดการใช้จ่ายย้อนหลังได้มากขึ้นนอกจากตัวเลข

6.4.3 ใช้ Web socket ในการตรวจสอบสถานะการเติมเงิน แทนการเรียกซ้ำๆ ซึ่งจะช่วยให้การตอบสนองหลังจากผู้ใช้งานการชำระเงินแล้ว หรือเกิดข้อผิดพลาดใด ระบบสามารถตอบสนองได้ทันที

บรรณานุกรม

- [1] แนะนำ NodeJS คืออะไร (2565) [ออนไลน์] เข้าถึงได้จาก :
<https://www.mindphp.com/developer/dev-node-js/8542-1-step-website-nodejs.html>
 (วันที่ค้นข้อมูล 17 ตุลาคม 2568)
- [2] แนะนำ Event loop ใน Node.JS คืออะไร (2565) [ออนไลน์] เข้าถึงได้จาก :
<https://www.mindphp.com/developer/dev-node-js/8578-15-step-website-nodejs.html>
 (วันที่ค้นข้อมูล 17 ตุลาคม 2568)
- [3] ทำความรู้จักกับ React และการทำงานเบื้องต้น (2567) [ออนไลน์] เข้าถึงได้จาก :
<https://www.borntodev.com/2024/05/13/ทำความรู้จักกับ-react/> (วันที่ค้นข้อมูล 17 ตุลาคม 2568)
- [4] รู้จักกับ React เครื่องมือจำเป็นสำหรับ Front-end Developer (2568) [ออนไลน์] เข้าถึงได้จาก : <https://www.techupth.com/articles/react> (วันที่ค้นข้อมูล 17 ตุลาคม 2568)
- [5] MariaDB และ MySQL แตกต่างกันอย่างไร [ออนไลน์] เข้าถึงได้จาก :
<https://aws.amazon.com/th/compare/the-difference-between-mariadb-vs-mysql/>
 (วันที่ค้นข้อมูล 17 ตุลาคม 2568)
- [6] What Is MariaDB? [ออนไลน์] เข้าถึงได้จาก :
<https://www.purestorage.com/nl/knowledge/what-is-mariadb.html> (วันที่ค้นข้อมูล 17 ตุลาคม 2568)
- [7] รู้จัก MariaDB คืออะไร (2560) เข้าถึงได้จาก :
<https://www.thaicreate.com/tutorial/mariadb-mysql.html> (วันที่ค้นข้อมูล 17 ตุลาคม 2568)
- [8] Spring Boot Tutorial (2568) [ออนไลน์] เข้าถึงได้จาก :
<https://www.geeksforgeeks.org/advance-java/spring-boot> (วันที่ค้นข้อมูล 18 ตุลาคม 2568)
- [9] Difference between Spring and Spring Boot (2568) [ออนไลน์] เข้าถึงได้จาก :
<https://www.geeksforgeeks.org/java/difference-between-spring-and-spring-boot> (วันที่ค้นข้อมูล 18 ตุลาคม 2568)
- [10] Pros and Cons of Using Spring Boot (2564) [ออนไลน์] เข้าถึงได้จาก :
<https://bambooagile.eu/insights/pros-and-cons-of-using-spring-boot> (วันที่ค้นข้อมูล 18 ตุลาคม 2568)

- [11] Omise Payment Gateway (โอมิเซ เพย์เม้น เกตเวย์) คืออะไร (2561) [ออนไลน์] เข้าถึงได้จาก : <https://www.mindphp.com/บทความ/157-e-commerce/4823-what-is-omise.html> (วันที่ค้นข้อมูล 18 ตุลาคม 2568)
- [12] Omise Step by Step Guide (2562) [ออนไลน์] เข้าถึงได้จาก : <https://docs.omise.co/step-by-step-guide> (วันที่ค้นข้อมูล 10 มิถุนายน 2568)
- [13] สร้างระบบชำระเงินบนเว็บไซต์ของเราด้วย Omise (2565) [ออนไลน์] เข้าถึงได้จาก : <https://www.borntodev.com/2022/02/14/สร้างระบบชำระเงินด้วย-omise/> (วันที่ค้นข้อมูล 10 มิถุนายน 2568)
- [14] ใช้งาน Git & GitHub สำหรับผู้เริ่มต้น ในปี 2024 (2024) [ออนไลน์] เข้าถึงได้จาก : <https://devhub.in.th/blog/git-github> (วันที่ค้นข้อมูล 18 ตุลาคม 2568)
- [15] Openstack Keystone API (2568) [ออนไลน์] เข้าถึงได้จาก : <https://docs.openstack.org/api-ref/identity/v3/> (วันที่ค้นข้อมูล 18 ตุลาคม 2568)
- [16] Ant Design of React [ออนไลน์] เข้าถึงได้จาก : <https://ant.design/docs/react/introduce> (วันที่ค้นข้อมูล 18 ตุลาคม 2568)
- [17] Echarts [ออนไลน์] เข้าถึงได้จาก : <https://echarts.apache.org/en/api.html#echarts> (วันที่ค้นข้อมูล 1 กันยายน 2568)
- [18] 60 essential Linux commands (2568) [ออนไลน์] เข้าถึงได้จาก : <https://www.hostinger.com/tutorials/linux-commands> (วันที่ค้นข้อมูล 8 มิถุนายน 2568)
- [19] Configure a Gradle project (2568) [ออนไลน์] เข้าถึงได้จาก : <https://kotlinlang.org/docs/gradle-configure-project.html> (วันที่ค้นข้อมูล 1 มิถุนายน 2568)
- [20] Node.js process.config Property (2568) [ออนไลน์] เข้าถึงได้จาก : <https://www.geeksforgeeks.org/node-js/node-js-process-config-property> (วันที่ค้นข้อมูล 1 มิถุนายน 2568)